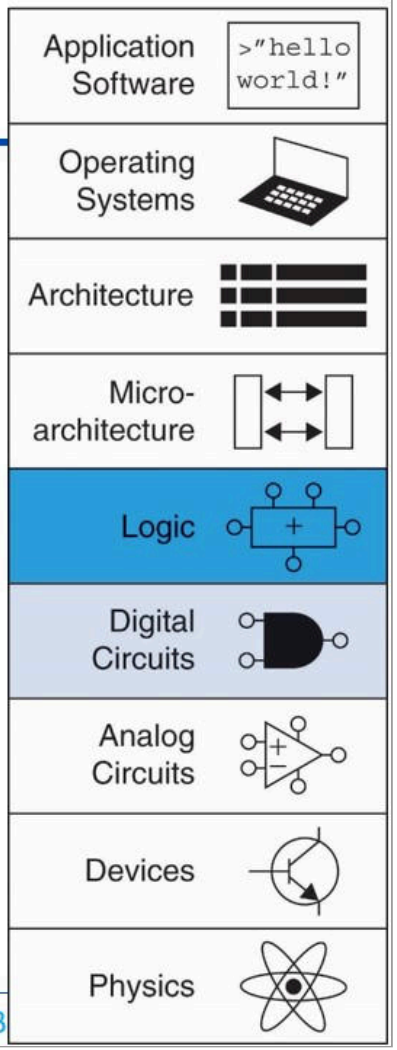
Chapter 5 Digital Building Blocks

Chapter 5 :: Topics

- 引言
- 算术运算电路
- 数制系统（重点介绍浮点数）
- 时序电路模块
- 存储器阵列

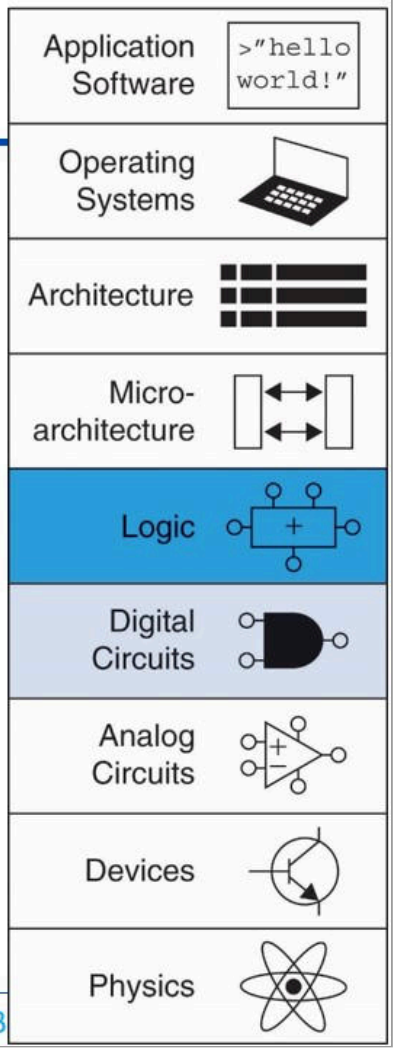


Introduction

- 数字系统中常见的**组合电路模块**和**时序电路模块**:
  - 逻辑门, 多路选择器, 译码器, 寄存器, 算术运算电路, 计数器, 存储阵列, 逻辑阵列
- 这些模块有很重要的作用，也说明了**层次化**，**模块化**和**规整化**的原则:
  - 复杂模块可以用层次化的方法由更简单的模块组成
  - 每个模块都有定义好的接口
  - 每一个规整结构的模块都非常容易扩展为更大的模块
- 在第7章会介绍使用上述这些模块**构成一个微处理器**

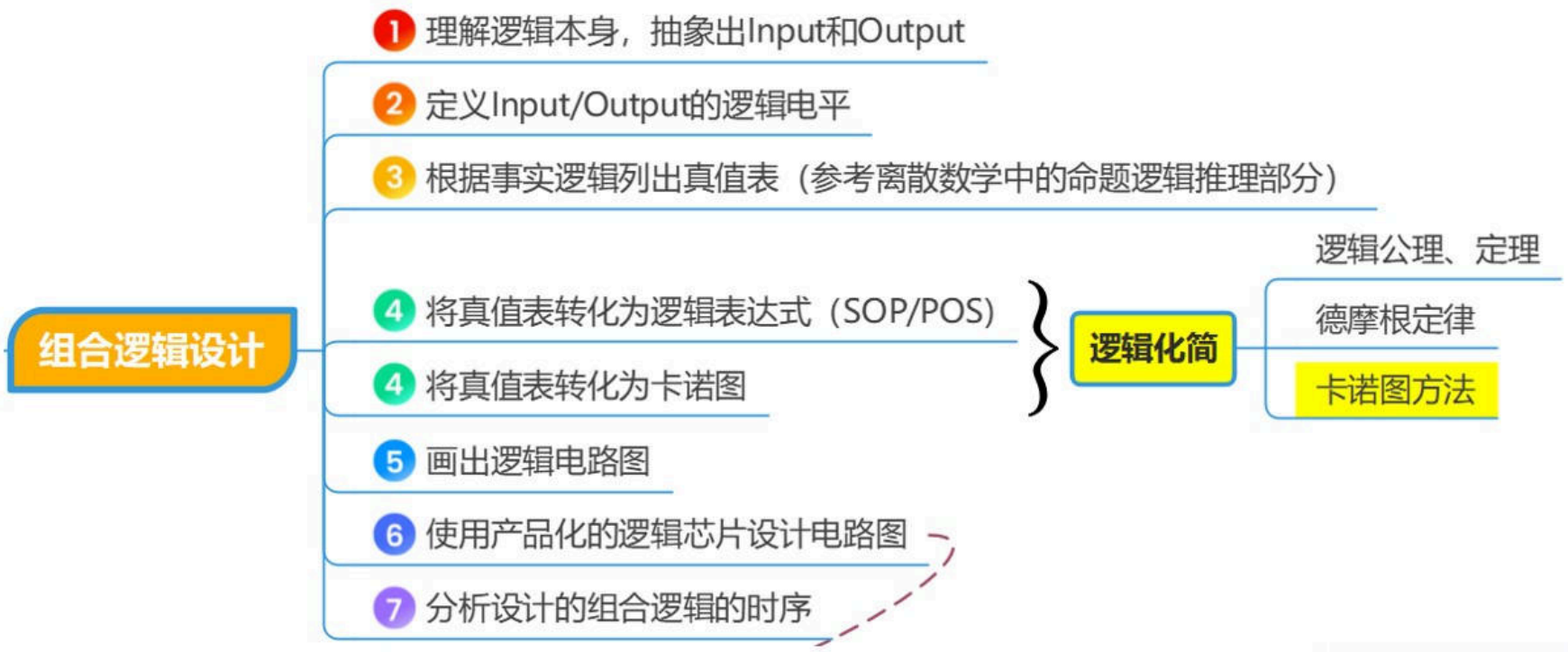
Chapter 5 :: Topics

- 引言
- **算术运算电路**
  - **多位加法器（1位→多位→带溢出判断→减法器→加减法器）**
  - **算术逻辑单元 ALU**
  - **更快速的多位加法器（行波进位→先行进位→前缀进位）**
  - **移位器→乘法器/除法器**
- 数制系统（重点介绍浮点数）
- 时序电路模块
- 存储器阵列



模块化方法设计一个 4-bit 加法器

- 回忆组合逻辑设计方法



雨课堂 Rain Classroom

模块化方法设计一个 4-bit 加法器

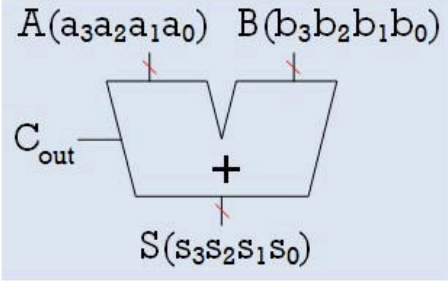
1 理解逻辑本身，抽象出Input和Output

① 理解逻辑本身，加法器运算规则

- 0000+0000 = 0 0000
- 0000+0001 = 0 0001
- ..... + ... = ...
- 0001+0111 = 0 1000 ---> 1+7 = 8 or -1 ? **是溢出吗？什么是溢出?**
- ..... + ... = ...
- 0001+1111 = 1 0000 ---> 1+(-1) = 0 or 1+15=16 ? **C_{out} 是进位吗** ?
- ..... + ... = ...

4-bit 加法器需要分析多少行的运算规则?

$$\begin{array}{rcccc} & a_3 & a_2 & a_1 & a_0 \\ + & b_3 & b_2 & b_1 & b_0 \\ \hline & s_3 & s_2 & s_1 & s_0 \end{array}$$

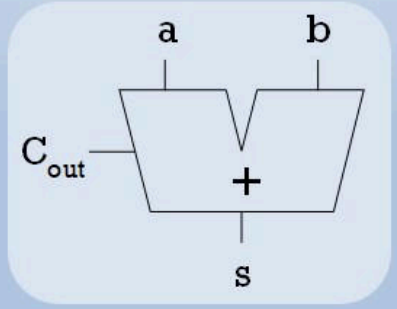


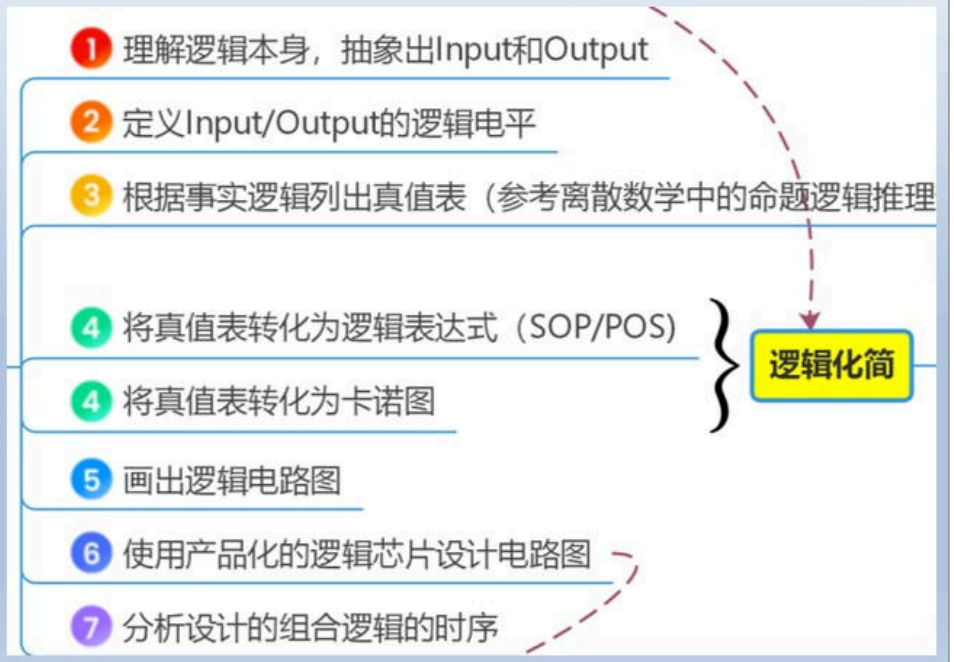
回忆第2章的组合逻辑设计 示例1

- 1位加法器（1-Bit Adders）

① 理解逻辑本身，加法器运算规则

- 0+0=0
- 0+1=1
- 1+0=1
- 1+1=10





Input = *a*, *b*（每个输入都是1-Bit信号）

Output = *s*, C_{out}（每个输出也都是1-Bit信号）

雨课堂 Rain Classroom

回忆第2章的组合逻辑设计 示例1

- 1位加法器（1-Bit Adders）

② 定义输入/输出的逻辑电平

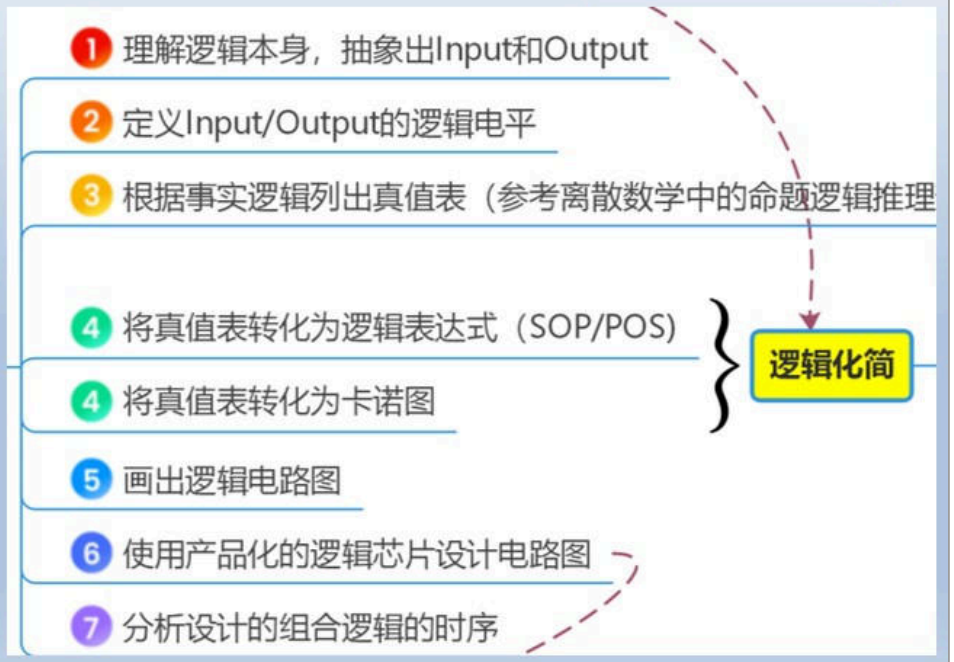
Input:

a, *b* --> 0(Gnd), 1(Vdd)

Output:

sum --> 0(Gnd), 1(Vdd)

Carry --> Yes(Vdd), No(Gnd)



雨课堂 Rain Classroom

回忆第2章的组合逻辑设计 示例1

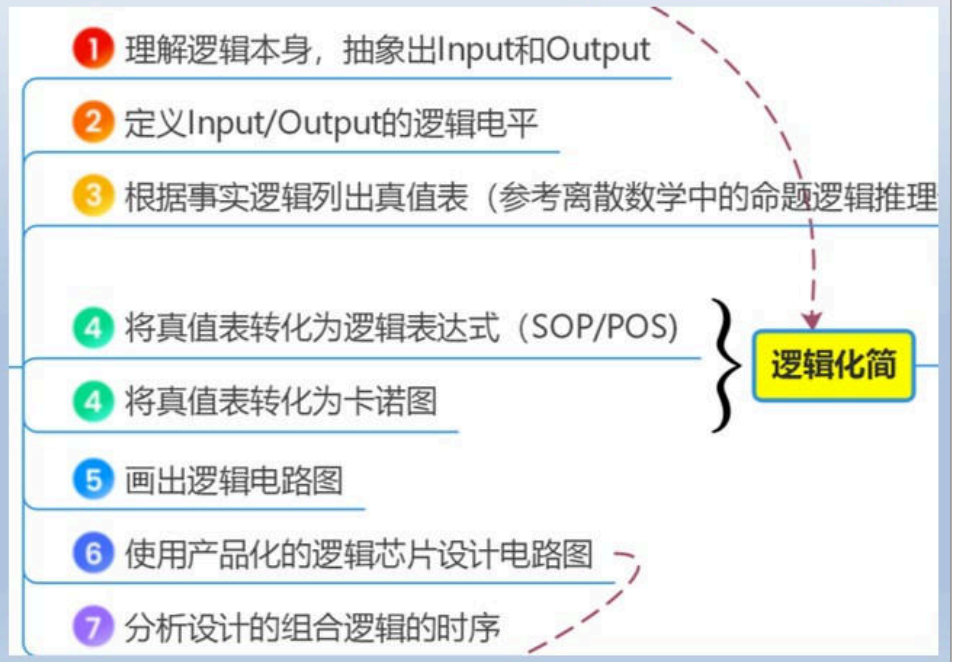
- 1位加法器（1-Bit Adders）

③ 列出真值表

a	b	s	C_{out}
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

④ 写出逻辑表达式

$$s = a'b + ab' = a \oplus b, \quad C_{out} = ab$$



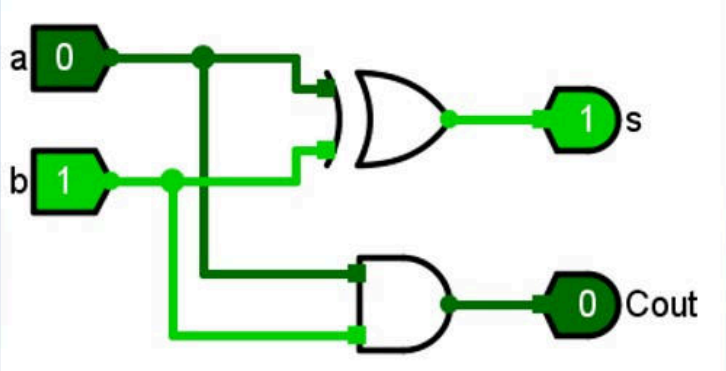
回忆第2章的组合逻辑设计 示例1

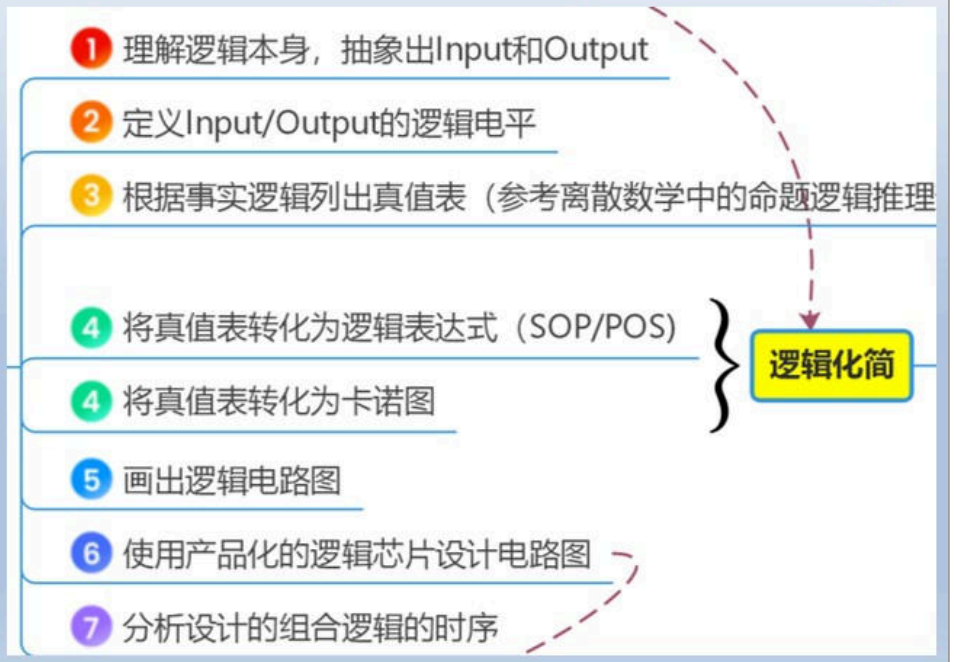
- 1位加法器（1-Bit Adders）

④ 写出逻辑表达式

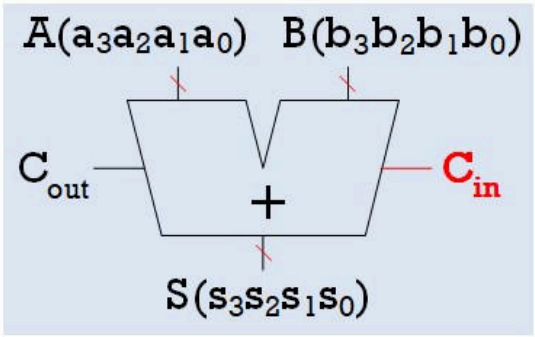
$$s = a'b + ab' = a \oplus b, \quad C_{out} = ab$$

⑤ 画出逻辑电路图



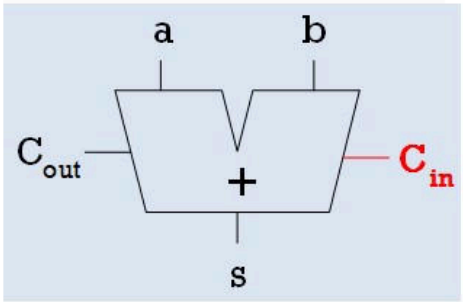


模块化方法设计一个 4-bit 加法器

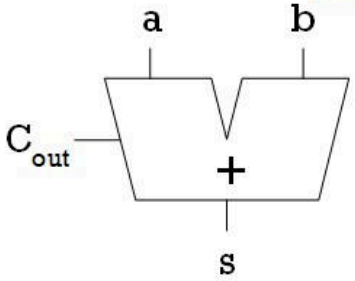


$$\begin{array}{r} a_3\ a_2\ a_1\ a_0 \\ +\ b_3\ b_2\ b_1\ b_0 \\ \hline s_3\ s_2\ s_1\ s_0 \end{array}$$

1-bit **half**-adder



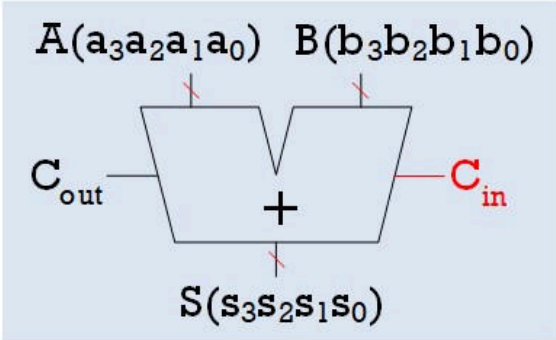
Full-adder

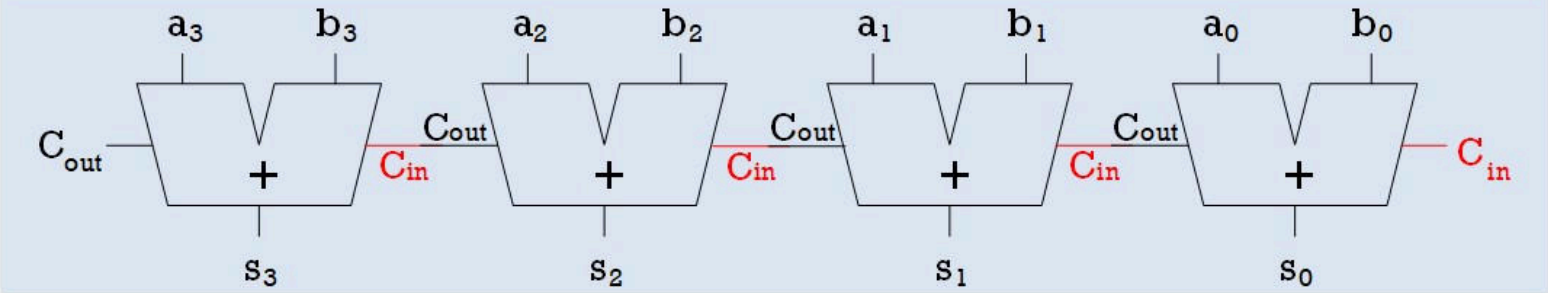


half-adder

模块化方法设计一个 4-bit 加法器

$$
\begin{array}{rcccc}
 & a_3 & a_2 & a_1 & a_0 \\
+ & b_3 & b_2 & b_1 & b_0 \\
\hline
 & s_3 & s_2 & s_1 & s_0
\end{array}
$$

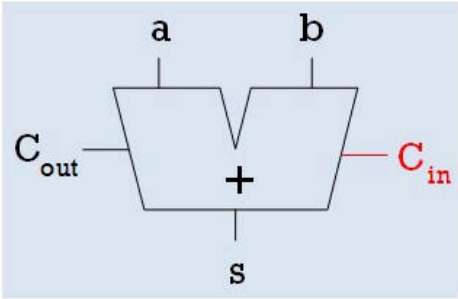




设计 1-Bit 全加器

1. 理解逻辑本身，抽象出Input和Output

- 加法器运算规则
  - 0+0+0=0
  - 0+0+1=1
  - 0+1+0=1
  - ...+...+... = ...



Input = *a*, *b*, C_{in}（每个输入都是1-Bit信号）

Output = *s*, C_{out}（每个输出也都是1-Bit信号）

设计 1-Bit 全加器

2 定义Input/Output的逻辑电平

Input:

a, b, C_{in} --> 0(Gnd), 1(Vdd)

Output:

sum --> 0(Gnd), 1(Vdd)

C_{out} --> Yes(Vdd), No(Gnd)

雨课堂 Rain Classroom

设计 1-Bit 全加器

3 根据事实逻辑列出真值表（参考离散数学中的命题逻辑推理部分）

a	b	C_{in}	s	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

4 将真值表转化为逻辑表达式（SOP/POS）

$$s = a'b'C_{in} + a'bC_{in}^{'} + ab'C_{in}^{'} + abC_{in}$$
$$= a \oplus b \oplus C_{in}$$

$$C_{out} = a'bC_{in} + ab'C_{in} + abC_{in}^{'} + abC_{in}$$
$$= ab + aC_{in} + bC_{in}$$

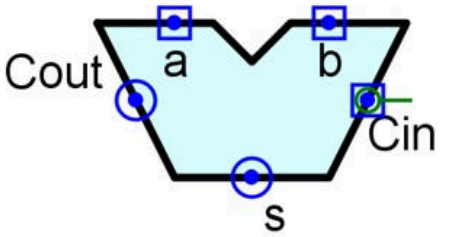
设计 1-Bit 全加器

4 将真值表转化为逻辑表达式（SOP/POS）

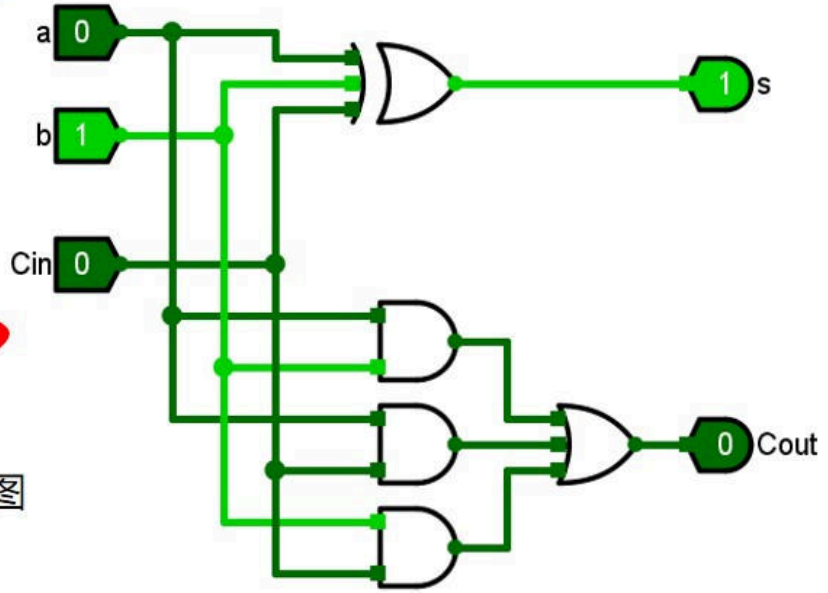
$$s = a \oplus b \oplus C_{in}$$

$$C_{out} = ab + aC_{in} + bC_{in}$$

5 画出逻辑电路图

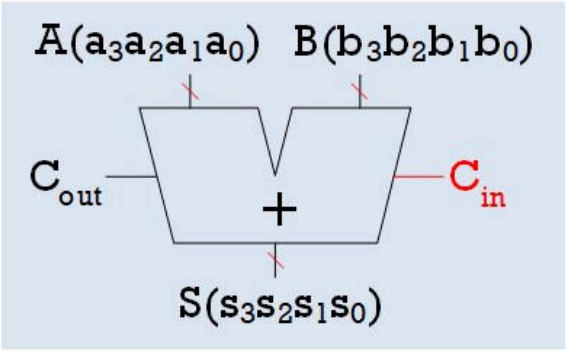


封装功能图

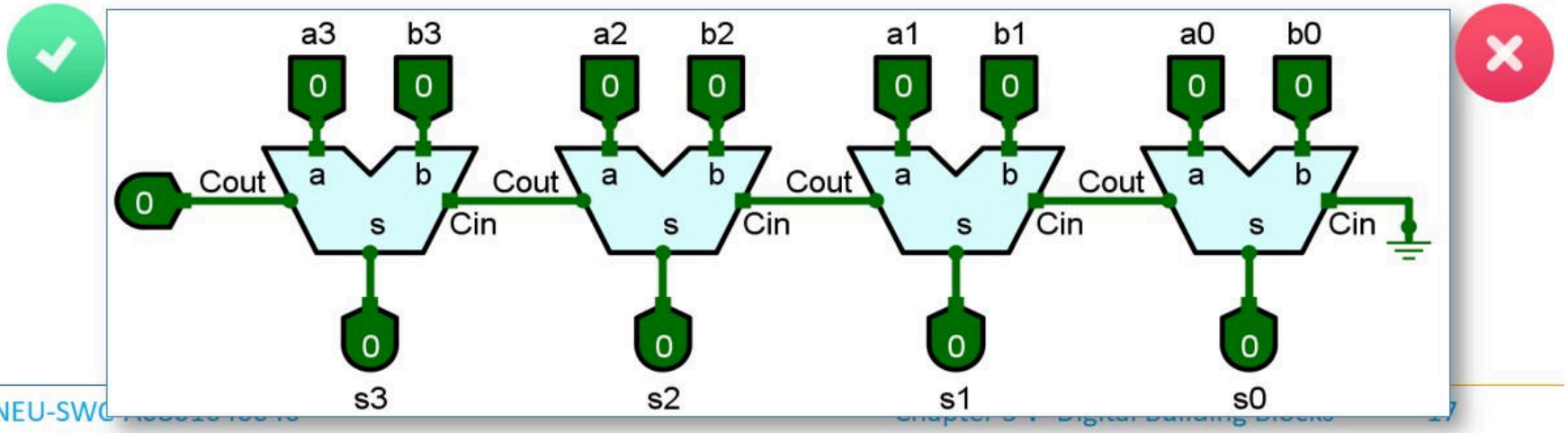


模块化方法设计一个 4-bit 加法器

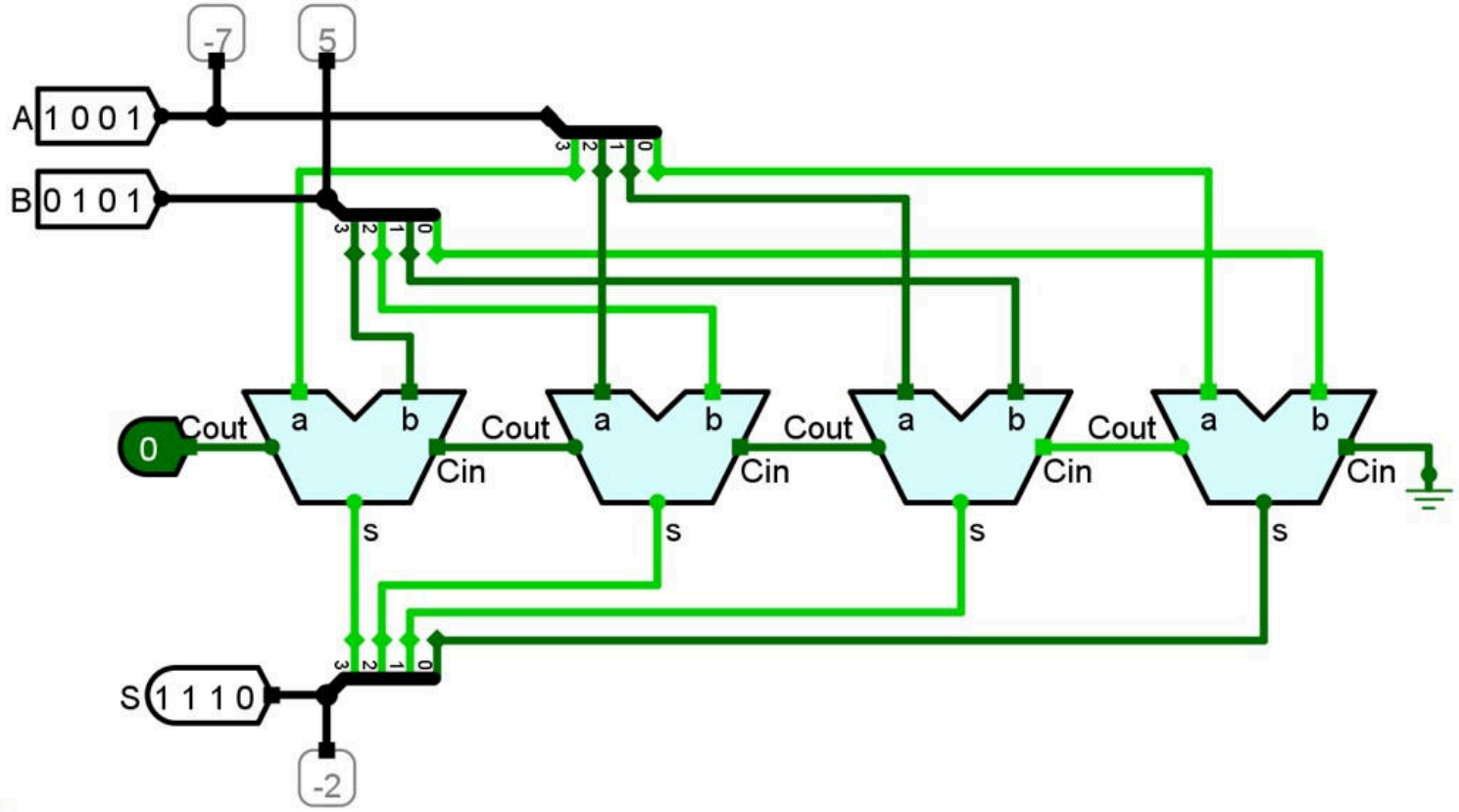
$$\begin{array}{r} a_3\ a_2\ a_1\ a_0 \\ +\ b_3\ b_2\ b_1\ b_0 \\ \hline s_3\ s_2\ s_1\ s_0 \end{array}$$



➤ 设计的加法器正确吗?

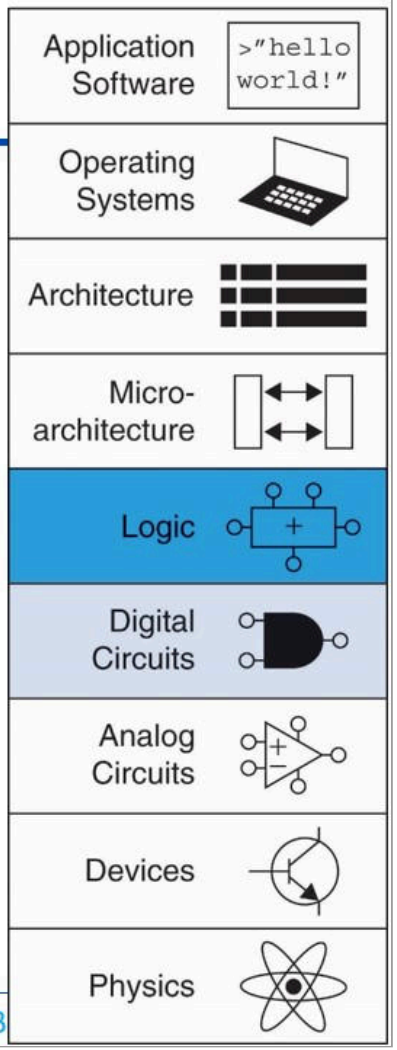


Logisim仿真实现4-bit加法器



Chapter 5 :: Topics

- 引言
- **算术运算电路**
  - **多位加法器（1位→多位→带溢出判断→减法器→加减法器）**
  - **算术逻辑单元 ALU**
  - **更快速的多位加法器（行波进位→先行进位→前缀进位）**
  - **移位器→乘法器/除法器**
- 数制系统（重点介绍浮点数）
- 时序电路模块
- 存储器阵列



溢出 Overflow

- 数字系统常常对固定长度的数字进行操作
- 如果加法的结果太大，超出了数字表示的范围，就产生了**溢出**
- 例如，1011_2+0110_2
  - 如果，将两个操作数看成 4-bits 无符号数据: 11 + 6 =17

```
  111
   1011
+  0110
------
  10001
```

  - 如果，将两个操作数看成4-bits 有符号数据（补码表示）：-5 + 6=1

雨课堂 Rain Classroom

溢出判断

- **什么情况产生溢出?**
  - 运算结果超出表示范围，一定产生溢出。
  - **对于无符号数来说**，只要产生进位就是溢出。
  - **对于有符号数来说**，显然：
    - 两个不同符号的数相加，或者两个相同符号的数相减，一定不会导致溢出。
    - 两个同符号数相加或者两个异符号数相减才有可能产生溢出。
- 溢出是错误，必须要被识别出来。

溢出问题(2038年问题)

- “千年虫”解决后，“2038年”提出新的挑战。
- 大多数C语言程序都使用到一个叫做“标准时间库”的程序库，这个时间库用一个标准的4字节也就是32位的形式来储存时间信息。
- 32位存储空间表示有符号数时最大值是2147483647，从1970年1月1日0点开始计时，于2038年1月19日凌晨03:14:07达到最大值，在那惊心动魄的最后一秒后，会发生什么?

```
Binary  : 01111111 11111111 11111111 11110000
Decimal : 2147483632
Date    : 2038-01-19 03:13:52 (UTC)
Date    : 2038-01-19 03:13:52 (UTC)
```

补码表示的有符号数运算结果的溢出判断

- 既然减法运算可以转换成加法，我们就**只讨论**如何判断**加法运算结果**是否**溢出**.
- **符号定义**:
  - 假设两个 *N*-bits 补码表示的有符号数

$$[X]_{补码} = S_X X_{n-2} \ldots X_1 X_0$$

$$[Y]_{补码} = S_Y Y_{n-2} \ldots Y_1 Y_0$$

  - *N*-bits 补码表示的运算结果

$$[F]_{补码} = S_F F_{n-2} \ldots F_1 F_0$$

$$\begin{array}{cccccc} & S_X & X_3 & X_2 & X_1 & X_0 \\ C_F & S_{YC} & Y_3 & Y_2 & Y_1 & Y_0 \\ \hline & S_F & F_3 & F_2 & F_1 & F_0 \end{array}$$

雨课堂 Rain Classroom

补码表示的有符号数运算示例

(1) $X=3$ $Y=2$

3+2:

```
  0 0011
+ 0 0010
---------
  0 0101
```

correct

(2) $X=10$ $Y=7$

10+7:

```
  0 1010
+ 0 0111
---------
  1 0001
```

overflow

(3) $X=-3$ $Y=-2$

-3+(-2):

```
  1 1101
+ 1 1110
---------
  1 1011
```

correct

(4) $X=-10$ $Y=-7$

-10+(-7):

```
  1 0110
+ 1 1001
---------
  0 1111
```

underflow

(5) $X=6$ $Y=-4$

6+(-4):

```
  0 0110
+ 1 1100
---------
  0 0010
```

correct

(6) $X=-6$ $Y=4$

-6+4:

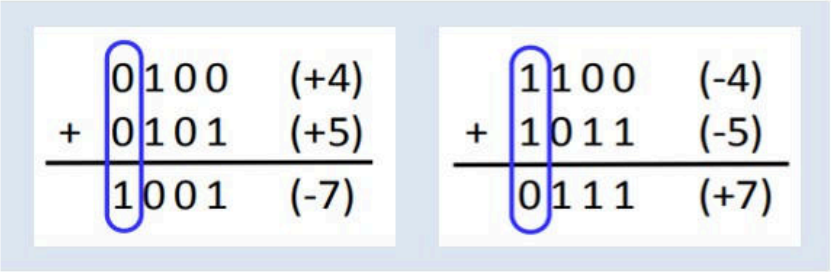
```
  1 1010
+ 0 0100
---------
  1 1110
```

correct

补码表示的有符号数运算结果的溢出判断

① 硬件逻辑判断方法1：使用两个操作数的符号S_X, S_Y和结果符号S_F

S_X	S_Y	S_F	Overflow
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0



$$Overflow = \bar{S}_X \bar{S}_Y S_F + S_X S_Y \bar{S}_F$$

雨课堂 Rain Classroom

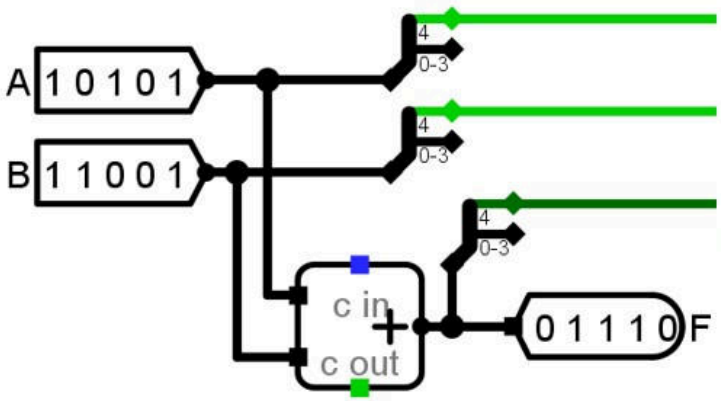
补码表示的有符号数运算结果的溢出判断

① 硬件逻辑判断方法1：使用两个操作数的符号S_X, S_Y和结果符号S_F

$$Overflow = \overline{S}_X \overline{S}_Y S_F + S_X S_Y \overline{S}_F$$

	0100	(+4)
+	0101	(+5)
	1001	(-7)

	1100	(-4)
+	1011	(-5)
	0111	(+7)



补码表示的有符号数运算结果的溢出判断

(1) $X=3$ $Y=2$

3+2:

Cf=0
C =0

correct

```
  0 0011
+ 0 0010
---------
  0 0101
```

(2) $X=10$ $Y=7$

10+7:

Cf=0
C =1

overflow

```
  0 1010
+ 0 0111
---------
  1 0001
```

(3) $X=-3$ $Y=-2$

-3+(-2):

Cf=1
C =1

correct

```
  1 1101
+ 1 1 1110
-----------
  1 1011
```

(4) $X=-10$ $Y=-7$

-10+(-7):

Cf=1
C =0

underflow

```
  1 0110
+ 1 1 1001
-----------
  0 1111
```

(5) $X=6$ $Y=-4$

6+(-4):

Cf=1
C =1

correct

```
  0 0110
+ 1 1 1100
-----------
  0 0010
```

(6) $X=-6$ $Y=4$

-6+4:

Cf=0
C =0

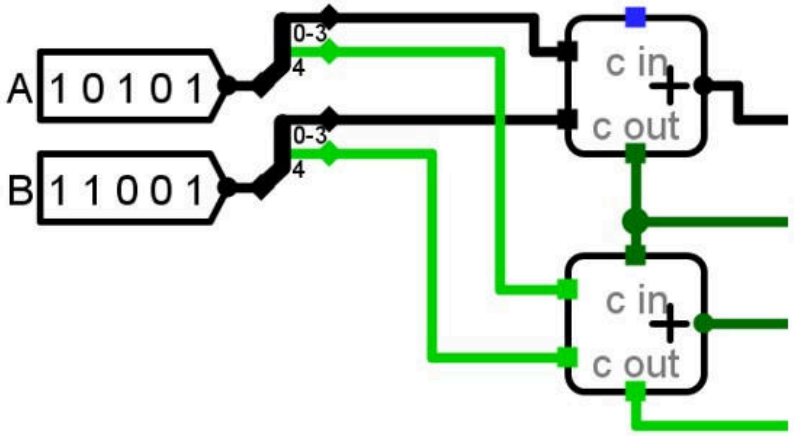
correct

```
  1 1010
+ 0 0100
---------
  1 1110
```

补码表示的有符号数运算结果的溢出判断

② 硬件逻辑判断方法2：使用数值位进位信号C和符号位进位信号C_F

$$Overflow = C_F \oplus C$$



补码表示的有符号数运算结果的溢出判断

(1) $X=3$ $Y=2$

3+2:

```
  00 0011
+ 00 0010
----------
  00 0101
```

correct

First Sf1

Second Sf2

(2) $X=10$ $Y=7$

10+7:

```
  00 1010
+ 00 0111
----------
  01 0001
```

overflow

(3) $X=-3$ $Y=-2$

-3+(-2):

```
  11 1101
+ 11 1110
----------
  11 1011
```

correct

(4) $X=-10$ $Y=-7$

-10+(-7):

```
  11 0110
+ 11 1001
----------
  10 1111
```

underflow

(5) $X=6$ $Y=-4$

6+(-4):

```
  00 0110
+ 11 1100
----------
  00 0010
```

correct

(6) $X=-6$ $Y=4$

-6+4:

```
  11 1010
+ 00 0100
----------
  11 1110
```

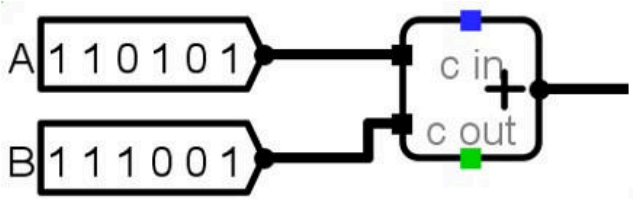
correct

雨课堂 Rain Classroom

补码表示的有符号数运算结果的溢出判断

③ 硬件逻辑判断方法2：使用运算结果的双符号位S_{F1}和 S_{F2}

$$Overflow = S_{F1} \oplus S_{F2}$$



补码表示的有符号数运算结果的溢出判断小结

① 硬件逻辑判断方法1：使用两个操作数的符号S_X, S_Y和结果符号S_F

$$Overflow = \overline{S}_X \overline{S}_Y S_F + S_X S_Y \overline{S}_F$$

② 硬件逻辑判断方法2：使用数值位进位信号C和符号位进位信号C_F

$$Overflow = C_F \oplus C$$

③ 硬件逻辑判断方法2：使用运算结果的双符号位S_{F1}和 S_{F2}

$$Overflow = S_{F1} \oplus S_{F2}$$

雨课堂 Rain Classroom

课后作业3

- 请同学们使用logisim设计仿真
  - 带有溢出判断和进位输出的4-bit无符号数加法器
  - 带有溢出判断和进位输出的4-bit**有符号数**的**补码**加法器

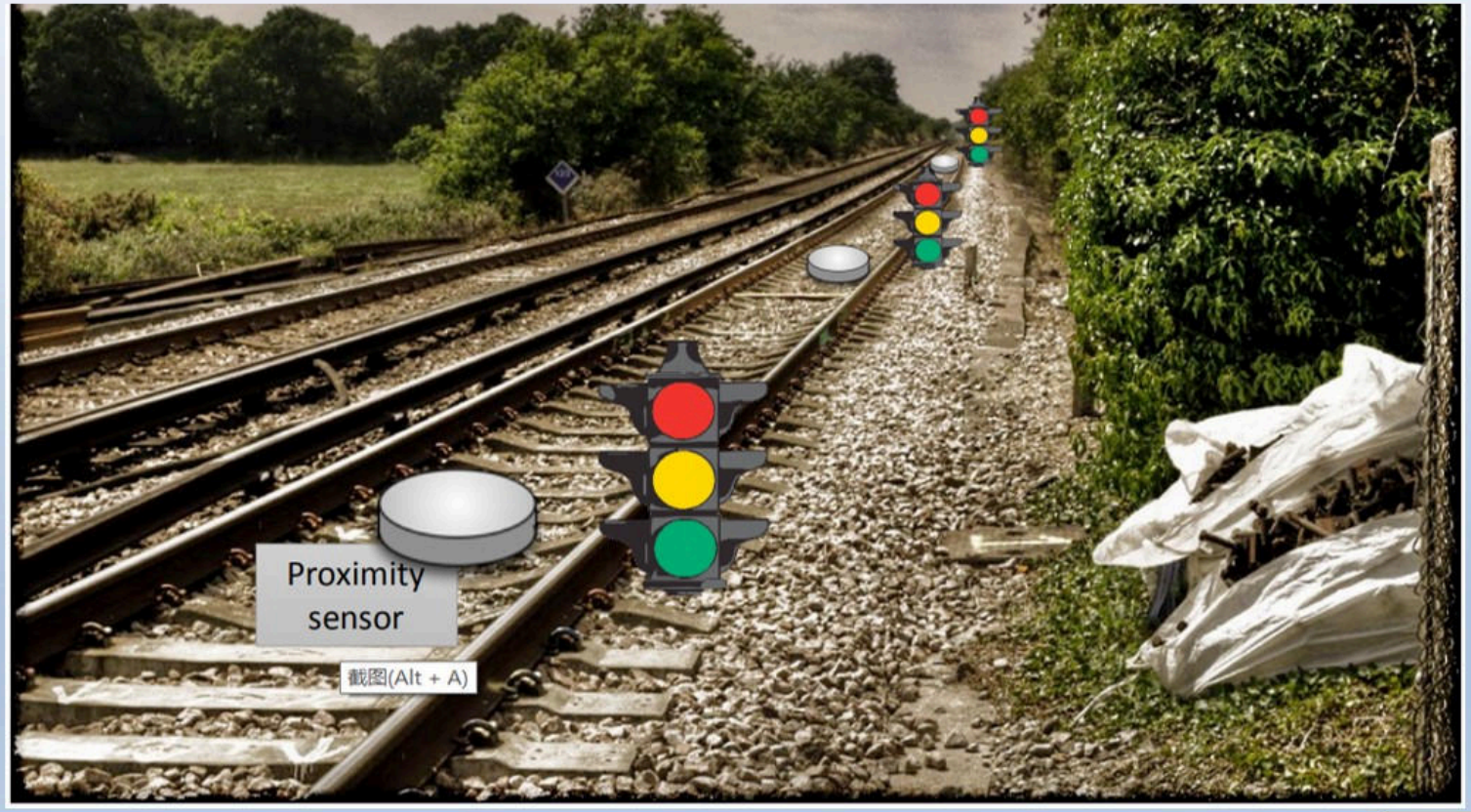
- 注意：请同学们以后所有的logisim上的仿真实验作业，都集成在一个电路文件中，文件命名规则：班级-学号-姓名-计算机组成原理.circ
- 每个电路的命名规则，参考第二章 课件。

课后讨论：模块化设计

- 构建一个系统，以帮助避免列车碰撞。
  - 假设共28段轨道，每段轨道：
    - 有传感器，以检测其上是否有列车。有车 (Train = 1)，没车(Train = 0)
    - 设有一个红/黄/绿三色信号灯，某时某刻只有一盏灯亮起
      - 若下一段轨道上有列车，则红灯亮起（R = 1）
      - 否则，若列车在相隔两段轨道处，则黄灯亮起（Y = 1）
      - 否则，绿灯亮起（G = 1）

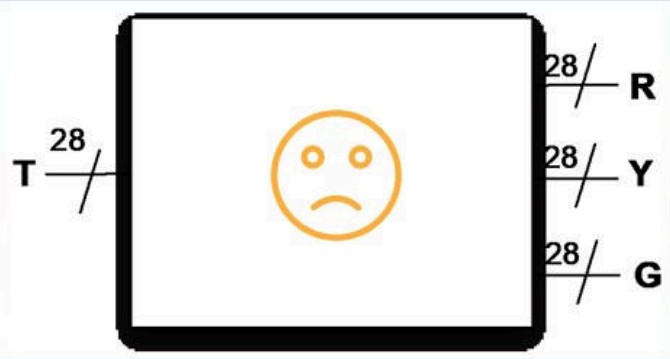
雨课堂 Rain Classroom

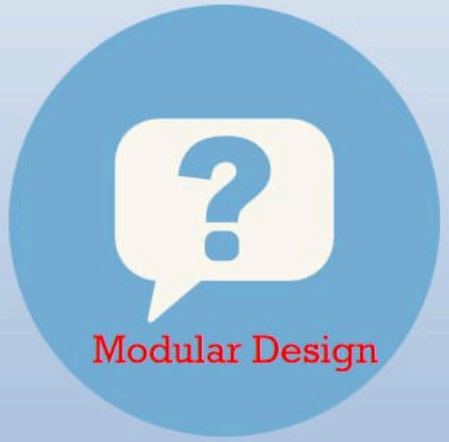
避免列车碰撞系统



模块化设计（讨论题）

- 如果设计一个“大”电路以实现上述功能





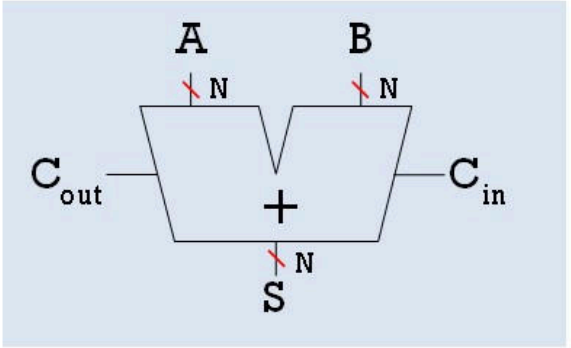
课后讨论：避免列车碰撞系统

- 请同学们以小组为单位讨论这个系统的实现
- 要求：
  - 分析使用“大”电路实现的可能性
  - 讨论使用模块化方法设计的可能性
  - 设计系统，并**使用logisim仿真（每名同学都添加到需要提交的电路文件中）**
  - 设计系统的时候考虑真正工程实施的成本、部署的方便性等。

雨课堂 Rain Classroom

多位加法器 (进位传播加法器CPAs)

- 三种进位传播加法器(carry propagate adders，CPAs):
  - 行波进位加法器 Ripple-carry (slow)
  - 先行进位加法器 Carry-lookahead (fast)
  - 前缀加法器 Prefix (faster)

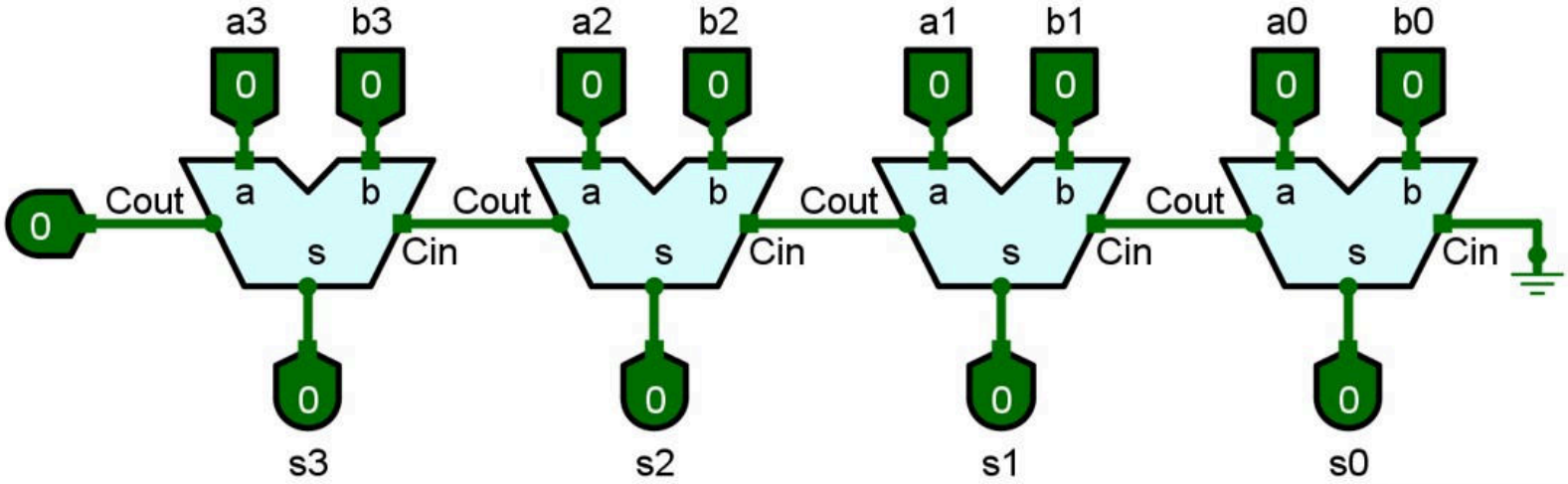


- 先行进位加法器和前缀加法器比行波进位加法器快，但是需要使用更多的硬件。

多位加法器

- 我们可以使用层次化的方法来构建多位加法器
  - 例如: 4-bit 加法器

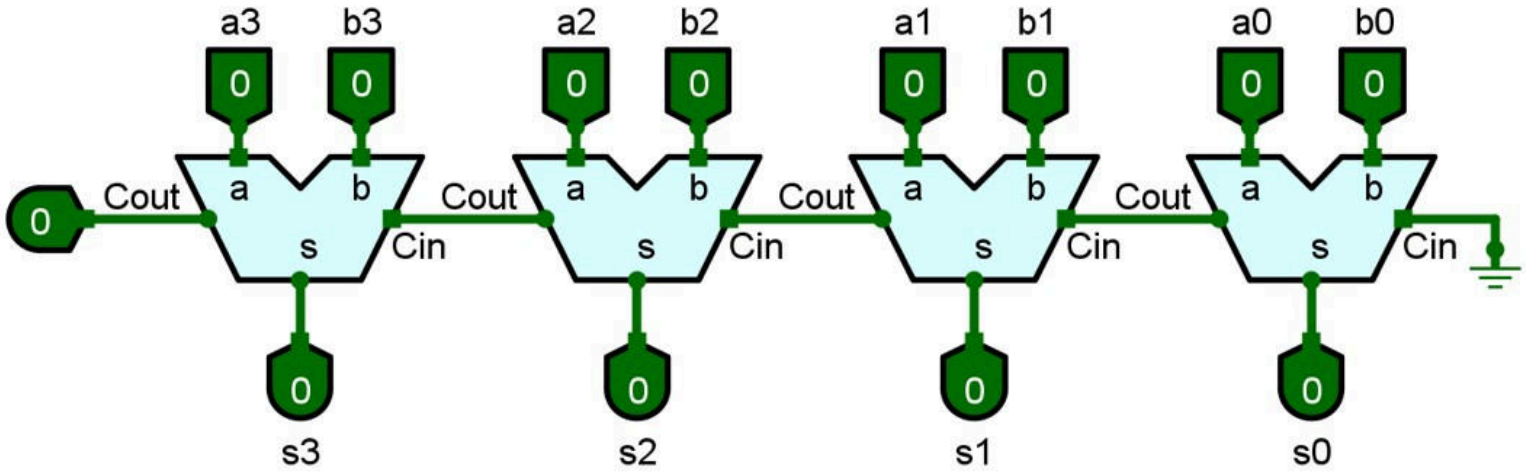
Cout	C_3	C_2	C_1	C_{in}	进 位
	a_3	a_2	a_1	a_0	被加数
+	b_3	b_2	b_1	b_0	加 数
	s_3	s_2	s_1	s_0	和



多位加法器

慕课堂练习

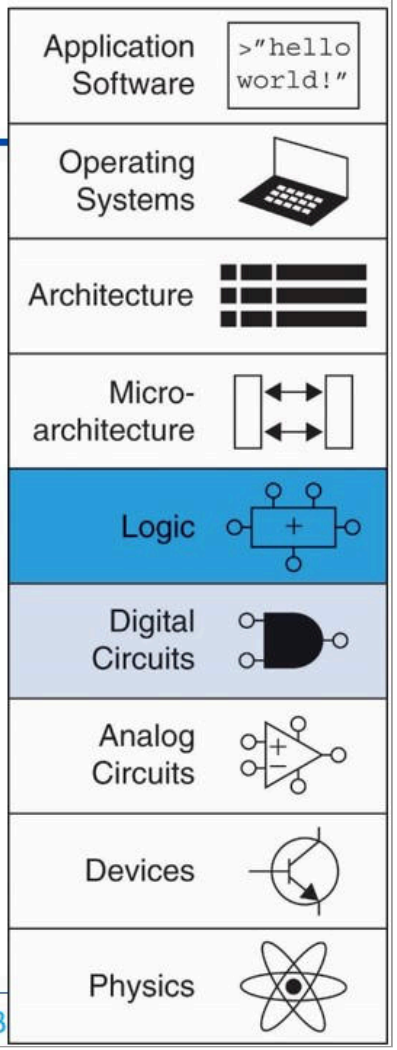
- 电路分析
  - A=1011, B=1110



- S1的值是什么? (a) 0 (b) 1

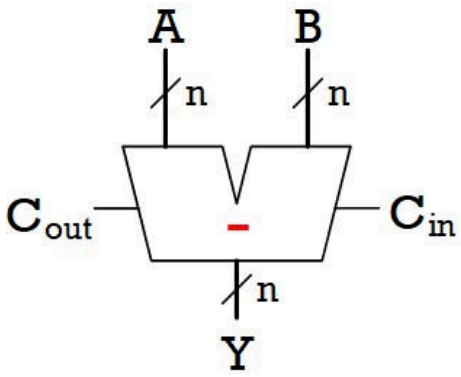
Chapter 5 :: Topics

- 引言
- **算术运算电路**
  - **多位加法器（1位→多位→带溢出判断→减法器→加减法器）**
  - **算术逻辑单元 ALU**
  - **更快速的多位加法器（行波进位→先行进位→前缀进位）**
  - **移位器→乘法器/除法器**
- 数制系统（重点介绍浮点数）
- 时序电路模块
- 存储器阵列

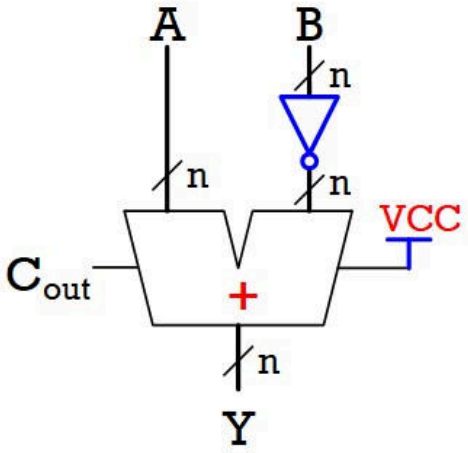


减法器 Subtracter

符号

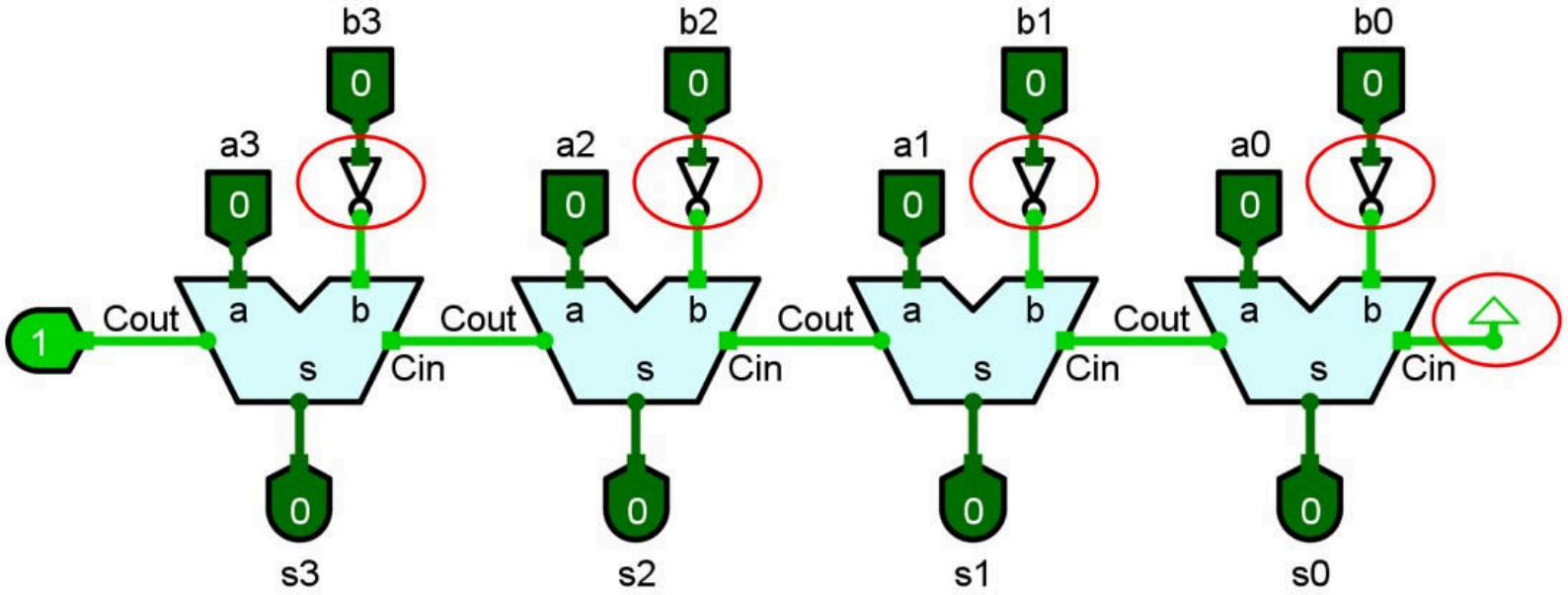


实现



多位减法器

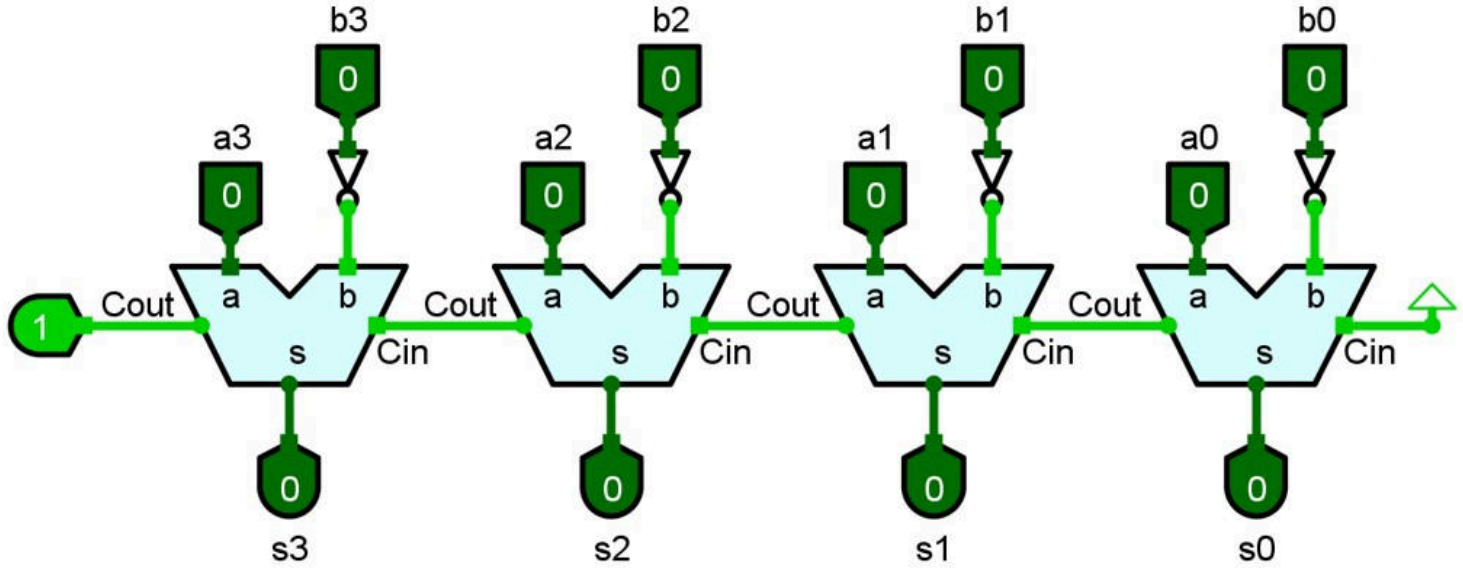
- 例如，4-bit 减法器



多位减法器

慕课堂练习

- 电路分析
  - A=0011 , B=1110

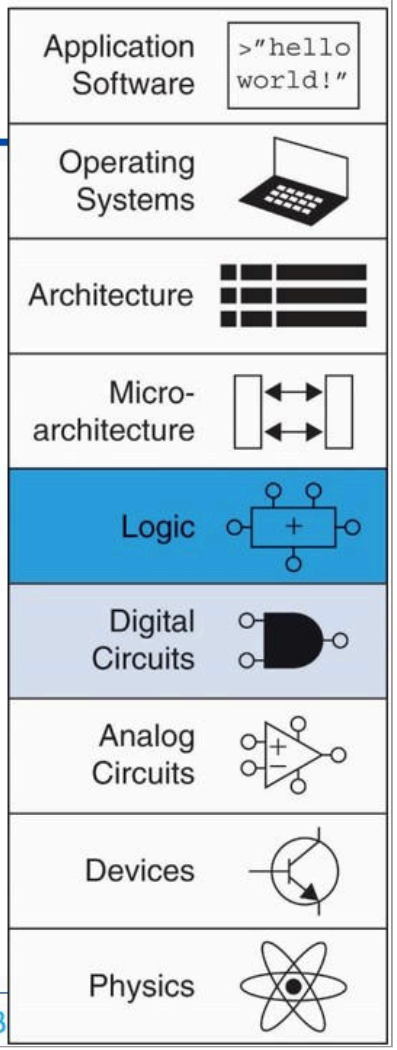


- S3的值是什么? (a) 0 (b) 1

雨课堂 Rain Classroom

Chapter 5 :: Topics

- 引言
- **算术运算电路**
  - **多位加法器（1位→多位→带溢出判断→减法器→加减法器）**
  - **算术逻辑单元 ALU**
  - **更快速的多位加法器（行波进位→先行进位→前缀进位）**
  - **移位器→乘法器/除法器**
- 数制系统（重点介绍浮点数）
- 时序电路模块
- 存储器阵列

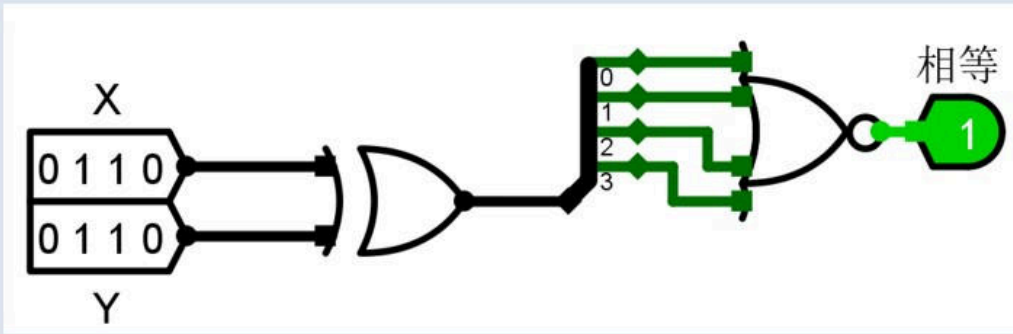


异或门XOR在数字逻辑中的多重作用

- **异或 XOR 运算的迷人特性之一：**
  - $X \oplus X = 0$
  - $X \oplus X' = 1$

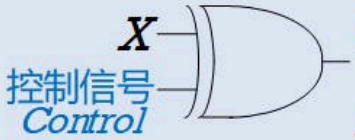
  } 相同信号异或为0，不同信号异或为1，**可以构建相等比较器**

Logisim仿真实现



异或门XOR在数字逻辑中的多重作用

- 异或 XOR 运算的迷人特性之二：



X ，如果控制信号为0，则输出数据 X本身

X' ，如果控制信号为1，则输出数据 X的非

- 逻辑电路中的逻辑电平(信号)，分成了“数据”和“控制信号”

雨课堂 Rain Classroom

异或门XOR在数字逻辑中的多重作用

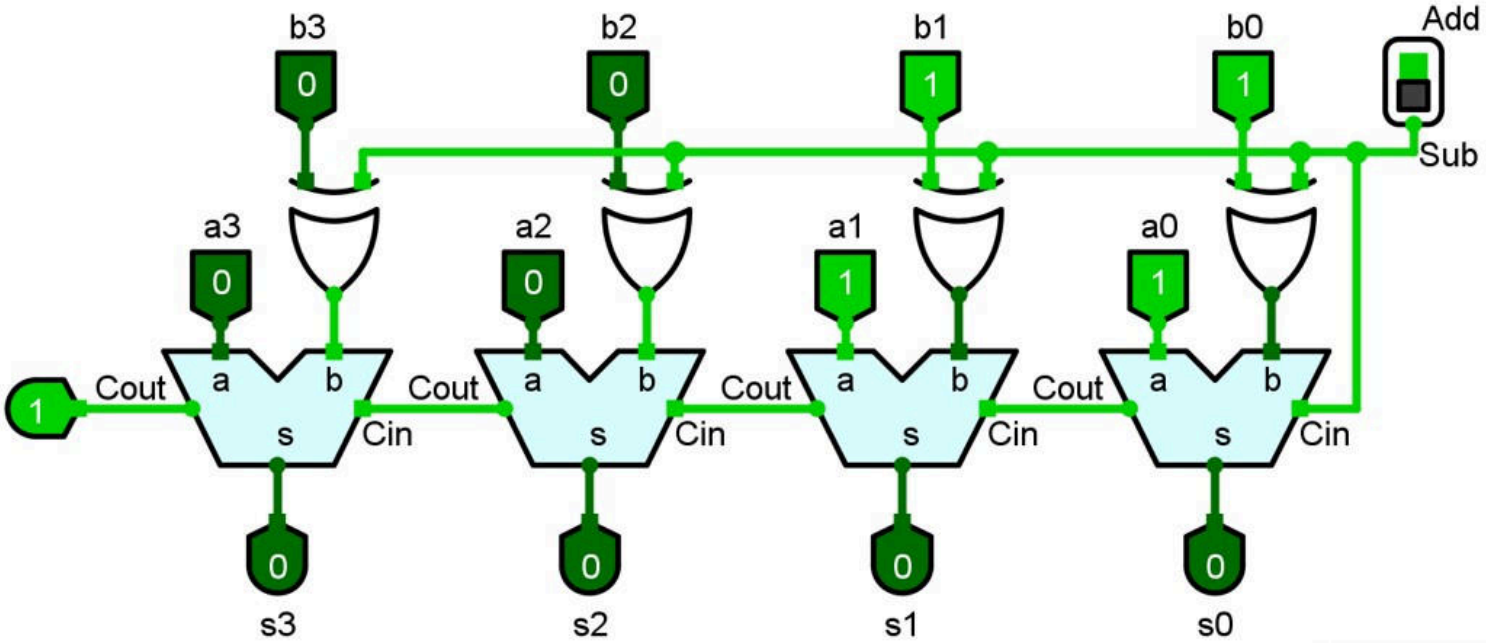
- **异或 XOR 运算的迷人特性之三：**
  - $X \oplus (Y \oplus Z) = (X \oplus Y) \oplus Z$ **结合律**
  - $X \oplus Y = Y \oplus X$ **交换律**
  - $X \oplus m \oplus m = X \oplus (m \oplus m) = X \oplus 0 = X$

    加密过程　解密过程　使用了以上特性

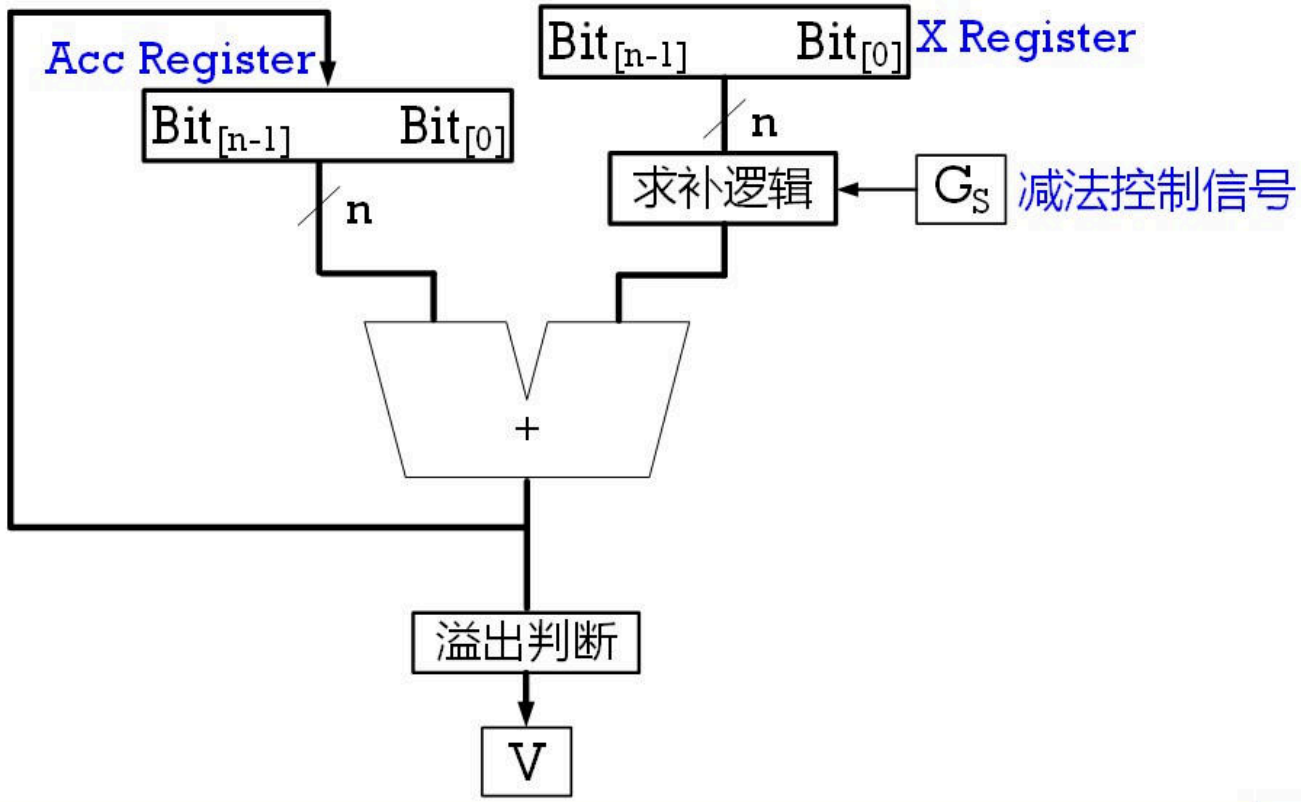
- 一次性密码本加密（常简称：一次一密）
  - 一次性密码本(one-time pad, OTP)：密码学经典加密方案
  - 明文、密钥、密文长度完全相同；密钥是完全随机、永不重复、仅使用一次；

在一个逻辑电路中既实现加法器也实现减法器

- 利用 XOR 门



小结：补码加减法器硬件电路



“数据通路”与“控制”

- 我们要从概念上区分硬件中的两种类型的信号：**数据**和**控制**

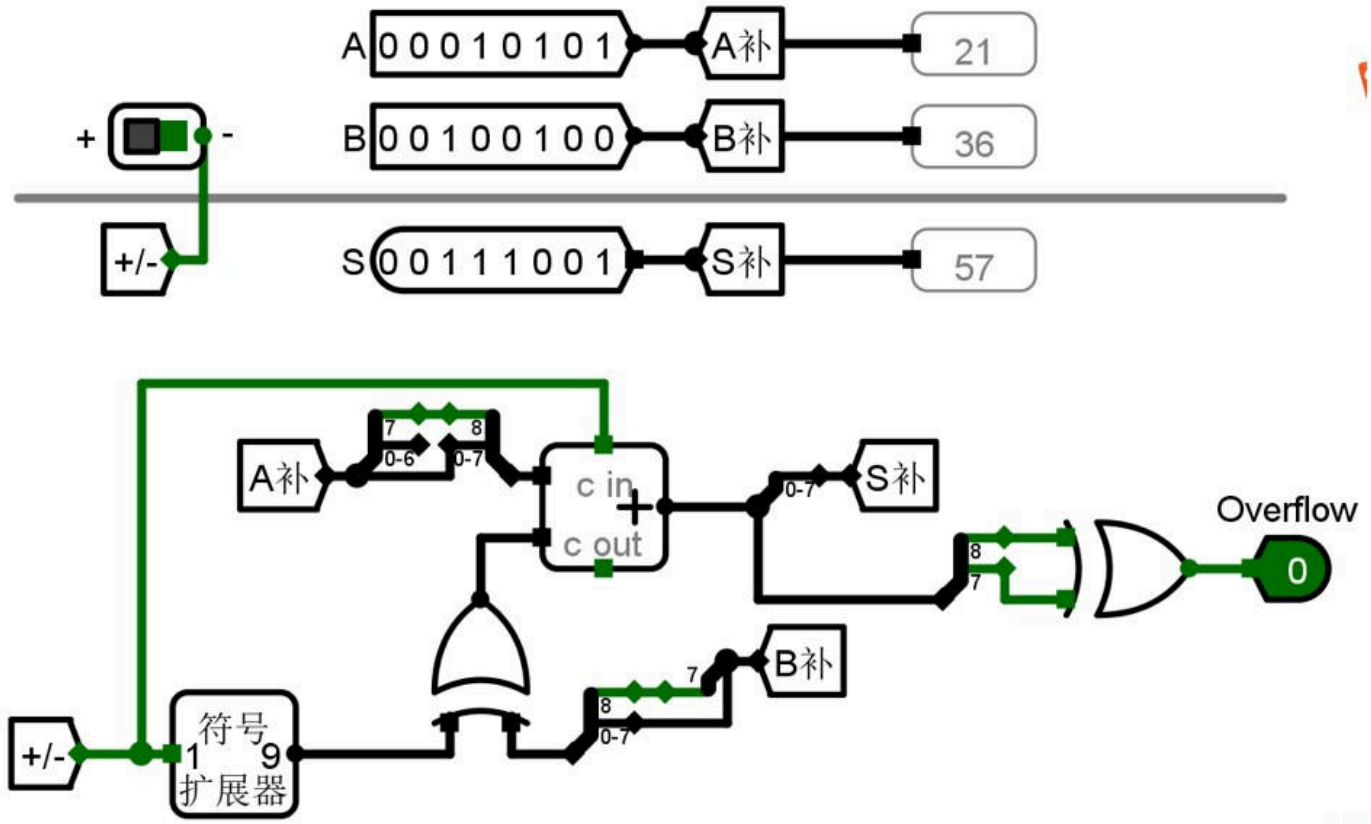
- **数据通路 Datapath**
  - 数据通路通常推动我们正在处理的数字
  - 例如，上图中加减法器逻辑中的输入 X, Y 和输出 S
- **控制信号**
  - 控制信号通常控制数据的流动方式和执行的操作
  - 例如，上图中的 SUB 信号.

雨课堂 Rain Classroom

课后作业4

- 设计加减法器，并使用Logisim实现
- 要求：
  - 加减法器是8-bit 总线的；
  - 操作数以补码表示；
  - 带有溢出判断。

补码加减法器硬件电路logisim仿真



第十二堂课结束了

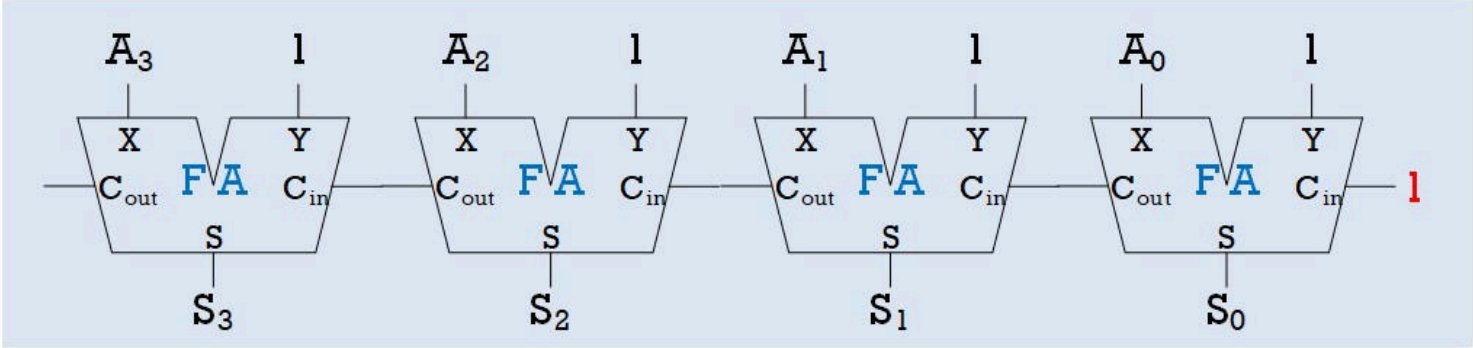
有问题同学们可以提问讨论

雨课堂 Rain Classroom

电路分析 (慕课堂练习)

- 假设输入输出数据是 4-bit 补码表示，该电路实现了什么数学运算?

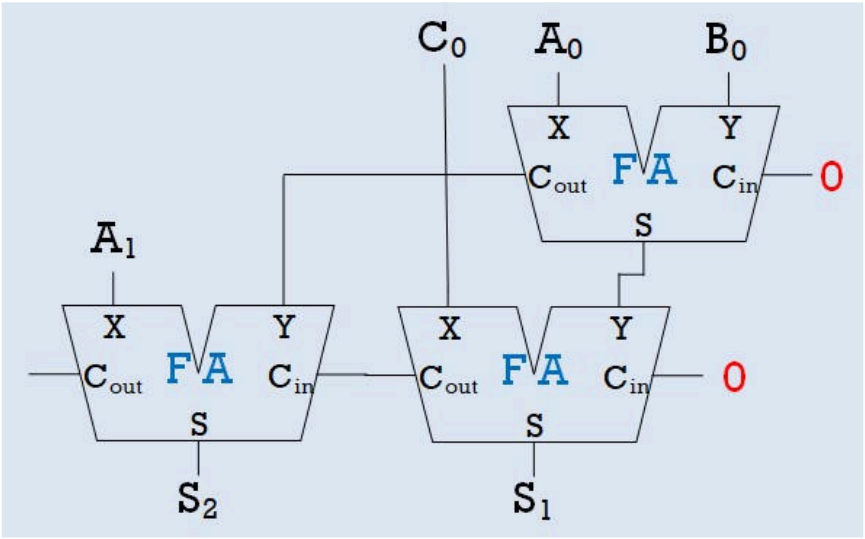
(a) S = A - 1 (b) S = A (c) S = A + 1 (d) S = A + 5



电路分析 (慕课堂练习)

- 假设A = $(11)_2$, B_0 = 1, C_0 =1, S_1的值是多少?

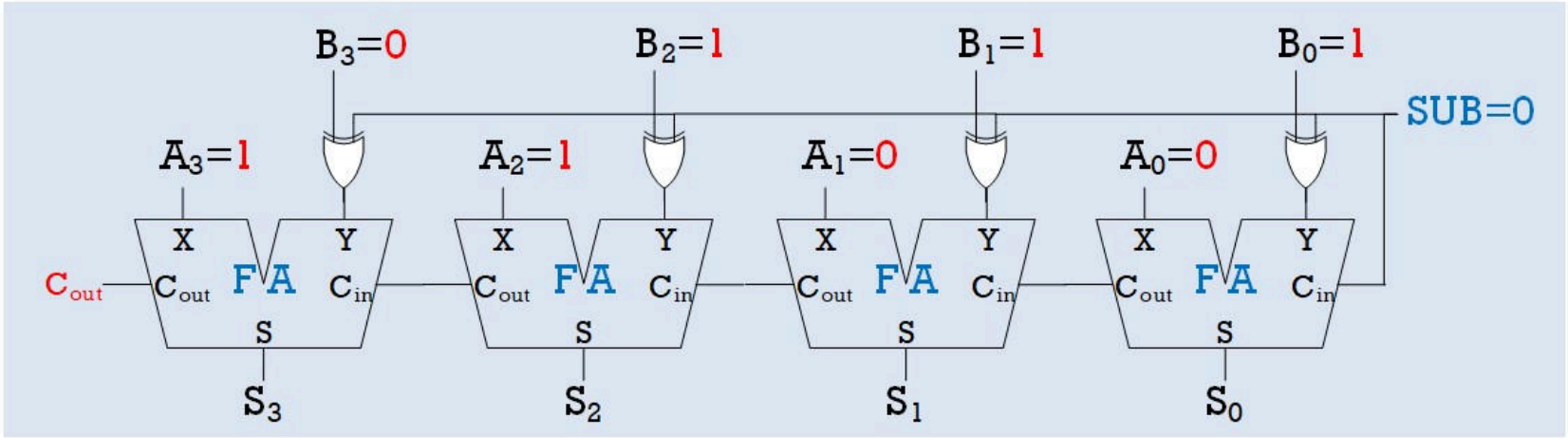
  (a) 0 (b) 1



电路分析 (慕课堂练习)

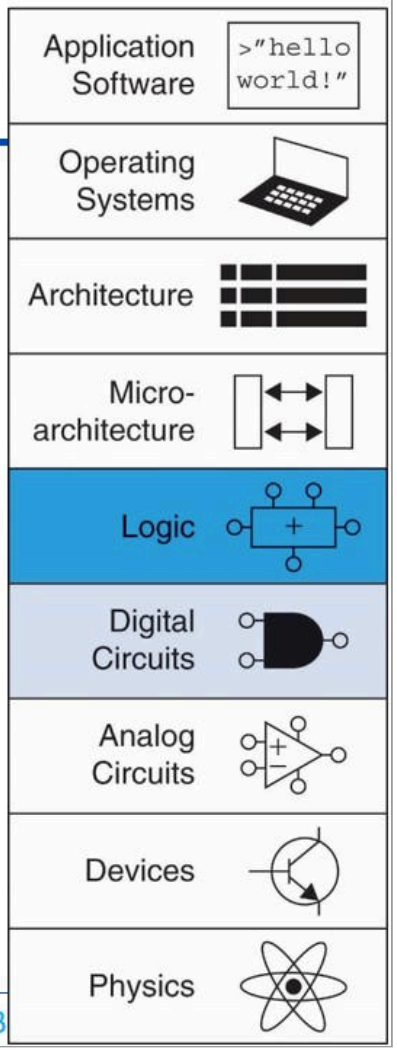
- 产生溢出了吗?

(a) 产生溢出　(b) 没有产生溢出　(c) 信息不足无法判断

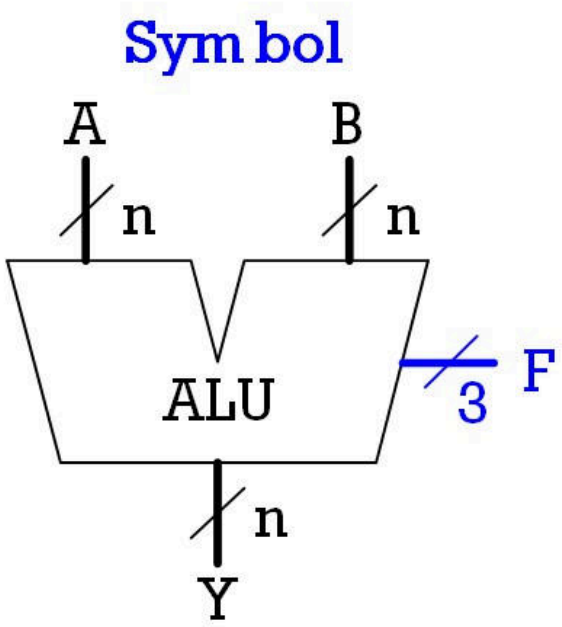


Chapter 5 :: Topics

- 引言
- **算术运算电路**
  - **多位加法器（1位→多位→带溢出判断→减法器→加减法器）**
  - **算术逻辑单元 ALU**
  - **更快速的多位加法器（行波进位→先行进位→前缀进位）**
  - **移位器→乘法器/除法器**
- 数制系统（重点介绍浮点数）
- 时序电路模块
- 存储器阵列



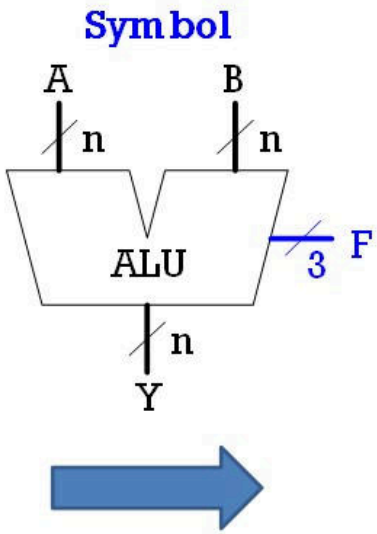
算术逻辑单元 Arithmetic Logic Unit (ALU)



$F_{2:0}$	功能
000	A & B
001	A | B
010	A + B
011	not used
100	A & ~B
101	A | ~B
110	A - B
111	SLT

算术逻辑单元 (ALU)

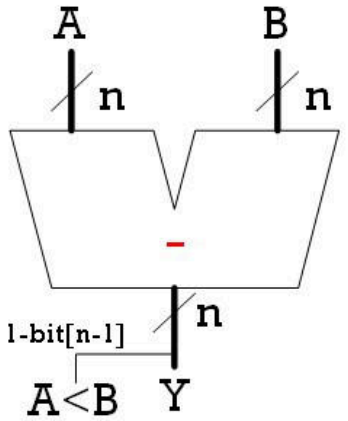
$F_{2:0}$	Function
000	A & B
001	A | B
010	A + B
011	not used
100	A & ~B
101	A | ~B
110	A - B
111	SLT



$F_{2:0}$	Function
000	A & B
100	A & ~B
001	A | B
101	A | ~B
010	A + B
110	A - B
011	not used
111	SLT

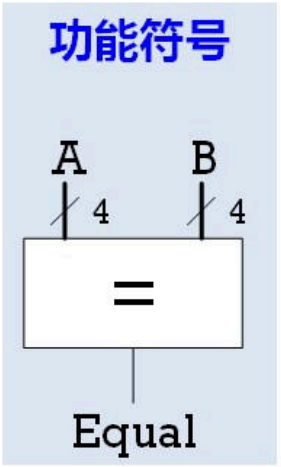
算术逻辑单元 (ALU)

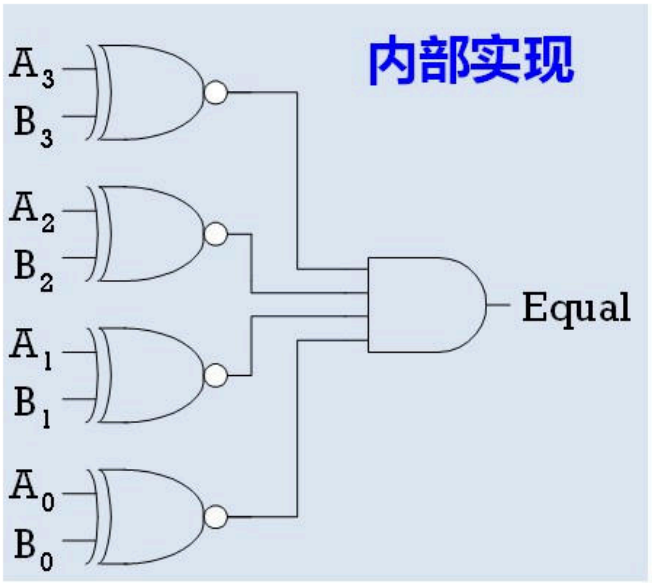
- Set Less Than (SLT, $F_{2:0}$=111) 数量比较操作
  - 产生一位有效输出，来说明A和B的关系值
    - A < B，则输出1
    - A≧B，则输出0
  - 实现方法：
    - 首先计算A-B的值，再检查结果的符号位（最高位）
    - 如果结果为负数，则A<B，否则A≧B



另一种比较器: 相等比较器Equality

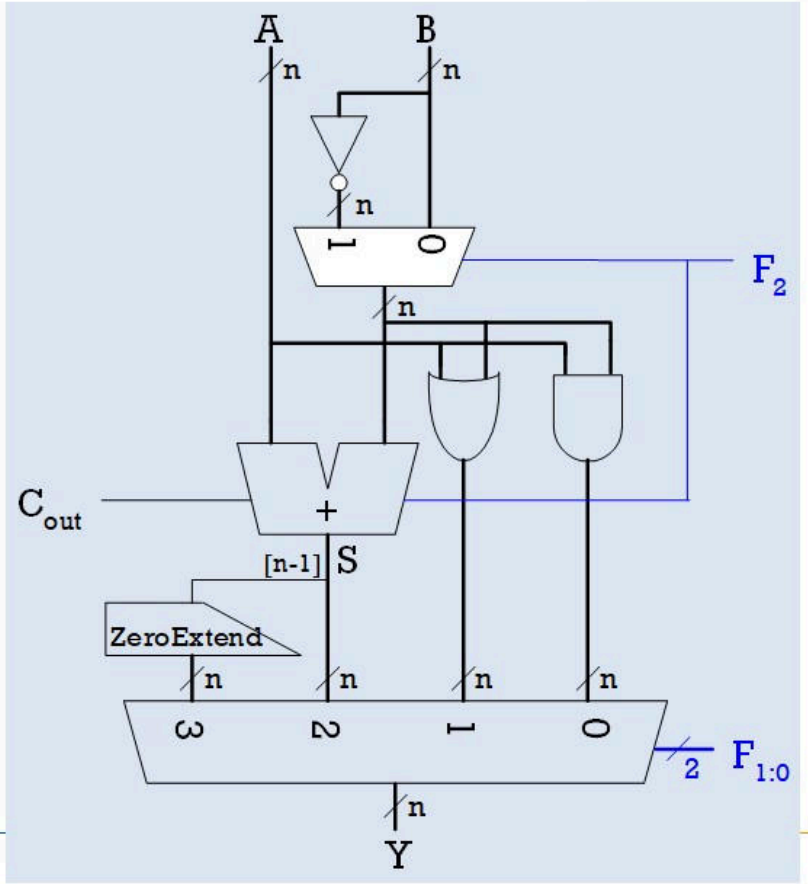
- 只能比较是否相等，不能比较大小，上述ALU中不适用





ALU 设计

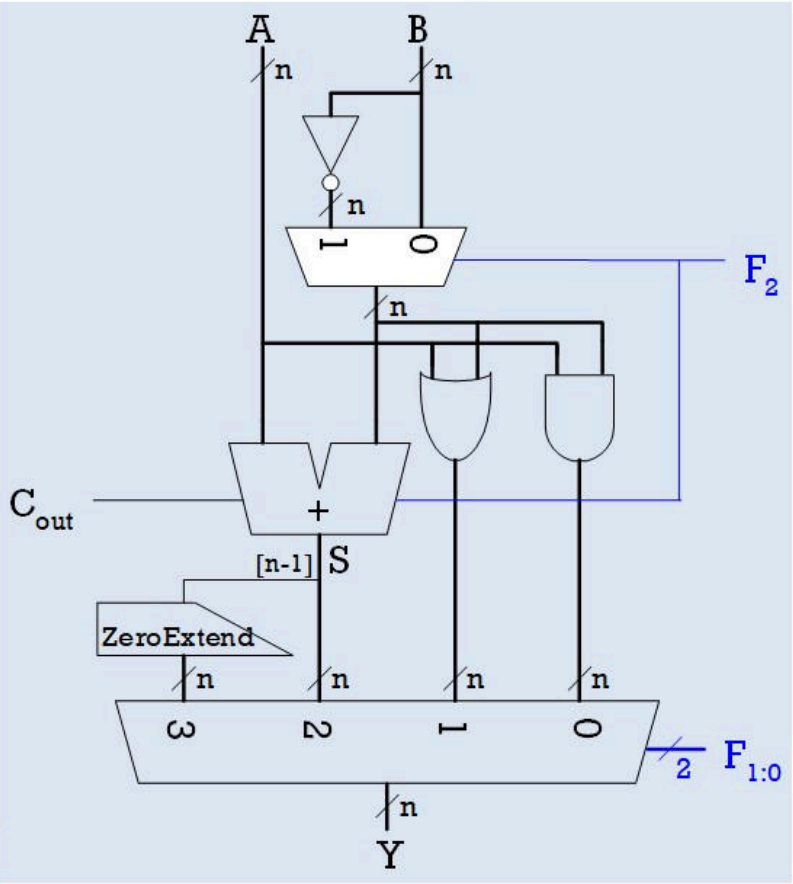
$F_{2:0}$	Function
000	A & B
100	A & ~B
001	A | B
101	A | ~B
010	A + B
110	A - B
011	not used
111	SLT



NEU-SWC-A0801040040

数量比较操作 SLT 示例

- 32-bit ALU 的 SLT(Set Less Than, 小于置位) 操作:
  - A = 25 ， B = 32
- 运算结果是?

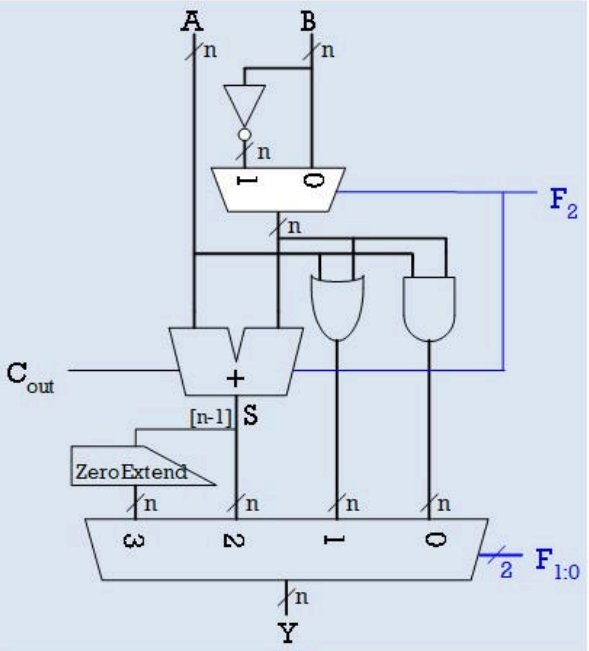


NEU-SWC-A0801040040

数量比较操作 SLT 示例

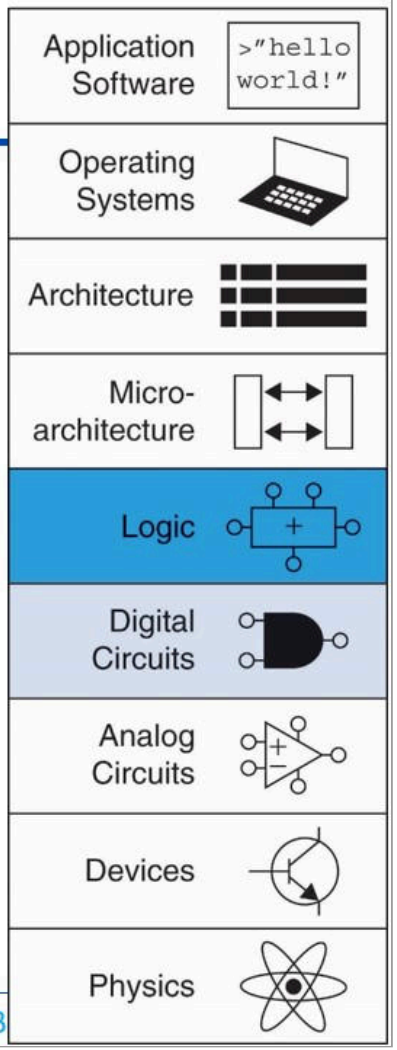
- 32-bit ALU 的 SLT(Set Less Than, 小于置位) 操作: A = 25， B = 32
  - A < B, 则 Y 应输出 32-bit 的数据“1” (0x00000001)
  - $F_{2:0}$ = 111
    - F_2 = 1， 加法器实际完成减法操作, 则25 - 32 = -7
    - -7 的最高位是 1 (S_{31} = 1)
    - $F_{1:0}$ = 11 多路选择器选择最左边的第3路

      Y = S_{31} (零扩展zero extended) = 0x00000001.



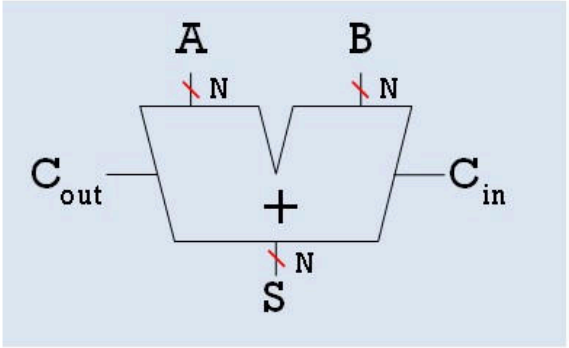
Chapter 5 :: Topics

- 引言
- **算术运算电路**
  - **多位加法器（1位→多位→带溢出判断→减法器→加减法器）**
  - **算术逻辑单元 ALU**
  - **更快速的多位加法器（行波进位→先行进位→前缀进位）**
  - **移位器→乘法器/除法器**
- 数制系统（重点介绍浮点数）
- 时序电路模块
- 存储器阵列



多位加法器 (进位传播加法器CPAs)

- 三种进位传播加法器(carry propagate adders，CPAs):
  - 行波进位加法器 Ripple-carry (slow)
  - 先行进位加法器 Carry-lookahead (fast)
  - 前缀加法器 Prefix (faster)

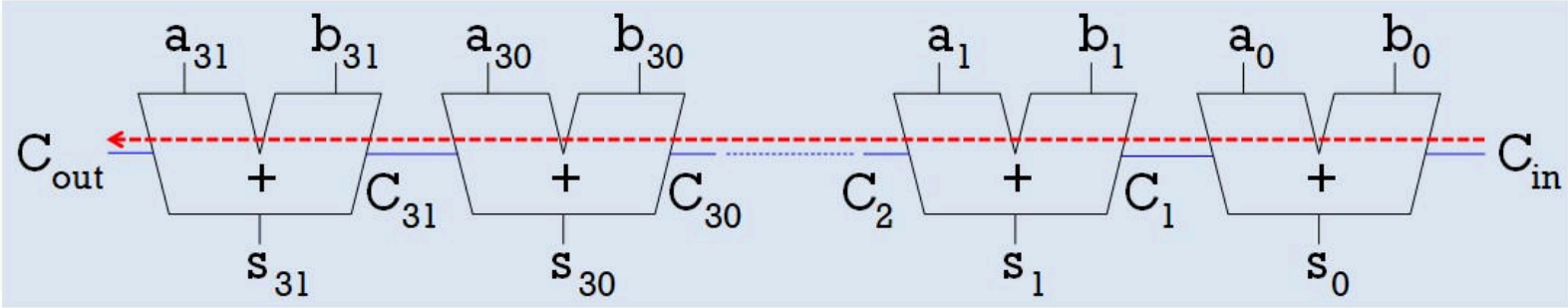


- 先行进位加法器和前缀加法器比行波进位加法器快，但是需要使用更多的硬件。

行波进位(Ripple-carry) 多位加法器

慕课堂练习

- 用1-bit 加法器级联，进位信号**逐级传递**通过每一位加法器

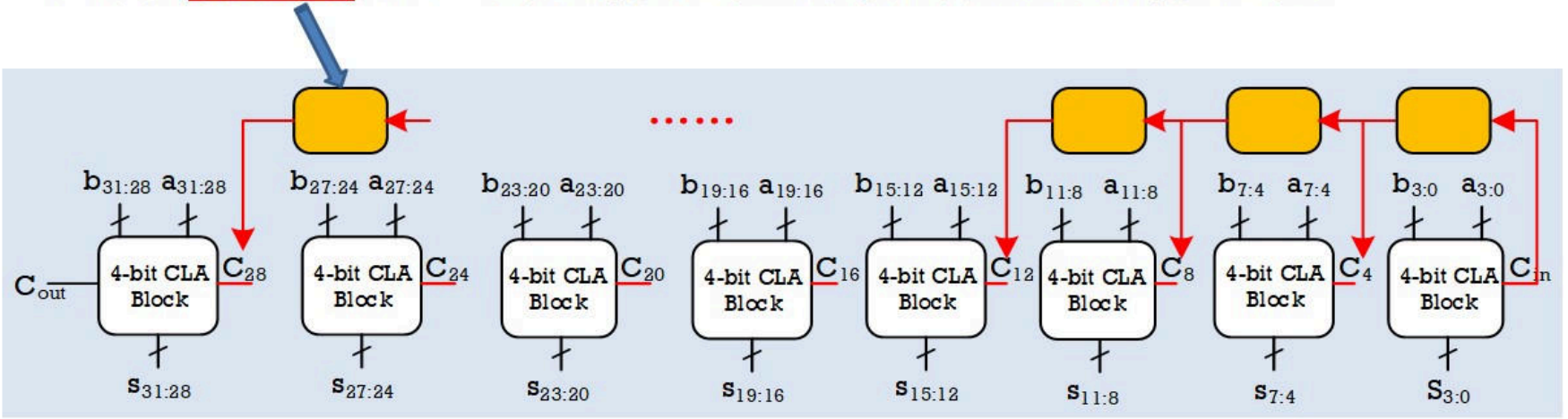


- 缺点: 慢
  - N位加法器的传输延迟： $t_{ripple} = Nt_{FA}$，其中 t_{FA} 是1-bit全加器的传输延迟

雨课堂 Rain Classroom

先行进位(Carry-Lookahead) 多位加法器

- 解决方案——分块并行
  - 例如把32位加法器，分成4位一块，共8块
  - 增加额外电路在每一块得到输入进位时就快速算出此块的输出进位。



先行进位(Carry-Lookahead) 多位加法器

- 关键问题：如何加速进位生成?

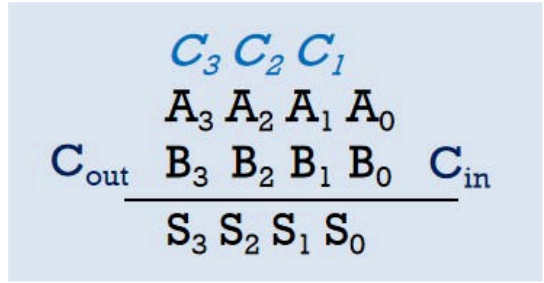
$$\begin{array}{cccccc} & & C_3 & C_2 & C_1 & \\ & & A_3 & A_2 & A_1 & A_0 \\ C_{out} & B_3 & B_2 & B_1 & B_0 & C_{in} \\ \hline & S_3 & S_2 & S_1 & S_0 & \end{array}$$

- 提出两个关键术语：
  - 进位生成信号 (Generate signals)： $G_i = A_i B_i$
  - 进位传递信号 (Propagate signals)： $P_i = A_i + B_i$

雨课堂 Rain Classroom

先行进位(Carry-Lookahead) 多位加法器

- 以4-bit 加法器为例，分析每一级进位输出C_{i+1}的情况：
  - $C_1 = A_0B_0 + (A_0 + B_0)C_{in}$
    $= G_0 + P_0C_{in}$
  - $C_2 = A_1B_1 + (A_1 + B_1)C_1$
    $= A_1B_1 + (A_1 + B_1)(A_0B_0 + (A_0 + B_0)C_{in})$
    $= A_1B_1 + (A_1 + B_1)A_0B_0 + (A_1 + B_1)(A_0 + B_0)C_{in}$
    $= G_1 + P_1G_0 + P_1P_0C_{in}$
  - $C_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_{in}$
  - $C_{out} = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_{in}$



先行进位(Carry-Lookahead) 多位加法器

$$C_1 = G_0 + P_0C_{in}$$
$$C_2 = G_1 + P_1G_0 + P_1P_0C_{in}$$
$$C_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_{in}$$

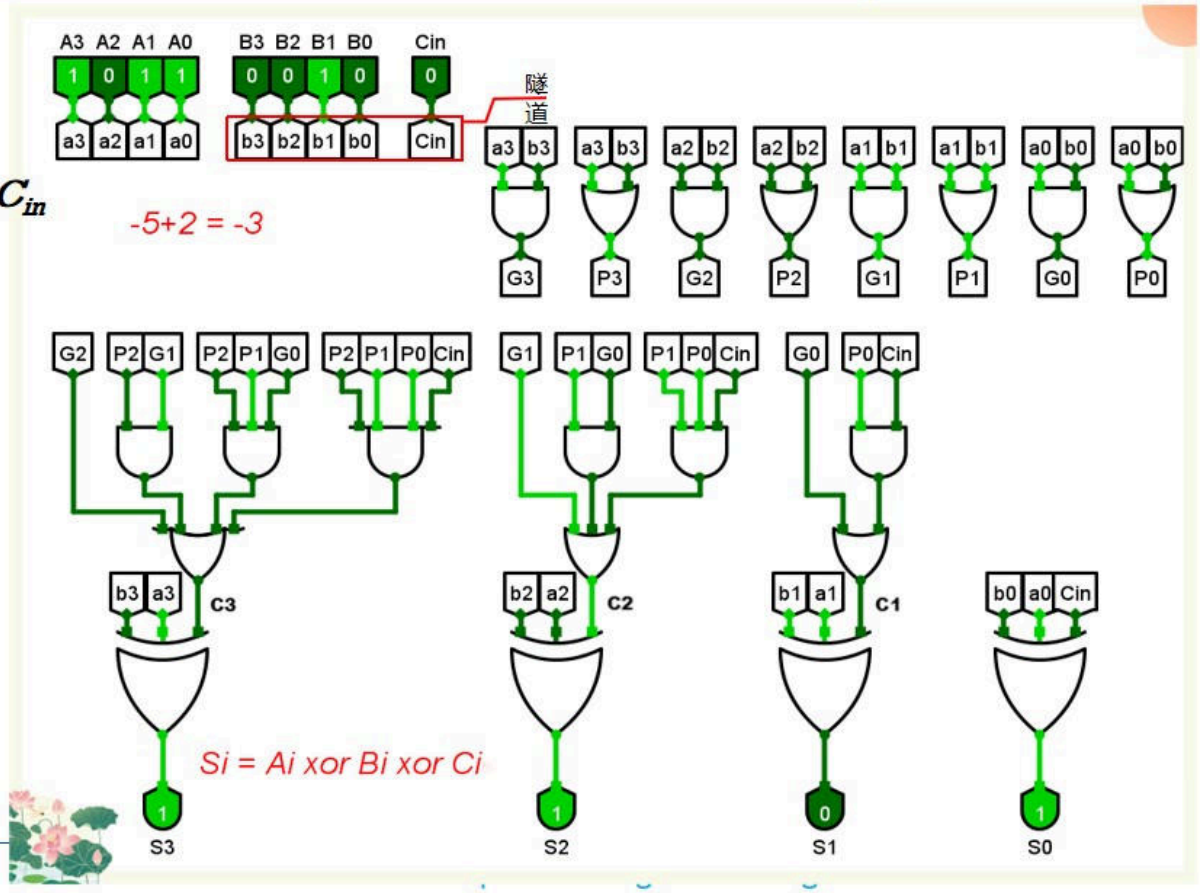
$C_3\ C_2\ C_1$

$A_3\ A_2\ A_1\ A_0$

C_{out} $B_3\ B_2\ B_1\ B_0$ C_{in}

$S_3\ S_2\ S_1\ S_0$

$$s_0 = a_0 \oplus b_0 \oplus C_{in}$$
$$s_1 = a_1 \oplus b_1 \oplus C_1$$
$$s_2 = a_2 \oplus b_2 \oplus C_2$$
$$s_3 = a_3 \oplus b_3 \oplus C_3$$



NEU-SWC-A0801040040

先行进位(Carry-Lookahead) 多位加法器

- 前述介绍的都是**逐位进位信号**的快速产生，对于进位生成和进位传递的概念也可以**扩展到**多位构成的**模块**
- 例如: 4-bit 模块的 进位生成逻辑 $G_{3:0}$ 和 进位传递逻辑 $P_{3:0}$:

$$G_{3:0} = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 = G_3+P_3(G_2+P_2(G_1+P_1G_0)$$

$$P_{3:0} = P_3P_2P_1P_0$$

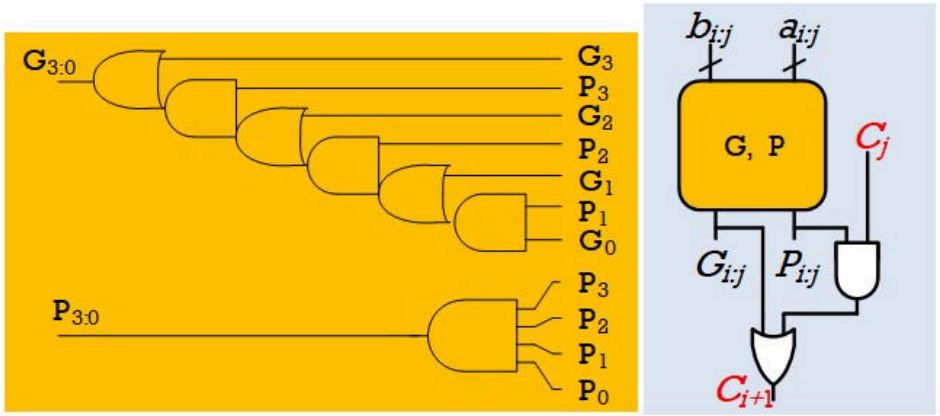
$$C_4 = G_{i:j} + P_{i:j}C_j$$

- 更高位的 4-bit 模块，假设 i = j+3

$$G_{i:j} = G_i + P_i(G_{i-1} + P_{i-1}(G_{i-2}+P_{i-2}G_j)$$

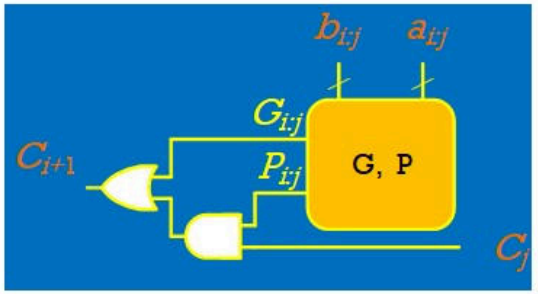
$$P_{i:j} = P_iP_{i-1}P_{i-2}P_j$$

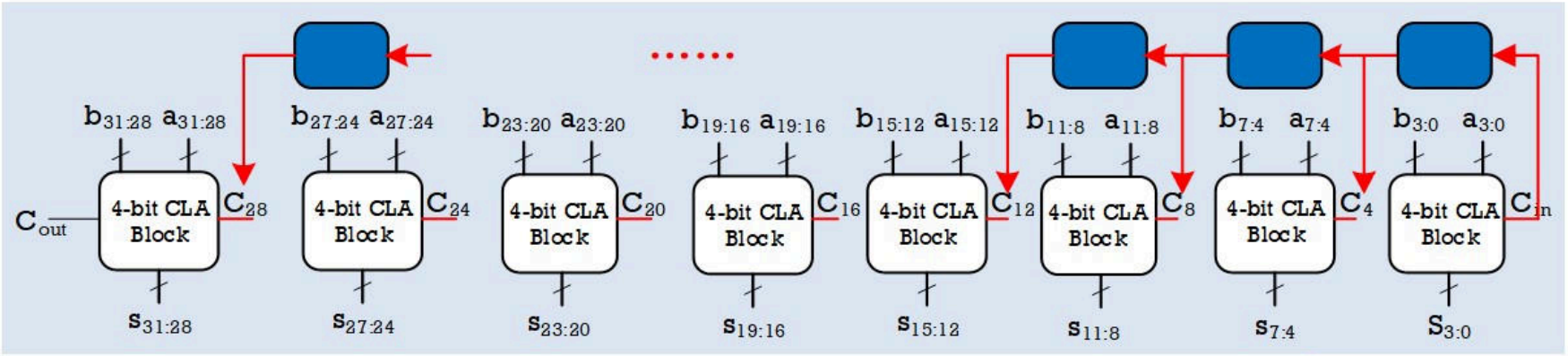
$$C_{i+1} = G_{i:j} + P_{i:j}C_j$$



先行进位(Carry-Lookahead) 多位加法器

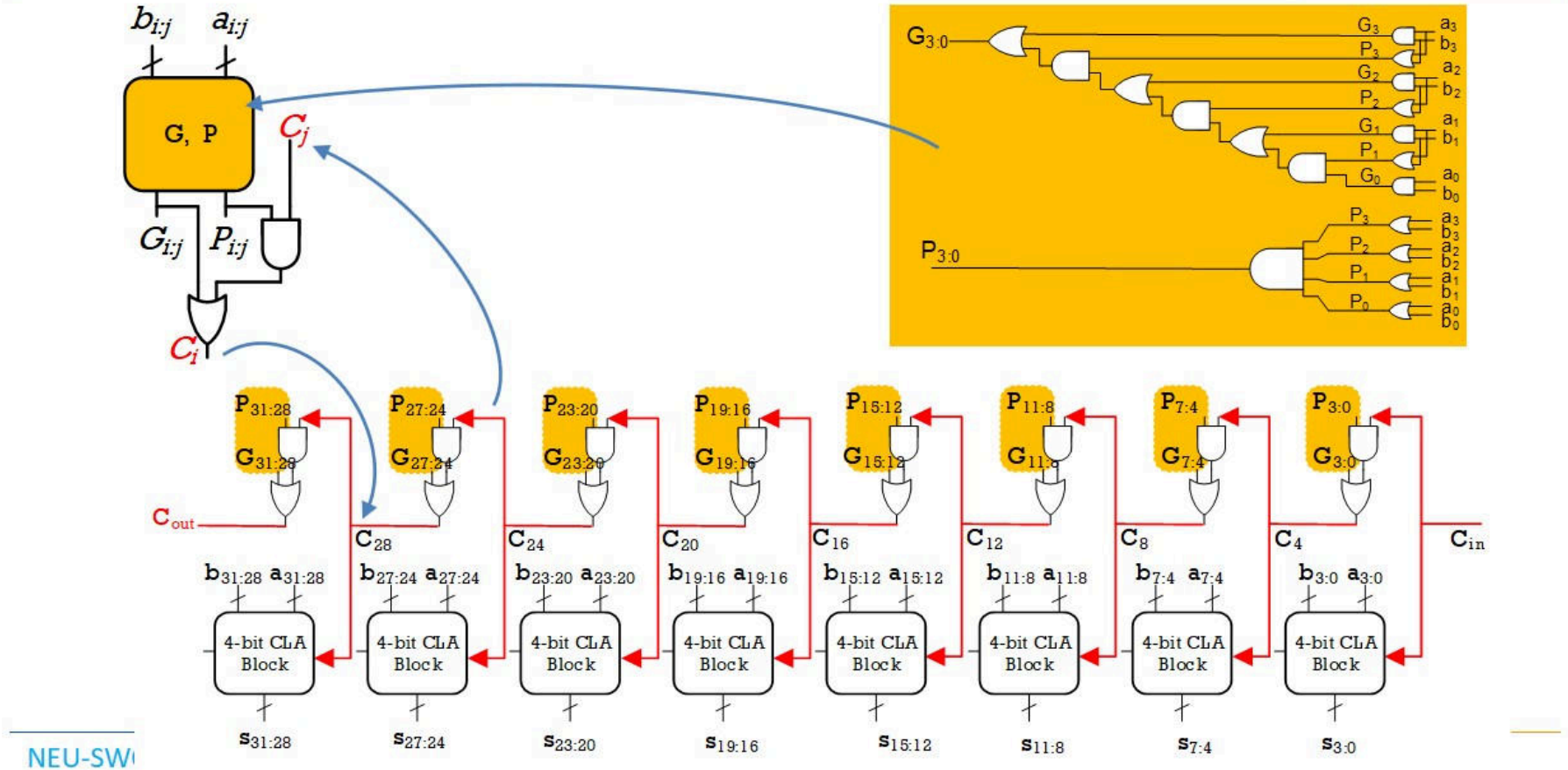
- 每个4-bits模块的 进位输出 $C_{i+1} = G_{i:j} + P_{i:j}C_j$





32-bit 先行进位加法器(4-bits一个模块)

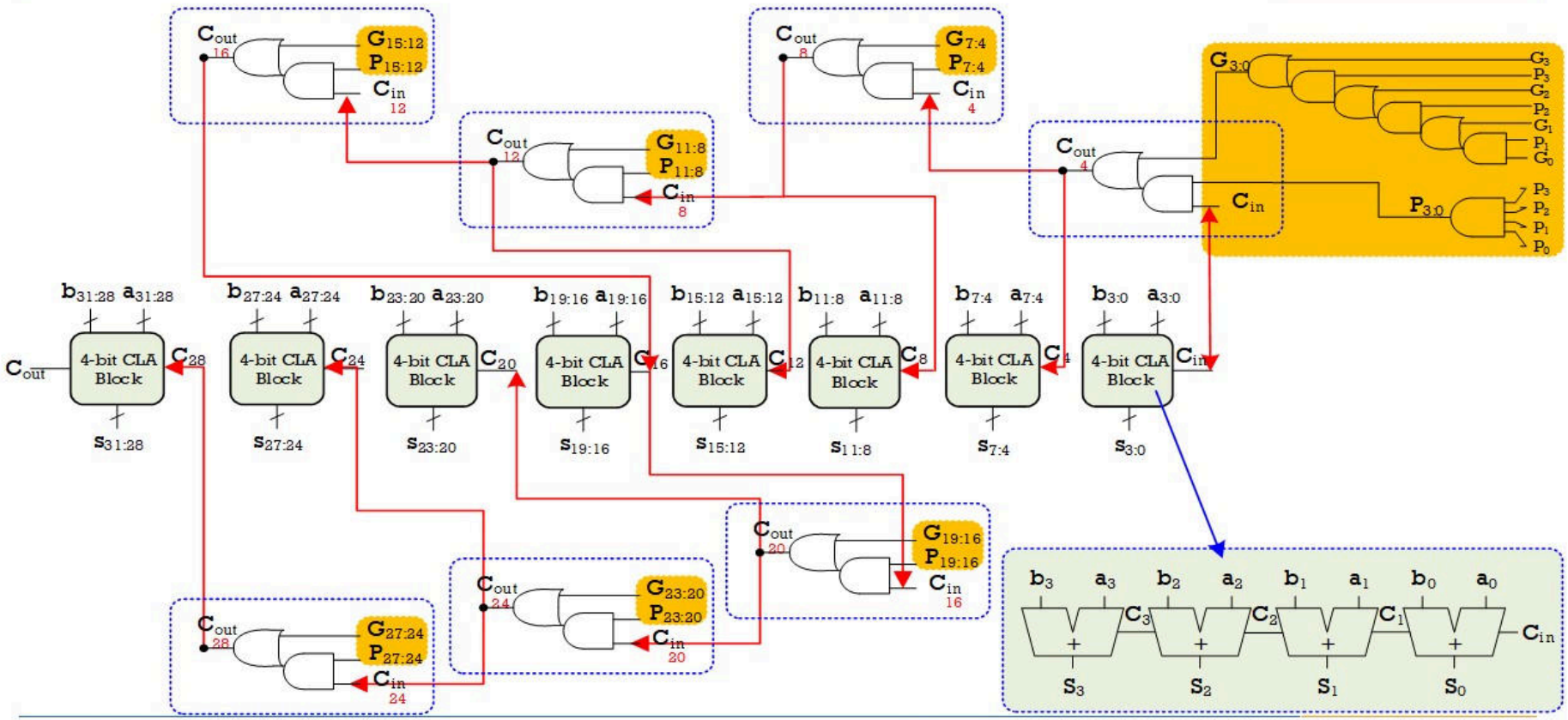
慕课堂练习



NEU-SW

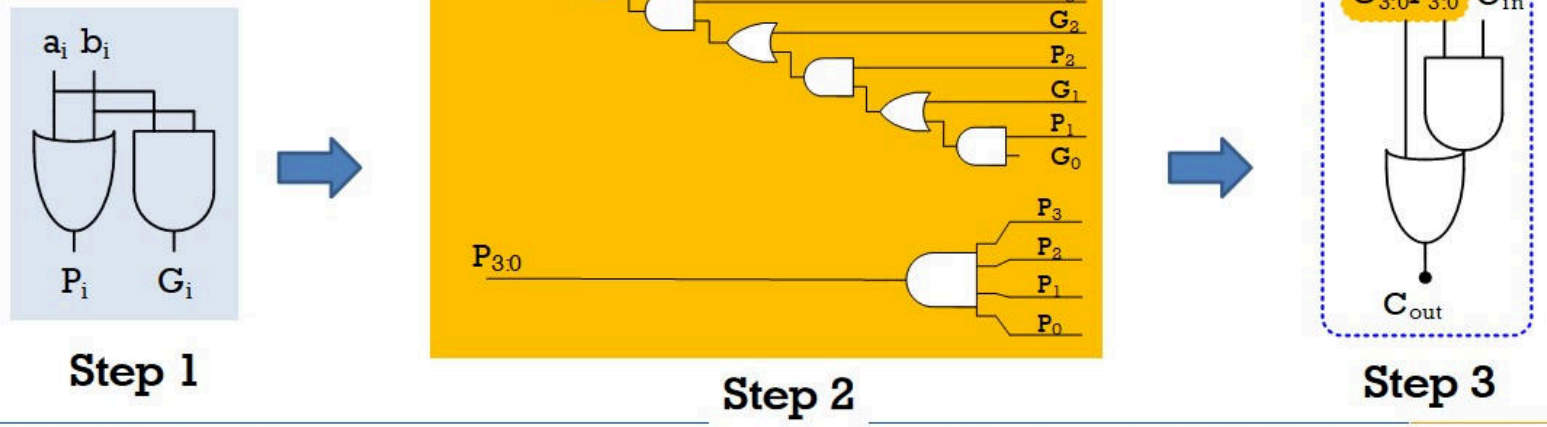
雨课堂 Rain Classroom

32-bit 先行进位加法器(4-bits一个模块)



先行进位(Carry-Lookahead) 多位加法器的延迟时间

- 步骤 1: 同时(并行)计算所有位的 *Gi* 和 *Pi*
- 步骤 2: 同时(并行)计算每一个k-bit 模块的 *G* and *P*
- 步骤 3: *Cin* 逐级经过 AND/OR 门通过每一个 k-bit 模块



Step 1 Step 2 Step 3

先行进位(Carry-Lookahead) 多位加法器的延迟时间

- 对于 N-bit 先行进位加法器（k-bit 分一个模块）：

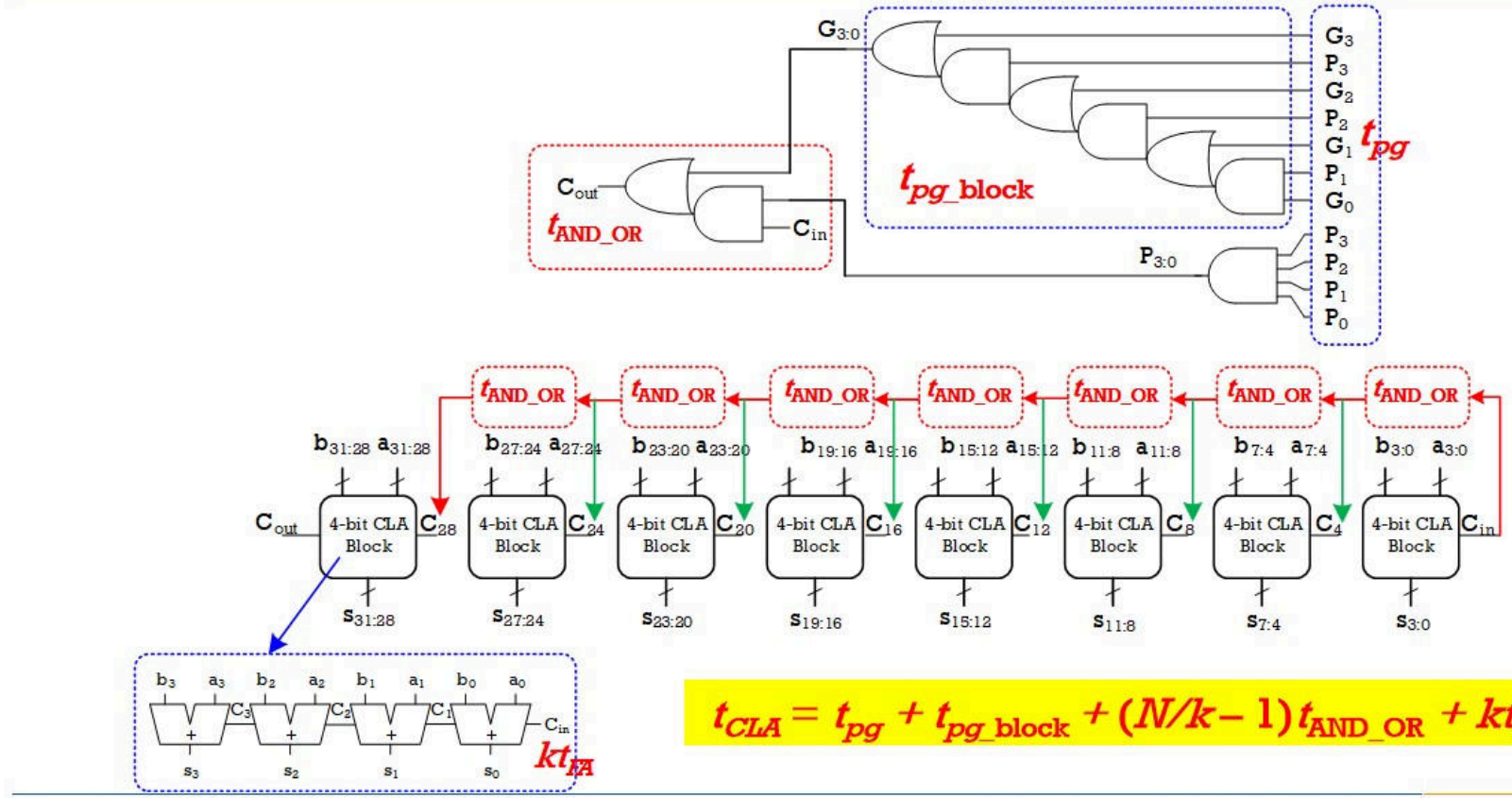
$$t_{CLA} = t_{pg} + t_{pg_block} + (N/k - 1)t_{AND_OR} + kt_{FA}$$

  - t_{pg}: 对于每一位的 P_i，G_i 的传输延迟
  - t_{pg_block}: 对于每一个模块的 $P_{i:j}$，$G_{i:j}$ 的传输延迟
  - t_{AND_OR}: *Cin* 逐级经过 AND/OR 门 生成每一个 k-bit 模块 *Cout* 的传输延迟

- 当 N >16时，一个*N*-bit 先行进位加法器通常比行波进位加法器速度更快。

雨课堂 Rain Classroom

先行进位(Carry-Lookahead) 多位加法器的延迟时间



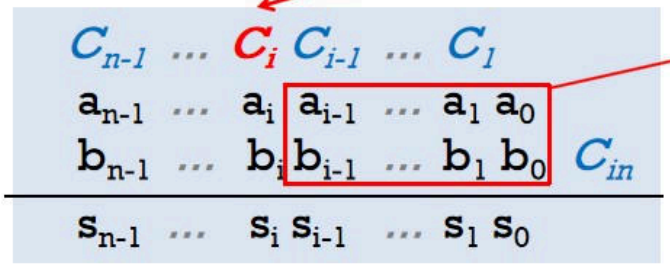
$$t_{CLA} = t_{pg} + t_{pg_block} + (N/k - 1)t_{AND_OR} + kt_{FA}$$

前缀(Prefix)进位 多位加法器

- 如果计算机能够计算出每一位的进位 C_i，那么就可以计算每一位的和:

$$s_i = a_i \oplus b_i \oplus C_i$$

- 所以，要想加快加法器的速度，关键要解决的是:
  - 如何快速计算出**每一位的进位 C_i**



如果将第0列-第i-1列看成一个虚拟block，

若已知该虚拟block的$G_{i-1:0}$和 $P_{i-1:0}$

则$C_i = G_{i-1:0} + P_{i-1:0}C_{in}$

雨课堂 Rain Classroom

前缀(Prefix)进位 多位加法器

- 要想加快加法器的速度，关键要解决的是：
  - 如何快速计算出**每一位的进位** C_i

C_{n-1}	…	C_i	C_{i-1}	…	C_1	C_{in}	
a_{n-1}	…	a_i	a_{i-1}	…	a_1	a_0	0
b_{n-1}	…	b_i	b_{i-1}	…	b_1	b_0	0
s_{n-1}	…	s_i	s_{i-1}	…	s_1	s_0	

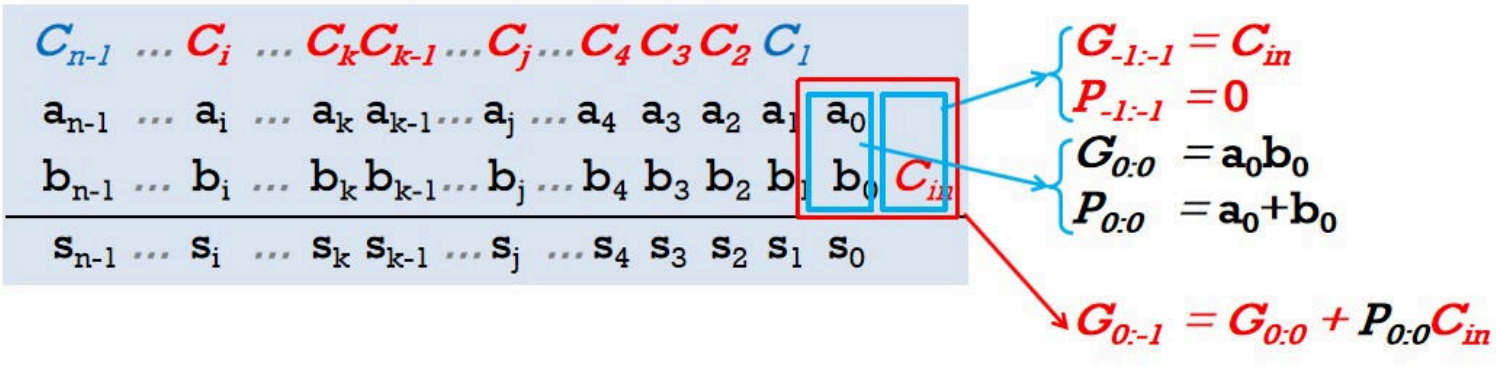
如果将C_{in}扩入，看成第-1列，那么可以将第-1列—第i-1列看成一个block，则该block无最低位的C_{in}

所以：$C_i = G_{i-1:-1}$

- 现在问题转化为：
  - 如何快速计算出每一个虚拟block的$G_{i-1:-1}$
  - 即$G_{0:-1}$，$G_{1:-1}$，$G_{2:-1}$，……，$G_{n-2:-1}$，叫前缀(prefixes)

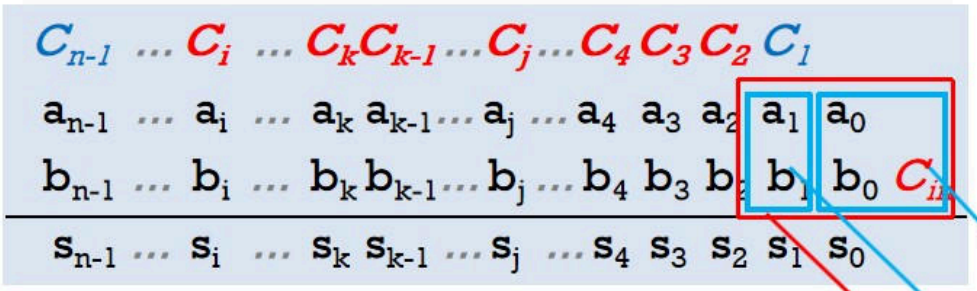
前缀(Prefix)进位 多位加法器

- 如何快速计算出每一个虚拟block的$G_{i-1:-1}$
- 即$G_{0:-1}$，$G_{1:-1}$，$G_{2:-1}$，……，$G_{n-2:-1}$，叫前缀(prefixes)



前缀(Prefix)进位 多位加法器

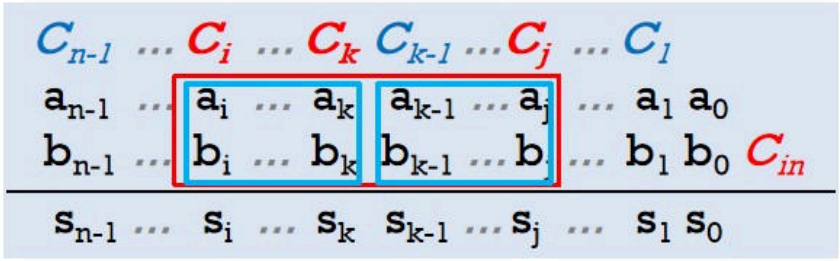
- 如何快速计算出每一个虚拟block的$G_{i-1:-1}$
- 即$G_{0:-1}$，$G_{1:-1}$，$G_{2:-1}$，……，$G_{n-2:-1}$，叫前缀(prefixes)



$$G_{-1:-1} = C_{in}$$
$$P_{-1:-1} = 0$$
$$G_{0:0} = a_0 b_0$$
$$P_{0:0} = a_0 + b_0$$
$$G_{0:-1} = G_{0:0} + P_{0:0} C_{in}$$
$$G_{1:1} = a_1 b_1$$
$$P_{1:1} = a_1 + b_1$$
$$G_{1:-1} = G_{1:1} + P_{1:1} G_{0:-1}$$

前缀(Prefix)进位 多位加法器

- 如何快速计算出每一个虚拟block的$G_{i-1:-1}$
- 即$G_{0:-1}$，$G_{1:-1}$，$G_{2:-1}$，……，$G_{n-2:-1}$，叫前缀(prefixes)



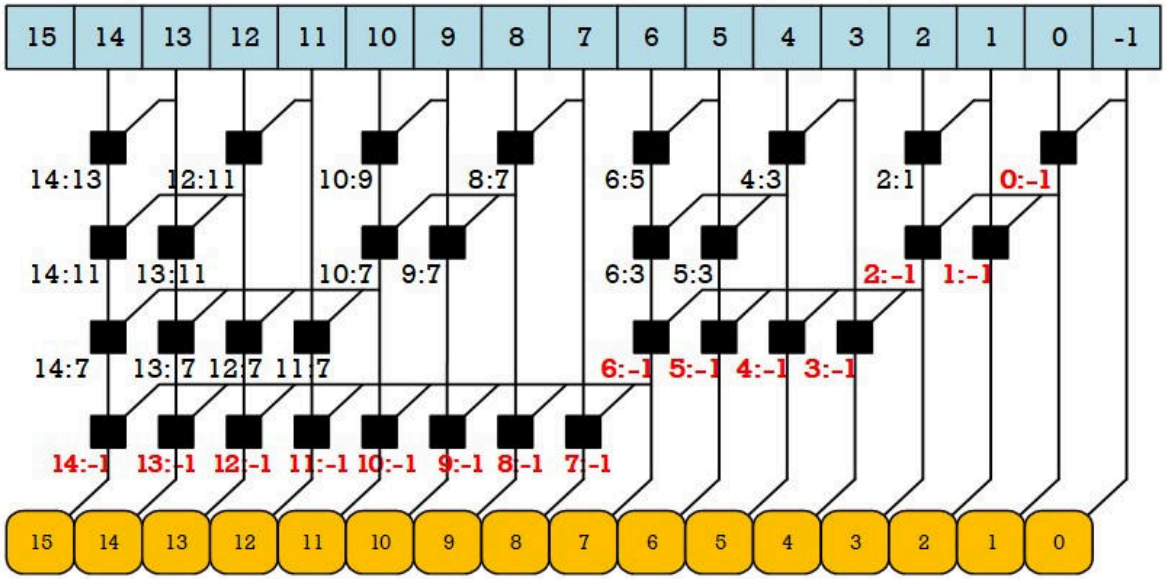
其实对于任意区间的虚拟block(如第i:j列)，我们都可以将它看成由两部分组成，即第i:k列和第k-1:j列，如果已知每部分的G和P($G_{i:k}$,$P_{i:k}$,$G_{k-1:j}$,$P_{k-1:j}$)，则

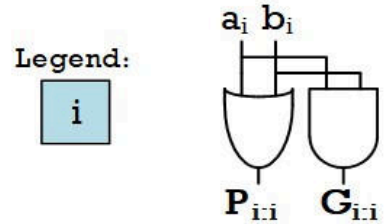
$$G_{i:j} = G_{i:k} + P_{i:k}\, G_{k-1:j}$$

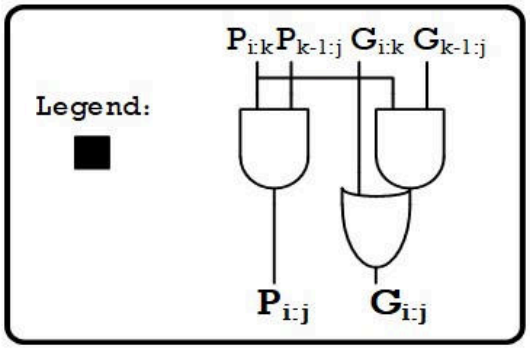
$$P_{i:j} = P_{i:k} P_{k-1:j}$$

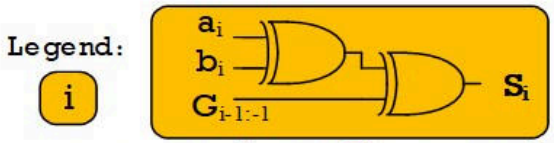
雨课堂 Rain Classroom

前缀(Prefix)进位 多位加法器







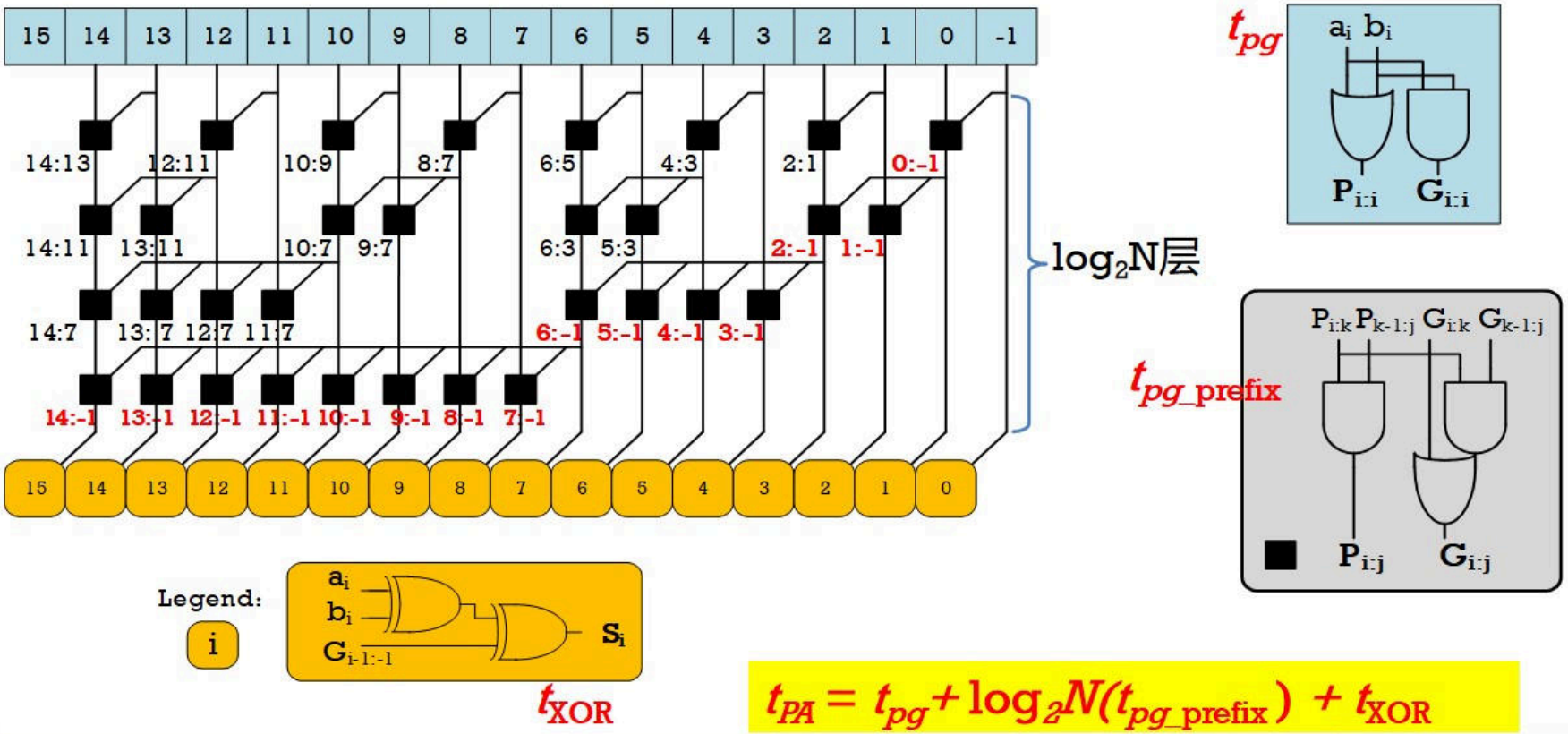


$$G_{i:j} = G_{i:k} + P_{i:k}\,G_{k-1:j}$$

$$P_{i:j} = P_{i:k}P_{k-1:j}$$

$$s_i = a_i \oplus b_i \oplus C_i = a_i \oplus b_i \oplus G_{i-1:-1}$$

前缀(Prefix)进位 多位加法器



$$t_{PA} = t_{pg} + \log_2 N(t_{pg_prefix}) + t_{XOR}$$

三种多位加法器的传输延迟对比

- 比较: 32-bit 行波进位, 先行进位, 前缀进位的传输延迟
  - 假设分块是 4-bit 一个模块
  - 2-input 逻辑门延迟 = 100 ps; 1-bit全加器延迟 = 300 ps

Adder Delay Comparisons

- 比较: 32-bit 行波进位, 先行进位, 前缀进位的传输延迟，假设分块是 4-bit 一个模块
  - 2-input 逻辑门延迟 = 100 ps; 1-bit全加器延迟 = 300 ps
- 行波进位： $t_{ripple} = Nt_{FA} = 32(300\text{ ps}) = 9.6\text{ ns}$
- 先行进位： $t_{CLA} = t_{pg} + t_{pg_block} + (N/k - 1)t_{AND_OR} + kt_{FA}$

  $= [100 + 600 + (7)200 + 4(300)]\text{ ps}$

  $= 3.3\text{ ns}$
- 前缀进位： $t_{PA} = t_{pg} + \log_2 N(t_{pg_prefix}) + t_{XOR}$

  $= [100 + \log_2 32(200) + 100]\text{ ps}$

  $= 1.2\text{ ns}$

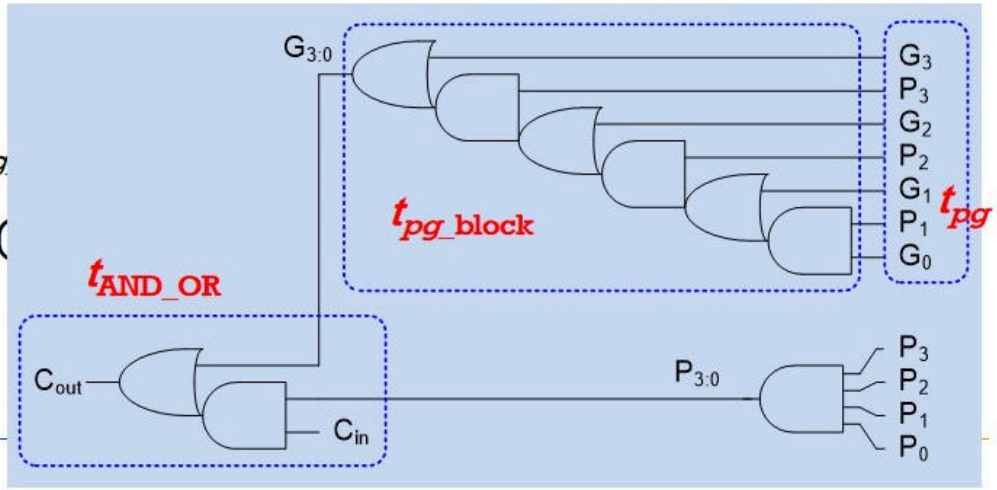
Adder Delay Comparisons

- 比较: 32-bit 行波进位, 先行进位, 前缀进位的传输延迟
  - 2-input 逻辑门延迟 = 100 ps; 1-bit全加器延迟 = 300 ps
- 行波进位: $t_{ripple} = Nt_{FA} = 32(300\ ps) = 9.6\ ns$
- 先行进位: $t_{CLA} = t_{pg} + t_{pg_block} + (N/k - 1)t_{AND_OR} + kt_{FA} = [100 + 600 + (7)200 + 4(300)]ps$

  $= 3.3\ ns$
- 前缀进位: $t_{PA} = t_{pg} + \log_2 N(t_{pg}$

  $= [100 + \log_2 32($

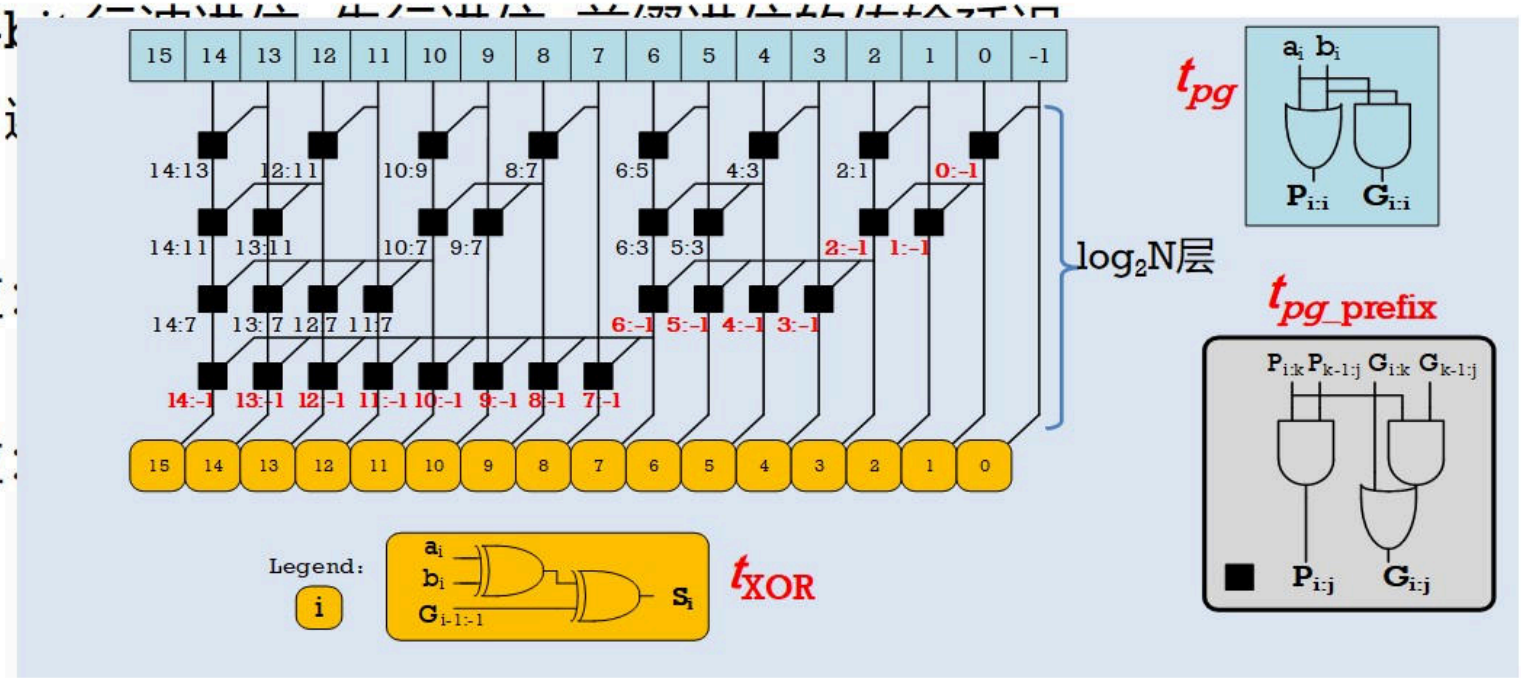
  $= 1.2\ ns$



NEU-SWC-A0801040040

Adder Delay Comparisons

- 比较: 32-b
  - 2-input
- 行波进位
- 先行进位



- 前缀进位: $t_{PA} = t_{pg} + \log_2 N(t_{pg_prefix}) + t_{XOR} = [100 + \log_2 32(200) + 100]\text{ps} = 1.2\text{ ns}$

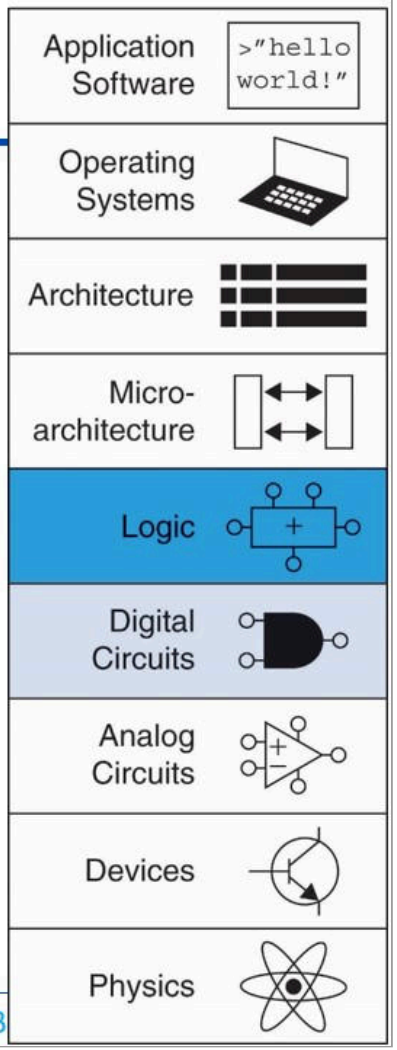
第十三堂课结束了

有问题同学们可以提问讨论

雨课堂 Rain Classroom

Chapter 5 :: Topics

- 引言
- **算术运算电路**
  - **多位加法器（1位→多位→带溢出判断→减法器→加减法器）**
  - **算术逻辑单元 ALU**
  - **更快速的多位加法器（行波进位→先行进位→前缀进位）**
  - **移位器→乘法器/除法器**
- 数制系统（重点介绍浮点数）
- 时序电路模块
- 存储器阵列

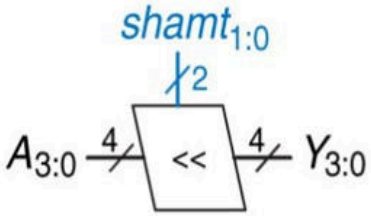


移位器 Shifters

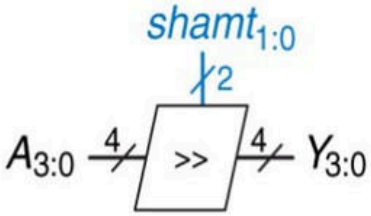
- 逻辑移位:
  - Ex: 11001 >> 2 = 00110
  - Ex: 11001 << 2 = 00100
- 算数移位:
  - Ex: 11001 >>> 2 = 11110
  - Ex: 11001 <<< 2 = 00100
- 循环移位:
  - Ex: 11001 ROR 2 = 01110
  - Ex: 11001 ROL 2 = 00111

移位器设计

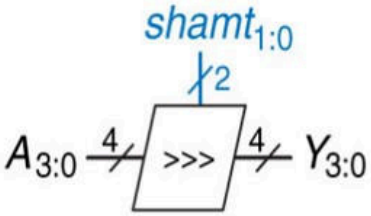
- 功能符号



(a)左移



(b) 逻辑右移



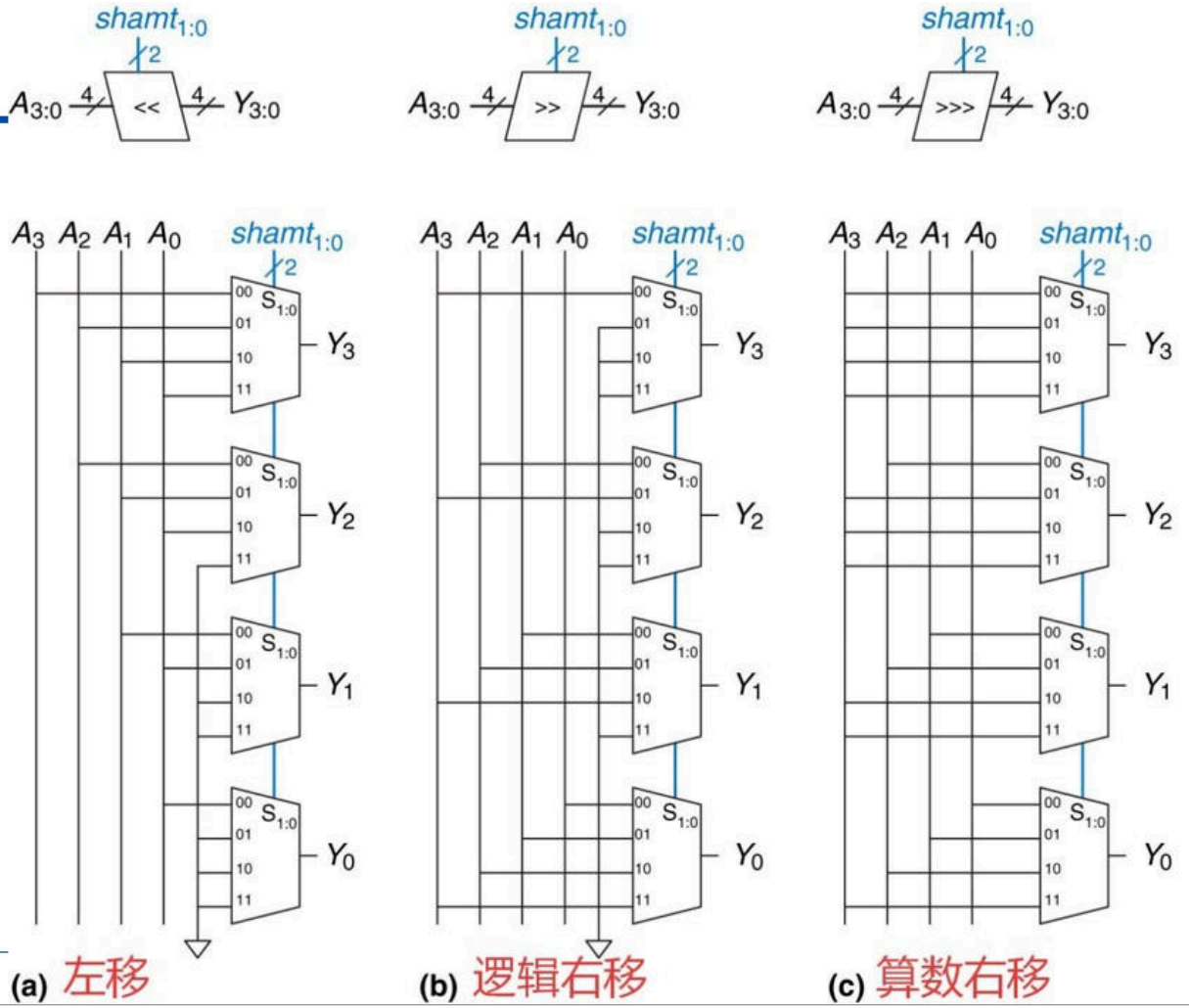
(c) 算术右移

- 左移：不分逻辑左移还是算术左移，除非溢出了，否则符号不会变

雨课堂 Rain Classroom

移位器设计

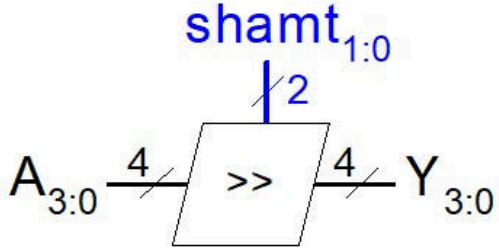
- 一个N位的移位器可以用 N 个 N:1 多路选择器构成。



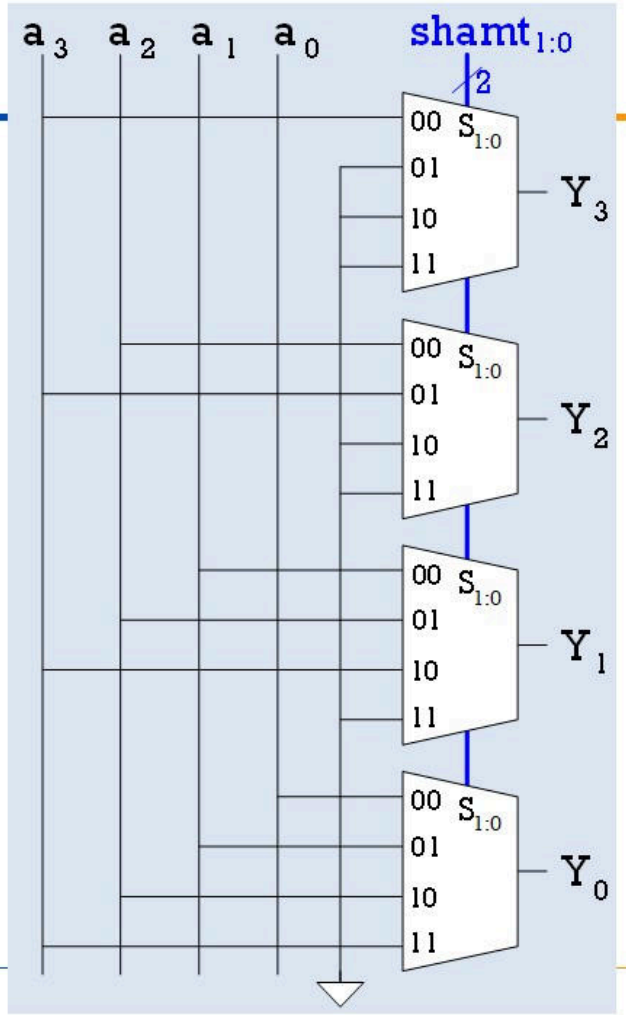
NEU-SWC-A0801040040

移位器设计

- 4-bit逻辑右移器的实现
  - 使用4:1多路选择器功能



功能符号



移位器 可以作为 乘法器，除法器

- 左移是乘法的特例，*N*位左移相当于对一个数乘以2^N倍

  $A << N = A \times 2^N$，例如：

  - 00001 << 2 = 00100 $(1 \times 2^2 = 4)$
  - 11101 << 2 = 10100 $(-3 \times 2^2 = -12)$

- 算术右移是除法的特例，*N*位的算术右移相当于对一个数除以2^N

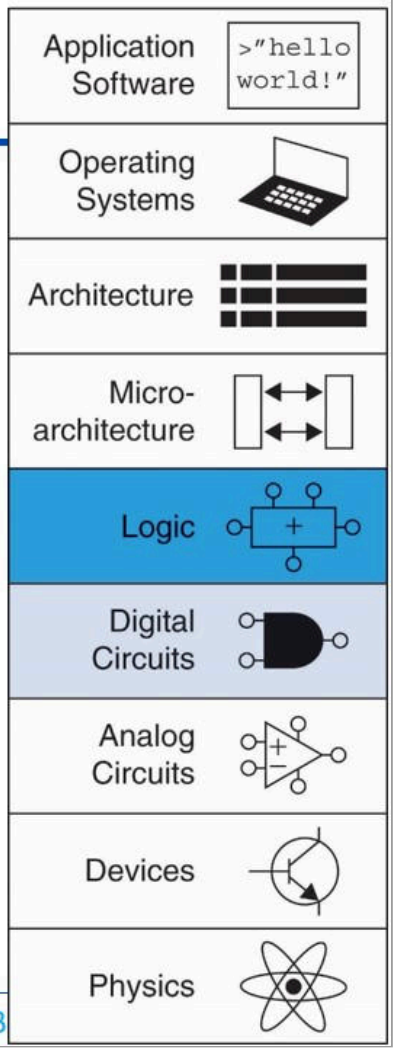
  $A >>> N = A \div 2^N$，例如：

  - 01000 >>> 2 = 00010 $(8 \div 2^2 = 2)$
  - 10000 >>> 2 = 11100 $(-16 \div 2^2 = -4)$

雨课堂 Rain Classroom

Chapter 5 :: Topics

- 引言
- **算术运算电路**
  - **多位加法器（1位→多位→带溢出判断→减法器→加减法器）**
  - **算术逻辑单元 ALU**
  - **更快速的多位加法器（行波进位→先行进位→前缀进位）**
  - **移位器→乘法器/除法器**
- 数制系统（重点介绍浮点数）
- 时序电路模块
- 存储器阵列



多位乘法器(Multipliers)设计

- 无符号二进制的乘法和十进制乘法很相似，只不过它每一位只有1和0而已。

十进制 230 × 42 = 9660

```
   230
 ×  42
 ----
   460
+ 920
 ----
  9660
```

被乘数
乘数
部分积

乘积结果

二进制 5 × 7 = 35

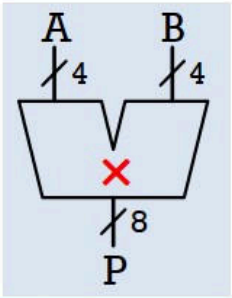
```
    0101
  × 0111
 ------
    0101
   0101
  0101
+ 0000
 -------
  0100011
```

- 部分积是由**一位乘数**与被乘数**相乘**构成的。
- **左移**部分积并**累加**得到最终乘积结果

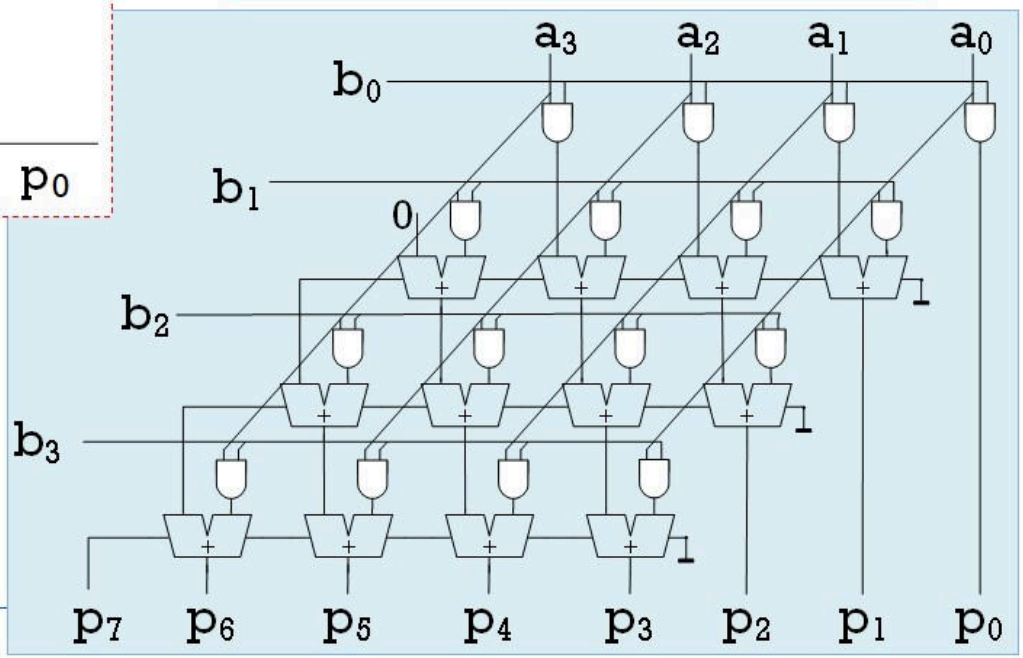
4 x 4 乘法器设计

$$
\begin{array}{rrrrrrrr}
 & & & a_3 & a_2 & a_1 & a_0 & \\
 & & & b_3 & b_2 & b_1 & b_0 & \\
\hline
 & & & a_3b_0 & a_2b_0 & a_1b_0 & a_0b_0 & \\
 & & a_3b_1 & a_2b_1 & a_1b_1 & a_0b_1 & & \\
 & a_3b_2 & a_2b_2 & a_1b_2 & a_0b_2 & & & \\
a_3b_3 & a_2b_3 & a_1b_3 & a_0b_3 & & & & \\
\hline
p_7 & p_6 & p_5 & p_4 & p_3 & p_2 & p_1 & p_0
\end{array}
$$

功能符号



加法器阵列方式实现，是组合逻辑电路



NEU-SWC-A0801040040

多位原码乘法器(Sign-Mag Multipliers)设计

➢ 分析原码乘法器的基本算法

- 假设计算 0.1101×1.1011

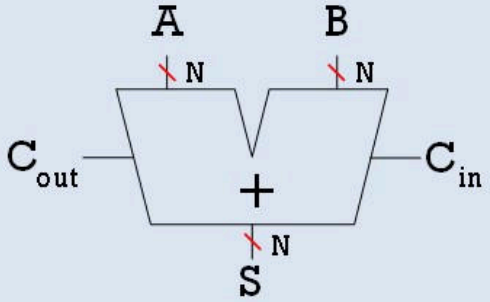
```
      0.1101
×     0.1011
-------------
       1101      ——— 部分积
      1101
     0000
+   1101
-------------
  .10001111      ——— 所有部分积相加
```

附上符号: 1.10001111

如果硬件实现会遇到的困难:

① 加法器都是双操作数，加数增多

② 加法器的位数是固定的，位数增多



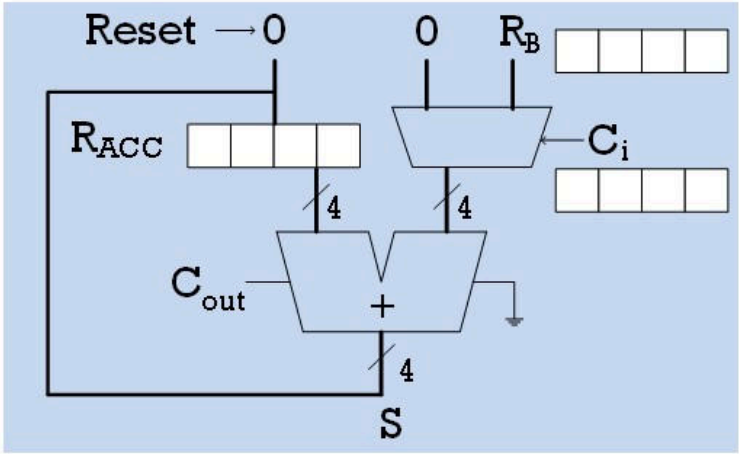
N-bit加法器符号图

多位原码乘法器(Sign-Mag Multipliers)设计

➢ 分析原码乘法器的基本算法

- 假设计算 0.1101×1.1011

```
        0.1101   ——寄存器 B
×       0.1011   ——寄存器 C
-----------------
          1101   ——累加器 ACC
         1101
        0000
+      1101
-----------------
      .10001111
```



附上符号: 1.10001111

① 解决加数多的问题

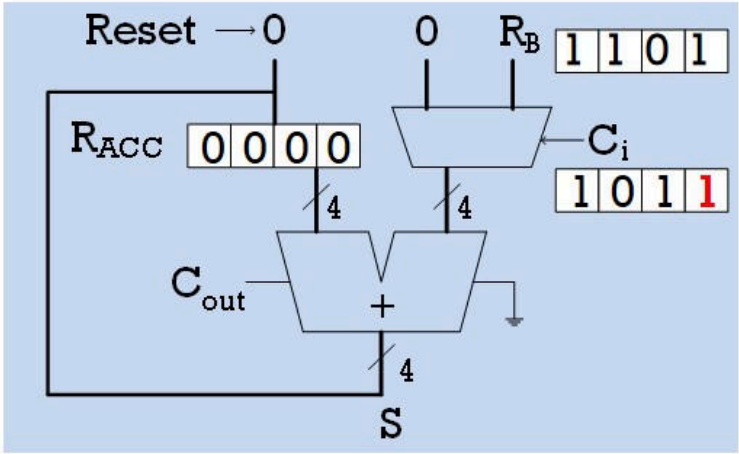
雨课堂 Rain Classroom

多位原码乘法器(Sign-Mag Multipliers)设计

➢ 分析原码乘法器的基本算法

- 假设计算 0.1101×1.1011

```
      0.1101   ——寄存器 B
×     0.1011   ——寄存器 C
-------------
       1101    ——累加器 ACC
      1101
     0000
+   1101
-------------
  .10001111
```



附上符号: 1.10001111

② 解决加法器位数多的问题

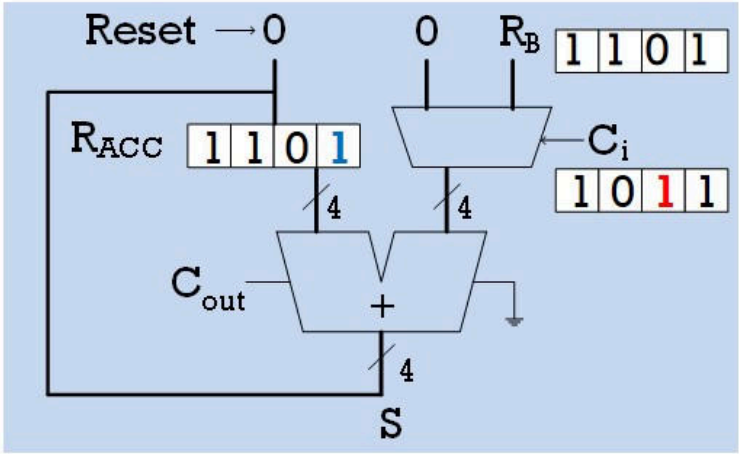
雨课堂 Rain Classroom

多位原码乘法器(Sign-Mag Multipliers)设计

➢ 分析原码乘法器的基本算法

- 假设计算 0.1101×1.1011

```
      0.1101   ——寄存器 B
×     0.1011   ——寄存器 C
-------------
       1101    ——累加器 ACC
      1101
     0000
+   1101
-------------
  .10001111
```



附上符号: 1.10001111

② 解决加法器位数多的问题

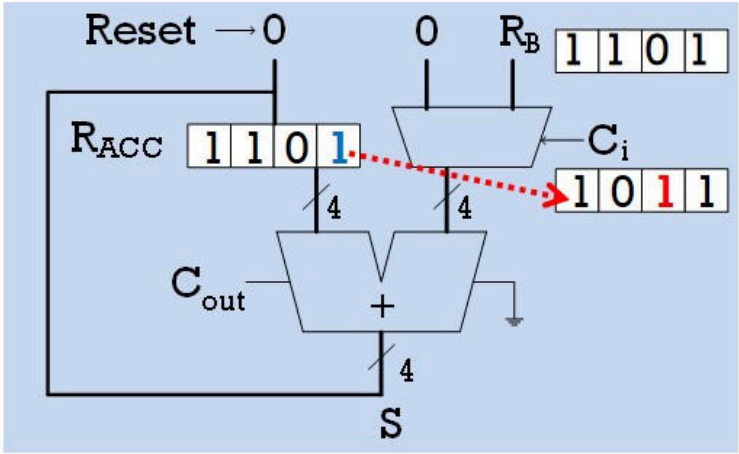
雨课堂 Rain Classroom

多位原码乘法器(Sign-Mag Multipliers)设计

➢ 分析原码乘法器的基本算法

- 假设计算 0.1101×1.1011

```
      0.1101   ——寄存器 B
×     0.1011   ——寄存器 C
-------------
       1101    ——累加器 ACC
      1101
     0000
+   1101
-------------
  .10001111
```



附上符号: 1.10001111

② 解决加法器位数多的问题

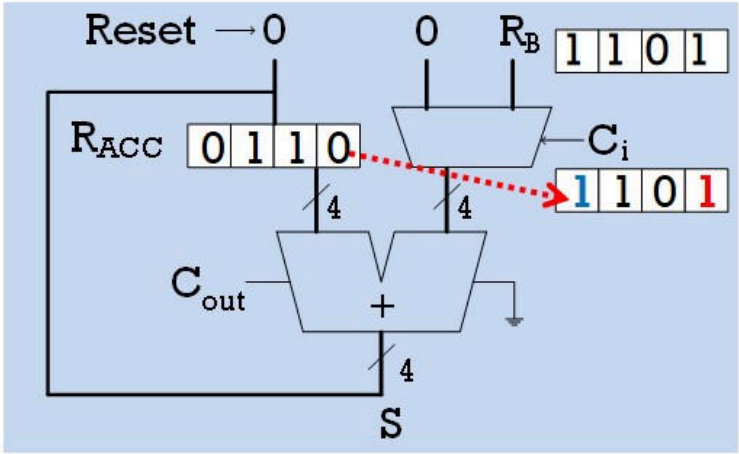
雨课堂 Rain Classroom

多位原码乘法器(Sign-Mag Multipliers)设计

➢ 分析原码乘法器的基本算法

- 假设计算 0.1101×1.1011

```
      0.1101   ——寄存器 B
×     0.1011   ——寄存器 C
-------------
       1101    ——累加器 ACC
      1101
     0000
+   1101
-------------
  .10001111
```



附上符号: 1.10001111

② 解决加法器位数多的问题

雨课堂 Rain Classroom

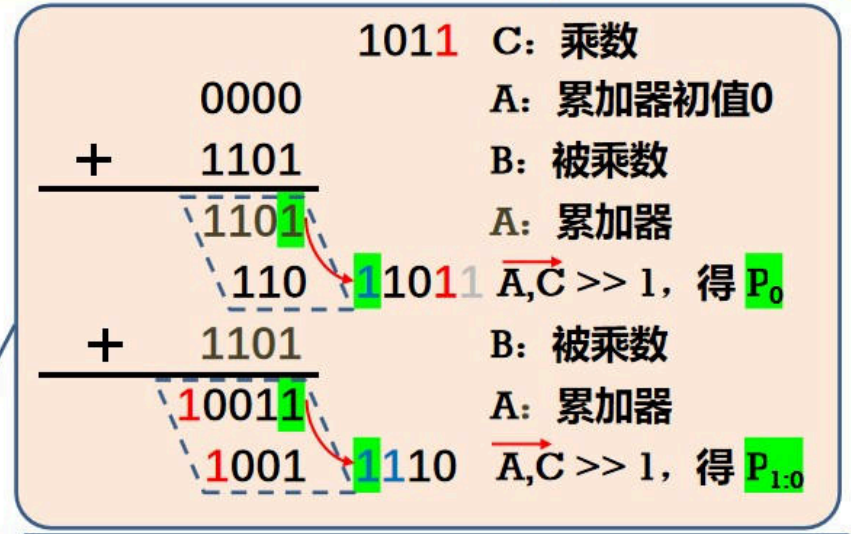
多位原码乘法器(Sign-Mag Multipliers)设计

➢ 分析原码乘法器的基本算法

- 假设计算 0.1101×1.1011

```
      0.1101   ——寄存器 B
  ×   0.1011   ——寄存器 C
 ------------
      1101    ——累加器 ACC
     1101
    0000
  + 1101
 ------------
  .10001111
```

附上符号: 1.10001111



1. 将累加器右移，与B相加，等同于将B左移，与累加器相加，解决B左移位数增多问题；
2. 将累加器不再参与运算的低位，存储到乘数不再参与判断的位数中，解决累加器位数增多的问题；
3. 新的部分积有可能出现溢出，采用双符号位，暂时存储溢出的位，右移之后假溢出就解决了。

NEU-SWC-A0801040040

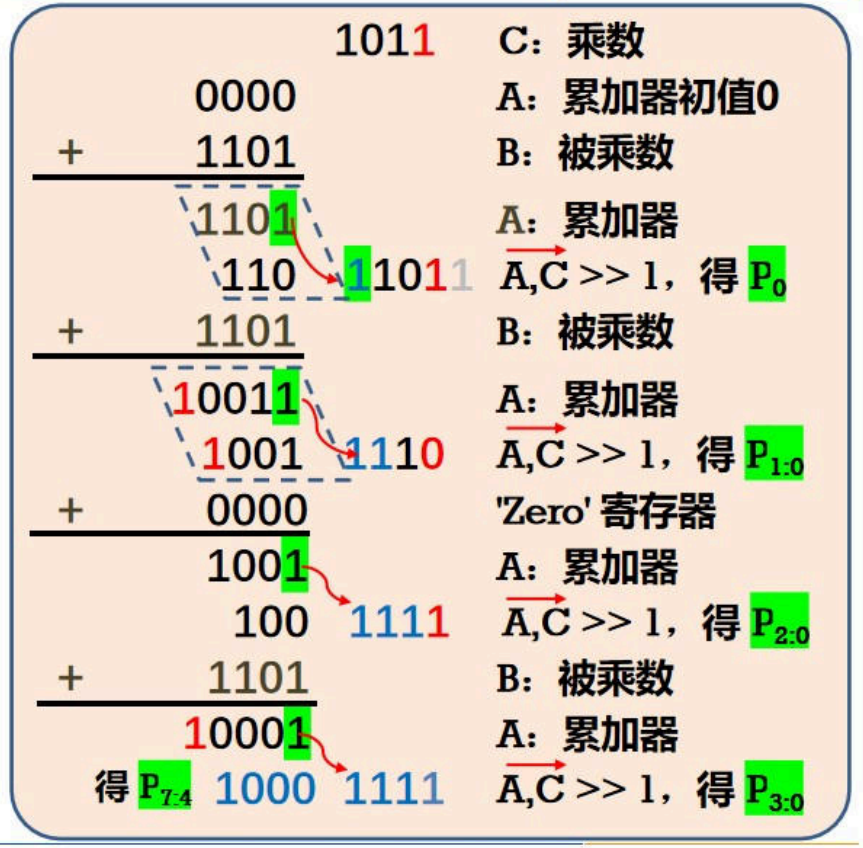
多位原码乘法器(Sign-Mag Multipliers)设计

➢ 分析原码乘法器的基本算法

- 假设计算 0.1101×1.1011

```
      0.1101   ——寄存器 B
×     0.1011   ——寄存器 C
-------------
      1101     ——累加器 ACC
     1101
    0000
+  1101
-------------
  .10001111
```

附上符号: 1.10001111



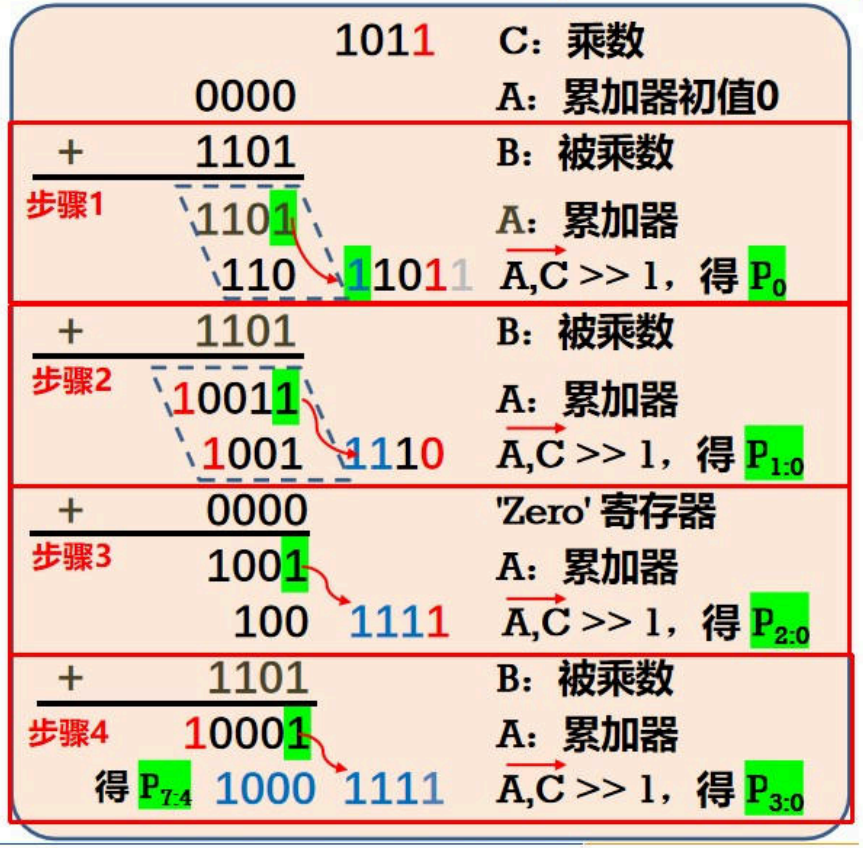
多位原码乘法器(Sign-Mag Multipliers)设计

➢ 分析原码乘法器的基本算法

- 假设计算 0.1101×1.1011

```
      0.1101   ——寄存器 B
×     0.1011   ——寄存器 C
-------------
      1101     ——累加器 ACC
     1101
    0000
+  1101
-------------
  .10001111
```

附上符号: 1.10001111

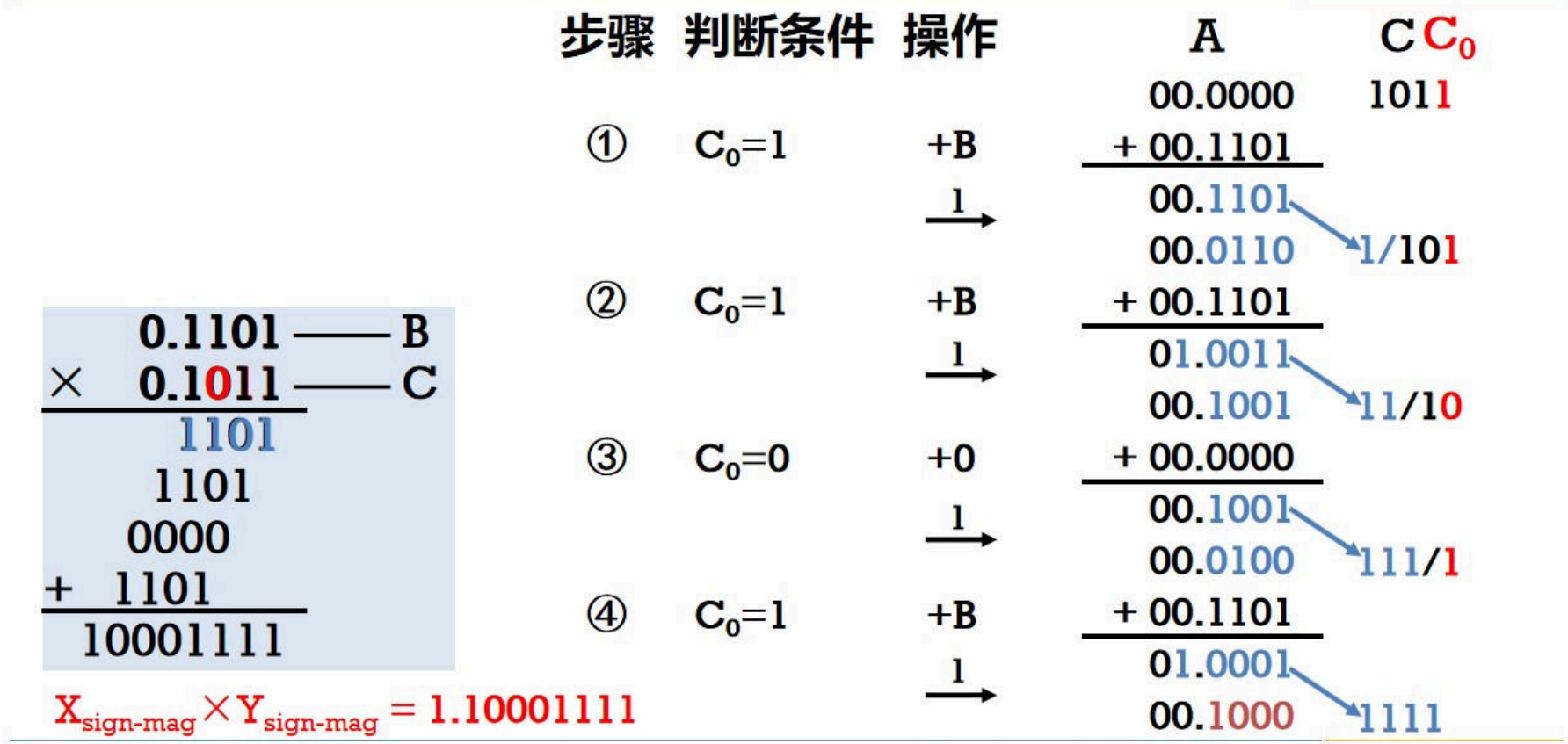


多位原码乘法器(Sign-Mag Multipliers)设计

- 硬件实现
  - 设置寄存器：
    - A：累加器，存储部分积的累加和，也是乘积结果的高位部分
    - B：存储被乘数
    - C：存储乘数，也是乘积结果的低位部分
  - 寄存器设初值：
    - A = 00.0000　　// 双符号位，初始化为0，存储部分积的累加和
    - B = |X| = 00.1101　　// 双符号位，初始化为被乘数绝对值
    - C = |Y| = .1011　　// 初始化为乘数绝对值

雨课堂 Rain Classroom

多位原码乘法器(Sign-Mag Multipliers)设计



多位原码乘法器(Sign-Mag Multipliers)设计

- 运算规则

(1) 操作数、结果均采用原码（sign-mag）表示；

(2) 按绝对值进行运算，符号位单独处理；

(3) 累加器 A (部分积累加和)和寄存器 B (被乘数)设置为**双符号位**；

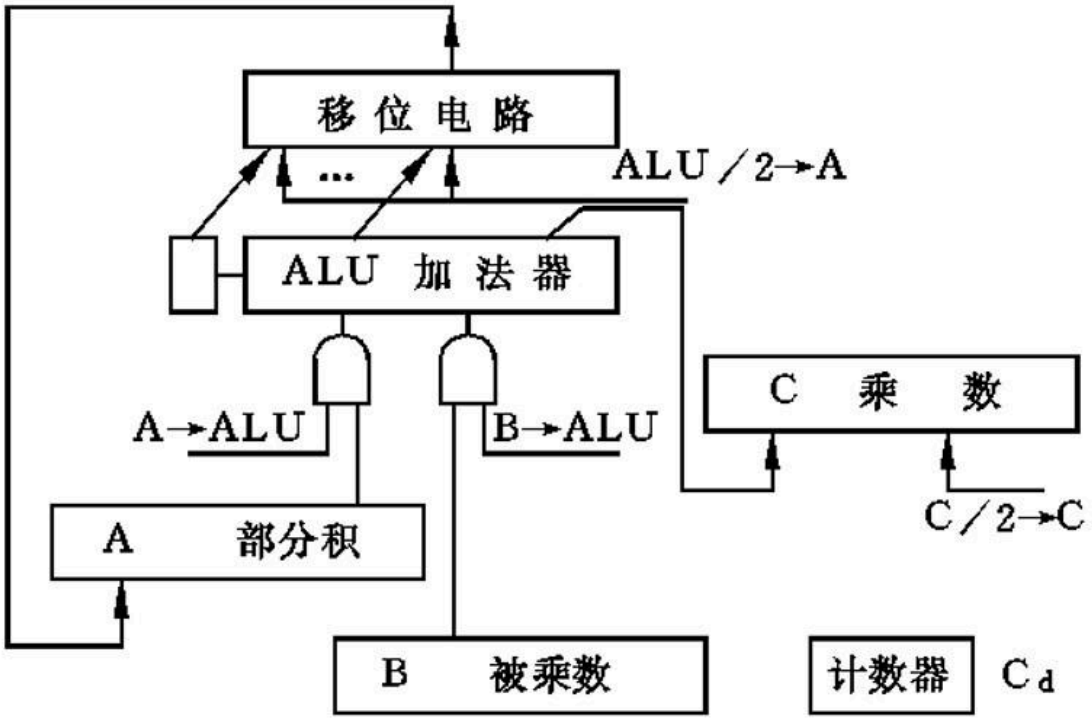
(4) 寄存器 C 的最低位 C_0 决定下一步的操作（+B/+0）；

(5) 循环 n 步（加部分积、右移一位）

雨课堂 Rain Classroom

多位原码乘法器(Sign-Mag Multipliers)设计

- 逻辑实现



更快速的多位原码乘法器(Sign-Mag Multipliers)设计

- 运用2-bits乘数的部分积，原则上应该执行效率提升一倍
- 对比分析如下：

```
    0.111111
  × 0.011110
  ----------
    000000
   111111
  111111
 111111
111111
+ 000000
```

```
    0.111111
  × 0.011110
  ----------
   1111110      A+2X, A>>2
 ???????       A+3X, A>>2
+ 111111        A+X, A>>2
```

更快速的多位原码乘法器(Sign-Mag Multipliers)设计

```
    0.1111111
  × 0.0111110
  ----------
    1111110
   ???????
+  111111
  ----------
```

$C_{i+1}C_i$	部分积	add up, shift
0 0 (0)	0	(A + 0) >> 2
0 1 (1)	X	(A + X) >> 2
1 0 (2)	2X	(A+2X) >> 2
1 1 (3)	3X	(A+3X) >> 2

怎样实现 +3X 操作?

$$(A+3X)=\begin{cases}\text{方案A}: & (A+2X+X)=(A+2X)+X\\ \text{方案B}: & (A-X+4X)=(A-X)+4X\end{cases}$$

慕课堂练习

更快速的多位原码乘法器(Sign-Mag Multipliers)设计

- 怎样实现 +3X 操作?　　(A+3X) = (A-X)+4X

```
        0.111111
      × 0.011110
  ①  ---------- + 1111110 ---------- ①
  ②  ----------  - 111111 ---------- ②  欠账触发器 C_J = 1
                + 11111100
  ③  ---------- + 111111 ---------- ③  A+2X
```

欠账触发器 C_J = { 0: 没欠账
1: 欠账，下一步要多做一个“+X”操作

更快速的多位原码乘法器(Sign-Mag Multipliers)设计

- 怎样实现 +3X 操作?　　(A+3X) = (A-X)+4X

```
    0.111111
  × 0.001110
  ------------
    + 1111110   ----------①
    - 111111    ----------②  欠账触发器 C_J = 1
  + 11111100
  + 000000      ----------③  A+X,  C_J = 0
  ------------
```


雨课堂 Rain Classroom

更快速的多位原码乘法器(Sign-Mag Multipliers)设计

- 怎样实现 +3X 操作?　　(A+3X) = (A-X)+4X

```
   0.111111
 × 0.011110
 ----------
  + 1111110 ----------①
  - 111111  ----------②  欠账触发器 C_J = 1
+ 11111100
+ 111111  ----------③  A+2X，C_J = 0
 ----------
```


雨课堂 Rain Classroom

更快速的多位原码乘法器(Sign-Mag Multipliers)设计

- 怎样实现 +3X 操作?　　(A+3X) = (A-X)+4X

```
    0.111111
  × 0.101110
  ------------
  + 1111110   ----------①
  - 111111    ----------②  欠账触发器 C_J = 1
+ 11111100
+ 1111110     ----------③  A+3X→A-X,  C_J = 1
  ------------
```


雨课堂 Rain Classroom

更快速的多位原码乘法器(Sign-Mag Multipliers)设计

- 怎样实现 +3X 操作?　　(A+3X) = (A-X)+4X

```
    0.111111
  × 0.111110
  ----------
    + 1111110   ----------①
    - 111111    ----------②  欠账触发器 Cj = 1
  + 11111100
  + --3X--      ----------③  A+4X→A+0, Cj = 1
  ----------
```


雨课堂 Rain Classroom

更快速的多位原码乘法器(Sign-Mag Multipliers)设计

C_{i+1}	C_i	C_J	Operation	
0	0	0	(A+ 0) >>2	0 → C_J
0	0	1	(A+ X) >>2	0 → C_J
0	1	0	(A+ X) >>2	0 → C_J
0	1	1	(A+2X) >>2	0 → C_J
1	0	0	(A+2X) >>2	0 → C_J
1	0	1	(A - X) >>2	1 → C_J
1	1	0	(A - X) >>2	1 → C_J
1	1	1	(A+ 0) >>2	1 → C_J

更快速的多位原码乘法器

慕课堂练习

- 课堂练习1：$X_{sign\text{-}mag}=1.111111$，$Y_{sign\text{-}mag}=0.111001$，$(XY)_{sign\text{-}mag}$?
  - 初始值：
    - A = 000.000000　　// 累加器，部分积累加和
    - B = |X| = 000.111111　　// 被乘数
    - DB = 001.111110　　// 2倍 被乘数B
    - NB = 111.000001　　// 被乘数B的相反数，即-B
    - C = |Y| = .111001　　// 乘数
    - $C_J = 0$　　// 欠账触发器

雨课堂 Rain Classroom

更快速的多位原码乘法器(Sign-Mag Multipliers)设计

```
    0.111111 —— B
×   0.111001 —— C
-----------------
      111111
   1111110
+ --3X--
-----------------
   10001111
```

步骤	判断条件 $C_1C_0C_j$	操作	A	C C_1C_0 C_j
			000.000000	111001 0
①	0 1 0	+B	+ 000.111111	
		2 →	000.111111	
			000.001111	11/1110 0
②	1 0 0	+2B	+ 001.111110	
		2 →	010.001101	
			000.100011	0111/11 0
③	1 1 0	-B	+ 111.000001	
		2 →	111.100100	
			111.111001	000111/ 1
+1	0 0 1	+B	+ 000.111111	
		return	000.111000	000111

$X_{sign\text{-}mag} \times Y_{sign\text{-}mag} = 1.111000000111$

雨课堂 Rain Classroom

更快速的多位原码乘法器(Sign-Mag Multipliers)设计

课堂练习2:

$X_{sign\text{-}mag}=0.00111$,

$Y_{sign\text{-}mag}=1.01001$,

$(XY)_{sign\text{-}mag}$?

```
      0.001110 ——B
×     0.010010 ——C
      0011100
     000000
+ 001110
000011111100
```

步骤	判断条件 $C_1C_0C_j$	操作	A(6bits)	C C_1C_0 C_j
			000.000000	010010 0
①	10 0	+2B	+ 000.011100	
		2 →	000.011100	
			000.000111	00/0100 0
②	00 0	+0	+ 000.000000	
		2 →	000.000111	
			000.000001	1100/01 0
③	01 0	+B	+ 000.001110	
		2 →	000.001111	
			000.000011	111100/ 0

$X_{sign\text{-}mag} \times Y_{sign\text{-}mag} = 1.0000111111$

雨课堂 Rain Classroom

更快速的多位原码乘法器(Sign-Mag Multipliers)设计

- 运算规则

(1) 操作数、结果均采用**原码**（sign-mag）表示，乘数C的数值位必须是**偶数**；

(2) 按绝对值进行运算，符号位单独处理；

(3) 累加器 A (部分积累加和)和寄存器 B (被乘数)设置为**三符号位**；

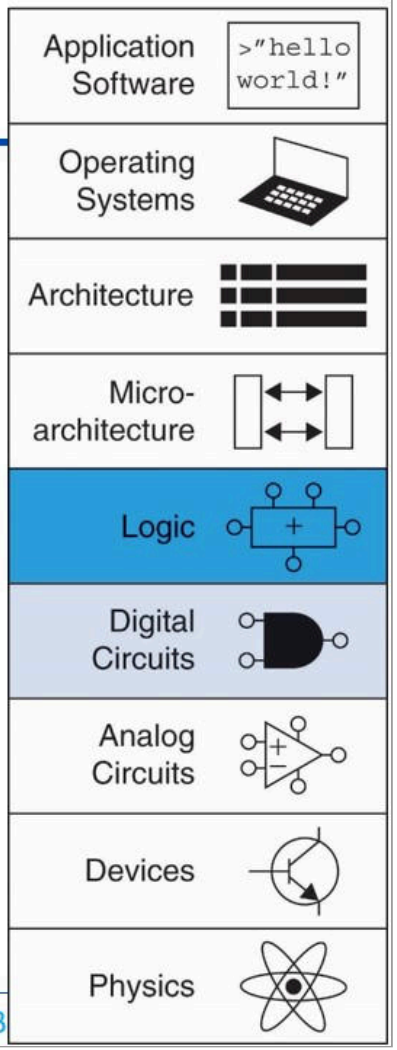
(4) 欠账触发器 C_J 的初值是 0. 它与乘数C的最低两位决定下一步的操作，但**欠账触发器不参与移位**

(5) 循环 $\frac{1}{2}$n 步. 有可能最终需要额外的一个步骤，即 **还账操作，该操作不再移位**.

雨课堂 Rain Classroom

Chapter 5 :: Topics

- 引言
- **算术运算电路**
  - **多位加法器（1位→多位→带溢出判断→减法器→加减法器）**
  - **算术逻辑单元 ALU**
  - **更快速的多位加法器（行波进位→先行进位→前缀进位）**
  - **移位器→乘法器/除法器**
- 数制系统（重点介绍浮点数）
- 时序电路模块
- 存储器阵列



4 x 4 除法器设计

- A/B = Q + R/B
  - 其中，A,B都是N位无符号数
  - Q是商，R是余数
- 算法:

R' = 0

for i = N-1 to 0

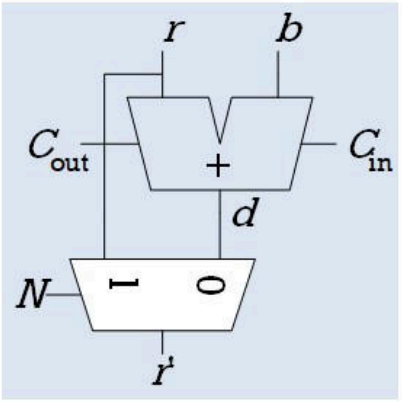
R = { R'<<1, A_i }

D = R - B

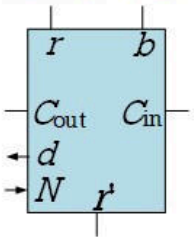
if D < 0, then Q_i=0, R' =R

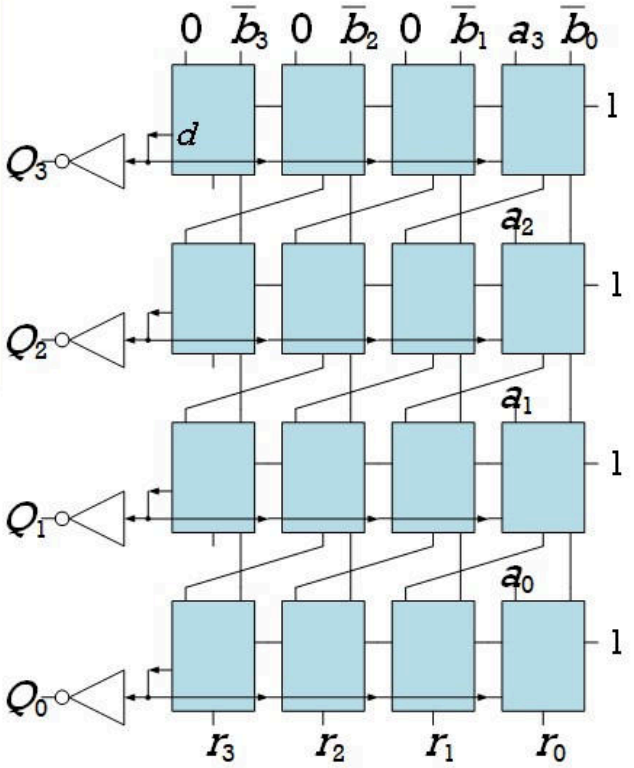
else Q_i=1, R' =D

R = R'



功能符号





多位原码除法器(Sign-Mag Divider)设计

1. 恢复余数法算法分析

例如计算 0.10110÷0.11111

商： 0.10110

余数：0.10110×2^{-5}

算法关键： 是否余数大于除数?

- 使用减法操作来判断

$$2\times R-\text{divisor}=R'\begin{cases}\text{正数：}\ Q_i=1\\ \text{负数：}\ Q_i=0,\ \text{恢复余数}\end{cases}$$

```
                0.10110
0.11111 ) 0.1011000
          - 11111
            11010
          - 11111
             -101
          + 11111
            110100
          -  11111
            101010
          -  11111
             10110
          -  11111
             -1001
          +  11111
0.00000 10110
```


雨课堂 Rain Classroom

多位原码除法器(Sign-Mag Divider)设计

1. 恢复余数法

- 算法示例: X = - 0.10110, Y = 0.11111, X/Y ?
  - 寄存器设初值:
    - A = |X| = 00.10110 // 被除数，也是余数
    - B = |Y| = 00.11111 // 除数
    - NB = 11.00001 // 除数B的相反数，即-B
    - C = |Q| = 0.00000 // 商

多位原码除法器(Sign-Mag Divider)设计

步骤	判断条件	操作	A(x)		C(Q)Cn	
	Sf		00.10110	r0	0.00000	
①		←1	01.01100	2r0		
		-B	+11.00001			
②	0		00.01101	r1	0.00001	Q1
		←1	00.11010	2r1		
		-B	+11.00001			
	1		11.11011	r2'	0.00010	Q2
②+	恢复余数	+B	+00.11111			
			00.11010	r2		
③		←1	01.10100	2r2		
		-B	+11.00001			
	0		00.10101	r3	0.00101	Q3
④		←1	01.01010	2r3		
		-B	+11.00001			
	0		00.01011	r4	0.01011	Q4
⑤		←1	00.10110	2r4		
		-B	+11.00001			
	1		11.10111	r5'	0.10110	Q5
⑤+)	恢复余数	+B	+00.11111			
			00.10110	r5		

X÷Y=	Q	...	R
+	+	+	+
+	-	-	+
-	-	+	-
-	+	-	-

$Q = -0.10110$

$R = -0.10110 \times 2^{-5}$

多位原码除法器(Sign-Mag Divider)设计

1. 恢复余数法

- 运算规则:
  - (1) 被除数 A 和除数 B 都采用双符号位，且|X|< |Y|
  - (2) 最终结果**余数要乘以2^{-n}**, 其余数的符号与被除数相同
- 恢复余数法的缺点：耗时且效率低下.
- 故，通常采用不恢复余数法，也称为“**加减交替法**”

雨课堂 Rain Classroom

多位原码除法器(Sign-Mag Divider)设计

2. 不恢复余数法

① 算法分析

第 i 步: $2r_{i-1} - B = r_i'$ ，如果$r_i' < 0$，商上0

额外加1步: $r_i' + B = r_i$ **(恢复余数)**

第 i+1步: $2r_i - B = r_i + 1$，**依据r_{i+1} 的正负**，决定商上0/1

那么，可否不恢复余数，就可得到r_{i+1}，$r_{i+1} = 2r_i - B = 2(r_i' + B) - B = 2r_i' + B$

② 算法实施

- 如果第 i 步的余数 ri 是**正数**，则 本位商 Qi=1，下一步(第i+1步)操作: $2r_i - B$;
- 如果第 i 步的余数 ri 是**负数**，则 本位商 Qi=0，下一步(第i+1步)操作: $2r_i + B$

雨课堂 Rain Classroom

多位原码除法器(Sign-Mag Divider)设计

2. 不恢复余数法

- 算法示例: X = - 0.10110, Y = 0.11111, X/Y ?
  - 寄存器设初值:
    - A = |X| = 00.10110 // 被除数，也是余数
    - B = |Y| = 00.11111 // 除数
    - NB = 11.00001 // 除数B的相反数，即-B
    - C = |Q| = 0.00000 // 商

雨课堂 Rain Classroom

多位原码除法器(Sign-Mag Divider)设计

步骤	判断条件 Sf	操作	A(x)		C(Q)Cn	
			00.10110	r0	0.00000	
①		←1	01.01100	2r0		
		-B	+11.00001			
②	0		00.01101	r1	0.00001	Q1
		←1	00.11010	2r1		
		-B	+11.00001			
	1		11.11011	r2'	0.00010	Q2
③		←1	11.10110	2r2'		
		+B	+00.11111			
	0		00.10101	r3	0.00101	Q3
④		←1	01.01010	2r3		
		-B	+11.00001			
	0		00.01011	r4	0.01011	Q4
⑤		←1	00.10110	2r4		
		-B	+11.00001			
	1		11.10111	r5'	0.10110	Q5
⑤+)	恢复余数	+B	+00.11111	r5		
			00.10110			

X÷Y=	Q…	R
+	+	+
+	-	+
-	+	-
-	+	-

Q = -0.10110

R = -0.10110×2^{-5}

多位原码除法器(Sign-Mag Divider)设计

2. 不恢复余数法

- 运算规则:

(1) 被除数 A 和除数 B 都采用双符号位，且|X|< |Y|

(2) 余数的符号决定了每一次上商的值和下一步做什么操作(左移-除数/左移+除数)

(3) 循环 n 步得到 n-bits 的商；如果在第n步(即最后一步)，余数是负数，则需要再额外增加第 n+1 步，只恢复余数(即余数+除数)，不进行移位.

雨课堂 Rain Classroom

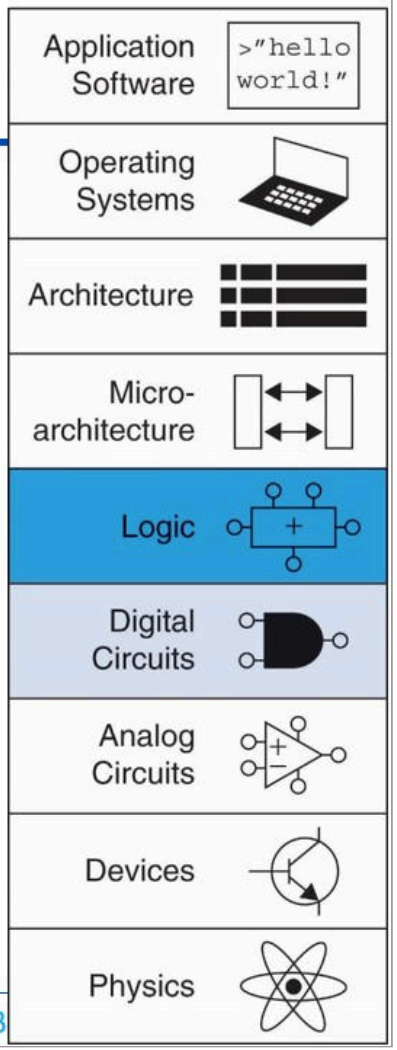
第十四堂课结束了

有问题同学们可以提问讨论

雨课堂 Rain Classroom

Chapter 5 :: Topics

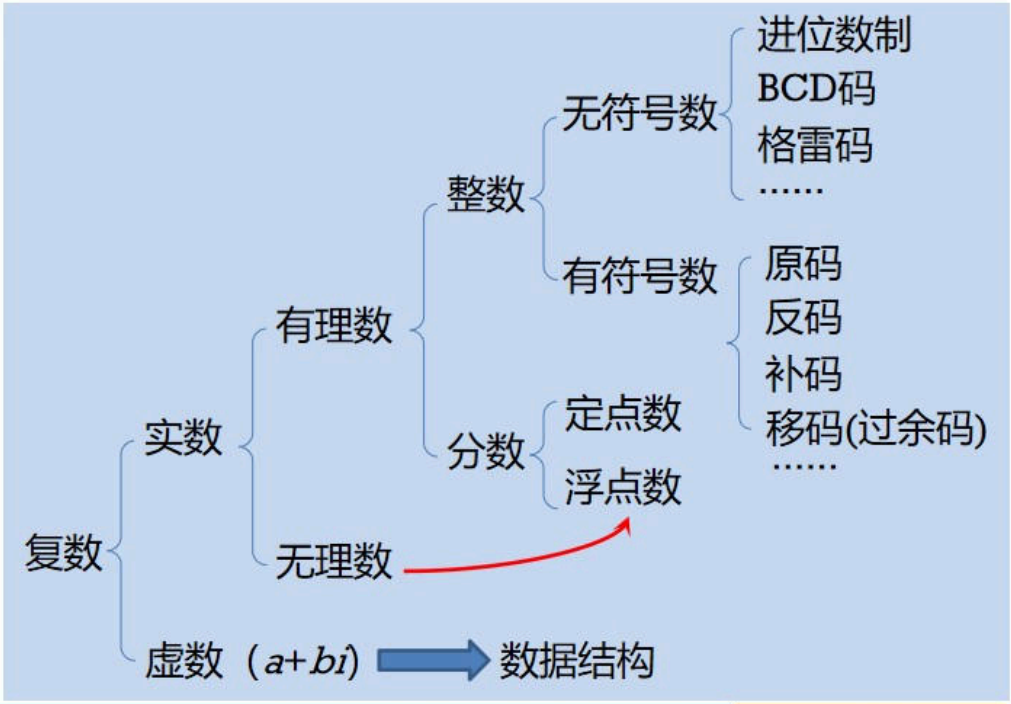
- 引言
- 算术运算电路
- **数制系统（重点介绍浮点数）**
- 时序电路模块
- 存储器阵列



数制系统

- 计算机可以对整数和小数操作，之前我们介绍了无符号整数、有符号整数及纯小数的表示（定点数）

- 那么分数怎么表示?
  - 数字都可以用二进制表示
  - 定点数
  - 浮点数



回忆：有符号的定点数

慕课堂练习

- 常用表示方法:
  - 原码 Sign/magnitude
  - 补码 Two's complement
- 例如:
  - 数字 -7.5_{10} 使用 8-bit 来表示，其中小数点隐含在高 4 位和低 4 位之间，最高位是符号位
    - 原码表示为:
    - 补码表示为:

雨课堂 Rain Classroom

浮点数Floating-Point Numbers表示

$1.011101101 \times 2^{1001110}$

$273 = 2.73 \times 10^2$

浮点数的小数点在最高位符号位的右侧，
类似于十进制的科学计数法

- 通常，一个浮点数会写成科学记数法形式：$\pm M \times B^E$

  其中：M = 尾数（mantissa），B = 基数（base），E = 阶码（exponent）

- 在上例中，M = 2.73, B = 10, E = 2

慕课堂提问?

对于某种表示，一个数的表示范围和一个数的精度如何体现?

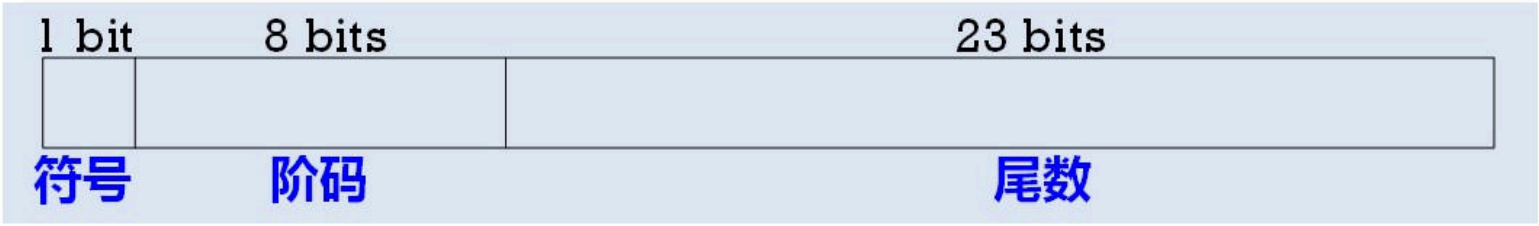
雨课堂 Rain Classroom

浮点数精度 Precision

- 单精度 Single-Precision:
  - 共 32-bit
  - 1-bit 符号, 8-bits 阶码, 23-bits 尾数

- 双精度 Double-Precision:
  - 共 64-bit
  - 1-bit 符号, 11-bits 阶码, 52-bits 尾数

浮点数Floating-Point Numbers表示



- 以实例说明浮点数的机器值如何表示:
  - 用 32-bit 表示十进制数值 228_{10} 的浮点数
- 我们通过三个版本的演化，来解释浮点数的最终表达形式
  - IEEE 754 浮点数表示的标准

浮点数表示 版本1

1. 将十进制数转换为二进制数 (步骤1和步骤2不能互换!):

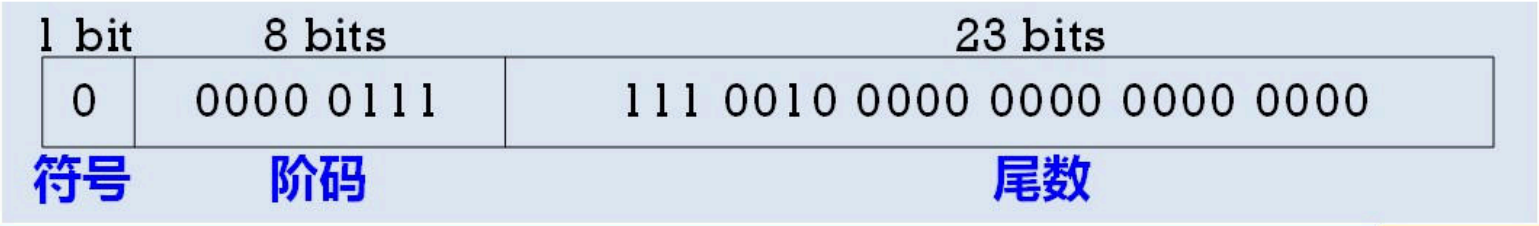
$$228_{10} = 11100100_2$$

2. 将数字写成 “二进制的科学记数法”：

$$11100100_2 = 1.11001_2 \times 2^7$$

3. 填写 32-bit 浮点数的各个字段:

- 符号位为正数（0）；8 位指数段表示数值 7；剩余 23 位为尾数。



浮点数表示 版本 2

- 尾数首位恒为 1：

$$228_{10} = 11100100_2 = 1.11001_2 \times 2^7$$

- 因此无需存储该位：**隐含尾数的首位 1**
- 仅将小数位存放至 23-bit 位字段中

1 bit	8 bits	23 bits
0	0000 0111	110 0100 0000 0000 0000 0000
符号	阶码	尾数

浮点数表示 版本 3

- 偏移指数：偏移量**bias = 127** (01111111_2)
  - 浮点数机器值的阶码 = 偏移量bias + 真实指数
  - 例如上述例子中，真实指数是 7： 127 + 7 = 134 = $0x10000110_2$
- 所以，**IEEE 754** 32-bit 浮点数(单精度浮点数)表达的 228_{10}

- 最终写成十六进制: 0x43640000

为什么浮点数的阶码使用偏移码表示？（慕课堂练习）

- 浮点数对阶
  - 两个浮点数相加，首先需要进行对阶（小数点对齐），也就是让两个浮点数的阶码部分相同

- 请问
  - 阶码小的向大的对齐
  - 还是阶码大的向小的对齐

偏移量取值

	X	$[X]_{补码}$	$[X]_{移码}$	
	decimal	binary	binary	decimal
精度的一半	-127	10000001	00000000	0
	-126	10000010	00000001	1
	-125	10000011	00000010	2
	------	------	------	------
	-1	11111111	01111110	126
	0	00000000	01111111	127
精度的另一半	1	00000001	10000000	128
	------	------	------	------
	126	01111110	11111101	253
	127	01111111	11111110	254
	128		11111111	255

IEEE 754 为什么偏移量要取127呢?

- exp的取值范围为[1,254];
- E作为最后小数的指数，它所表示的精度要对半分;
  - 非正数要占一半
  - 正数要占一半
- 既然exp只能取254个值
  - 自然最后E的范围只能为[-126,127]
  - 只有这样才能刚好对半分
- 根据Bias = exp-E，从而得到Bias的值为127;

浮点数表达 例题

- 求 -58.25_{10} 的IEEE 754 单精度浮点数表达

浮点数表达 例题

- 求 -58.25_{10} 的IEEE 754 单精度浮点数表达

1. 将十进制数转换为二进制数: $58.25_{10} = 111010.01_2$
2. 将数字写成 “二进制的科学记数法” : 1.1101001×2^5
3. 填写 32-bit 浮点数的各个字段:
   - 符号位: 1 (负数)
   - 8-bit 阶码: (127 + 5) = 132 = 10000100_2
   - 23-bit 尾数: 110 1001 0000 0000 0000 0000

- **写成十六进制:**
  - 0xC2690000

浮点数表达的特殊情况

	单精度 Single precision			
	符号位	阶码 e	尾数 m	值
正 0	0	0	0	0
负 0	1	0	0	-0
正无穷	0	255	0	∞
负无穷	1	255	0	-∞
NaN	0 or 1	255	!=0	NaN
正规格化非 0	0	0<e<255	f	$2^{e-127}(1.f)$
负规格化非 0	1	0<e<255	f	$-2^{e-127}(1.f)$
正非规格化非 0	0	0	f!=0	$2^{-126}(0.f)$
负非规格化非 0	1	0	f!=0	$-2^{-126}(0.f)$

注：

①非规格化数用于阶码下溢的情况，填补最小数和0之间的“空隙”。

②NaN：不是一个数

浮点数表达的特殊情况

	双精度 Double precision			
	符号位	阶码 e	尾数 m	值
正 0	0	0	0	0
负 0	1	0	0	-0
正无穷	0	2047	0	∞
负无穷	1	2047	0	-∞
NaN	0 or 1	2047	!=0	NaN
正规格化非 0	0	0<e<2047	f	$2^{e-1023}(1.f)$
负规格化非 0	1	0<e<2047	f	$-2^{e-1023}(1.f)$
正非规格化非 0	0	0	f!=0	$2^{-1022}(0.f)$
负非规格化非 0	1	0	f!=0	$-2^{-1022}(0.f)$

注：

①非规格化数用于阶码下溢的情况，填补最小数和0之间的“空隙”。

②NaN：不是一个数

浮点数精度 Precision

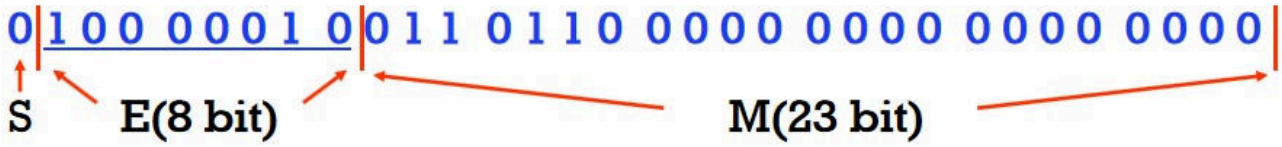
- **单精度** Single-Precision:
  - 共 32-bit，1-bit 符号, 8-bits 阶码, 23-bits 尾数
  - 阶码偏移量是127

- **双精度** Double-Precision:
  - 共 64-bit，1-bit 符号, 11-bits 阶码, 52-bits 尾数
  - 阶码偏移量是1023

浮点数表达转换为真值

例：IEEE754 格式的浮点数 X 机器值是 41360000_{16}，问其十进制真值?

1. 十六进制转换成二进制：



2. 计算指数的真值

$e = E-127 = 10000010-01111111 = 00000011 = 3_{10}$

3. 计算尾数的真值

$m = 1.M = 1.011\ 0110\ 0000\ 0000\ 0000\ 0000 = 1.011011$

综合得出该浮点数真值：

$X = (-1)^s \times 1.M \times 2^e$

$= +1.011011_2 \times 2^3$

$= +1011.011_2$

$= 11.375_{10}$

雨课堂 Rain Classroom

真值转换成IEEE 754 标准浮点数表示

例: 转换真值20.59375_{10}到 IEEE754 32-bit 单精度浮点数

1. 十进制转换成二进制：$20.59375_{10}=10100.10011$
2. 将数字写成 “二进制的科学记数法”

   $10100.10011 = 1.010010011\times2^4$　　$e=4$
3. 移码表示的阶码：$E=e+127 = 4+127 = 131_{10} = 1000\ 0011_2$
4. 填写各个字段：0 | 100 0001 | 1 010 0100 1100 0000 0000 0000 |

   S　E(8 bit)　M(23 bit)

➢ 最后将IEEE754 32-bit 浮点数机器值表达成十六进制：$41A4C000_{16}$

慕课堂课后练习

- 将下列十进制数表示成表示成IEEE754标准的32位浮点规格化数。

  (1) 27/64　　(2) -27/64

雨课堂 Rain Classroom

慕课堂课后练习

- 将十进制数-0.75表示成单精度的IEEE754标准编码。

慕课堂课后练习

- 将IEEE754单精度浮点数0C0A00000H用十进制数表示：

雨课堂 Rain Classroom

浮点数加法运算

- 浮点数加法不像二进制补码加减法那么简单，同符号的浮点数加法步骤如下：

1. 分开提取阶码和小数位
2. 小数位加上前导1（隐含的那个1），形成尾数
3. 比较阶码
4. 如果需要，对较小的尾数移位（进行对阶）
5. 尾数相加
6. 规格化尾数，并在需要时调整阶码
7. 结果做舍入处理
8. 把阶码和尾数组合成浮点数
9. 溢出判断

雨课堂 Rain Classroom

浮点数加法运算: 舍入处理

- 有四种舍入处理的规则(模式):
  - 向下舍down / 向上舍up / 向零舍Toward zero / 向最近端舍To nearest
- 例如: 对 1.100101_2 (1.578125_{10}) 进行舍入处理，只保留3-bits小数
  - 向下舍down: 1.100
  - 向上舍up: 1.101
  - 向零舍Toward zero: 1.100
  - 向最近端舍To nearest: 1.101 (类似四舍五入)
- 溢出判断
  - 上溢出Overflow: 阶码太大，表达不下，是错误，要报出。
  - 下溢出Underflow: 阶码太小，表达不了，做 0 处理，损失了精度。

浮点数加法运算 示例

- 计算如下两个浮点数的和:
  - 0x3FC00000, 0x40500000

雨课堂 Rain Classroom

浮点数加法运算 示例

- **手工计算**如下两个浮点数的和:

  0x3FC00000 = 0011 1111 1100 0000 0000 0000 0000 0000

  0x40500000 = 0100 0000 0101 0000 0000 0000 0000 0000

  - E1 = 01111111 – 01111111 = 0, E2 = 10000000 – 01111111 = 1
  - M1 = 1.1, M2 = 1.101
  - N1 = $1.1*2^0 = 1.5_{10}$, N2 = $1.101*2^1$ = 11.01 B = 3.25_{10}
  - N1+N2 = $1.5_{10} + 3.25_{10} = 4.75_{10}$
  - Transform 4.75_{10} to IEEE754 32-bit floating-point number: 0x40980000

雨课堂 Rain Classroom

浮点数加法运算 示例

1bit	8 bits	23 bits
0	0111 1111	100 0000 0000 0000 0000 0000
符号	阶码	小数（原码表示）

0	1000 0000	101 0000 0000 0000 0000 0000
符号	阶码	小数（原码表示）

1. 提取阶码和小数位

阶码	小数
0111 1111	100 0000 0000 0000 0000 0000
1000 0000	101 0000 0000 0000 0000 0000

雨课堂 Rain Classroom

浮点数加法运算 示例

1. 提取阶码和小数位

阶码	小数
0111 1111	100 0000 0000 0000 0000 0000
1000 0000	101 0000 0000 0000 0000 0000

2. 小数位加上前导1（隐含的那个1），形成尾数

阶码	小数
0111 1111	1.100 0000 0000 0000 0000 0000
1000 0000	1.101 0000 0000 0000 0000 0000

浮点数加法运算 示例

1. 提取阶码和小数位
2. 小数位加上前导1（隐含的那个1），形成尾数

阶码	小数
0111 1111	1.100 0000 0000 0000 0000 0000
1000 0000	1.101 0000 0000 0000 0000 0000

3. 比较阶码（对阶）

	0111 1111	1.100 0000 0000 0000 0000 0000
−	1000 0000	1.101 0000 0000 0000 0000 0000
	−1（移位数）	

雨课堂 Rain Classroom

浮点数加法运算 示例

1. 提取阶码和小数位
2. 小数位加上前导1（隐含的那个1），形成尾数
3. 比较阶码（对阶）

	0111 1111	1.100 0000 0000 0000 0000 0000
−	1000 0000	1.101 0000 0000 0000 0000 0000
		−1（移位数）

4. 小阶向大阶看齐，右移小阶的尾数，移动位数见上一步比较结果

1000 0000	0.110 0000 0000 0000 0000 0000 0
1000 0000	1.101 0000 0000 0000 0000 0000

雨课堂 Rain Classroom

浮点数加法运算 示例

3. 比较阶码（对阶）

4. 小阶向大阶看齐，右移小阶的尾数，移动位数见上一步比较结果

1000 0000	0.110 0000 0000 0000 0000 0000 0
1000 0000	1.101 0000 0000 0000 0000 0000

5. 尾数相加

1000 0000	0.110 0000 0000 0000 0000 0000 0
1000 0000	+ 1.101 0000 0000 0000 0000 0000
	10.011 0000 0000 0000 0000 0000 0

雨课堂 Rain Classroom

浮点数加法运算 示例

5. 尾数相加

1000 0000	0.110 0000 0000 0000 0000 0000 0
1000 0000	+ 1.101 0000 0000 0000 0000 0000
	10.011 0000 0000 0000 0000 0000 0

6. 规格化尾数，并调整阶码

1000 0000	10.011 0000 0000 0000 0000 0000 0 >>1
+1	
1000 0001	1.001 1000 0000 0000 0000 0000

7. 无需舍入处理

雨课堂 Rain Classroom

浮点数加法运算 示例

6. 规格化尾数，并调整阶码

1000 0000	10.011 0000 0000 0000 0000 0000 0	>>1
+1		
1000 0001	1.001 1000 0000 0000 0000 0000	

7. 无需舍入处理

8. 把阶码和小数组合成浮点数

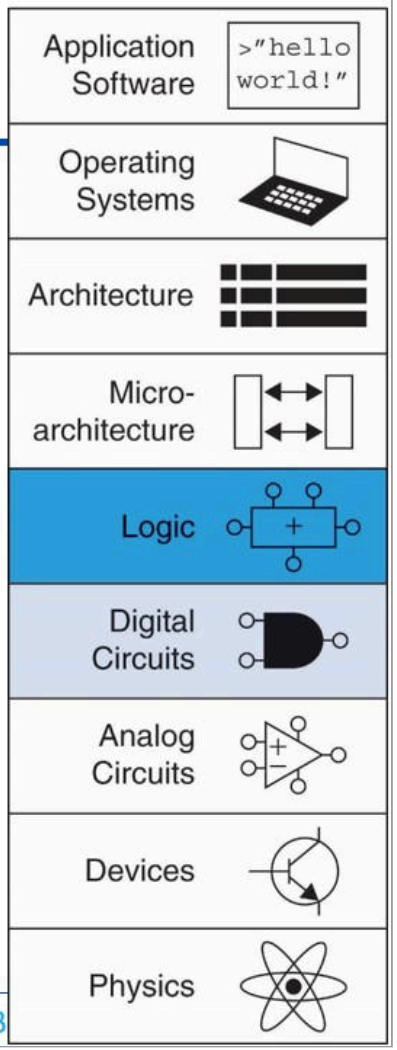
0	1000 0001	1.001 1000 0000 0000 0000 0000

9. 无溢出，转变成十六进制表示：0x4098 0000

雨课堂 Rain Classroom

Chapter 5 :: Topics

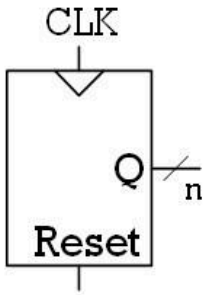
- 引言
- 算术运算电路
- 数制系统（重点介绍浮点数）
- **时序电路模块（计数器、移位寄存器）**
- 存储器阵列



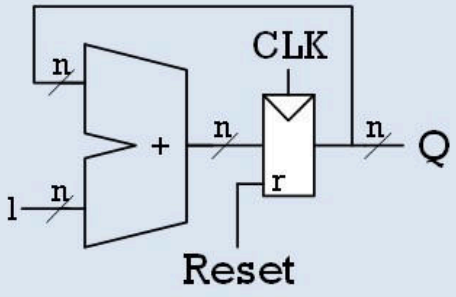
计数器 Counters

- 在每个时钟上升沿的时候递增1
  - 通常计数器的输出会循环往复的改变。例如, 000, 001, …, 110, 111, 000, …
- 计数器在计算机的组成中扮演着重要的角色，应用场合如:
  - 数字时钟的显示，乘除法器控制循环次数
  - **程序计数器(Program Counter, PC)**: 用于跟踪当前正在执行的指令

功能符号

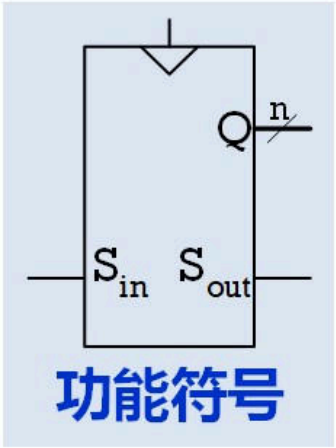


实现逻辑

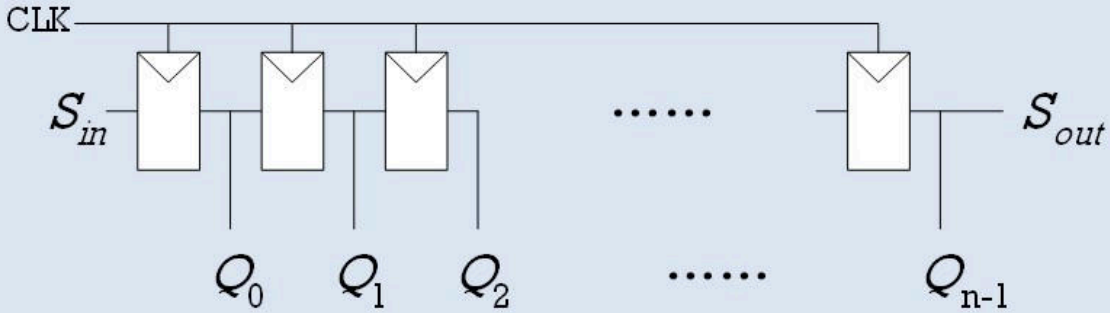


移位寄存器 Shift Registers

- 移位寄存器会在每个时钟沿**移进**一位新的数据，也**移出**一位数据。
  - 同时**并行输出** $Q_{n-1:0}$，即 $S_{in} \rightarrow Q_{0:n-1}$
  - 所以是一个**串转并**(serial-to-parallel)的移位寄存器



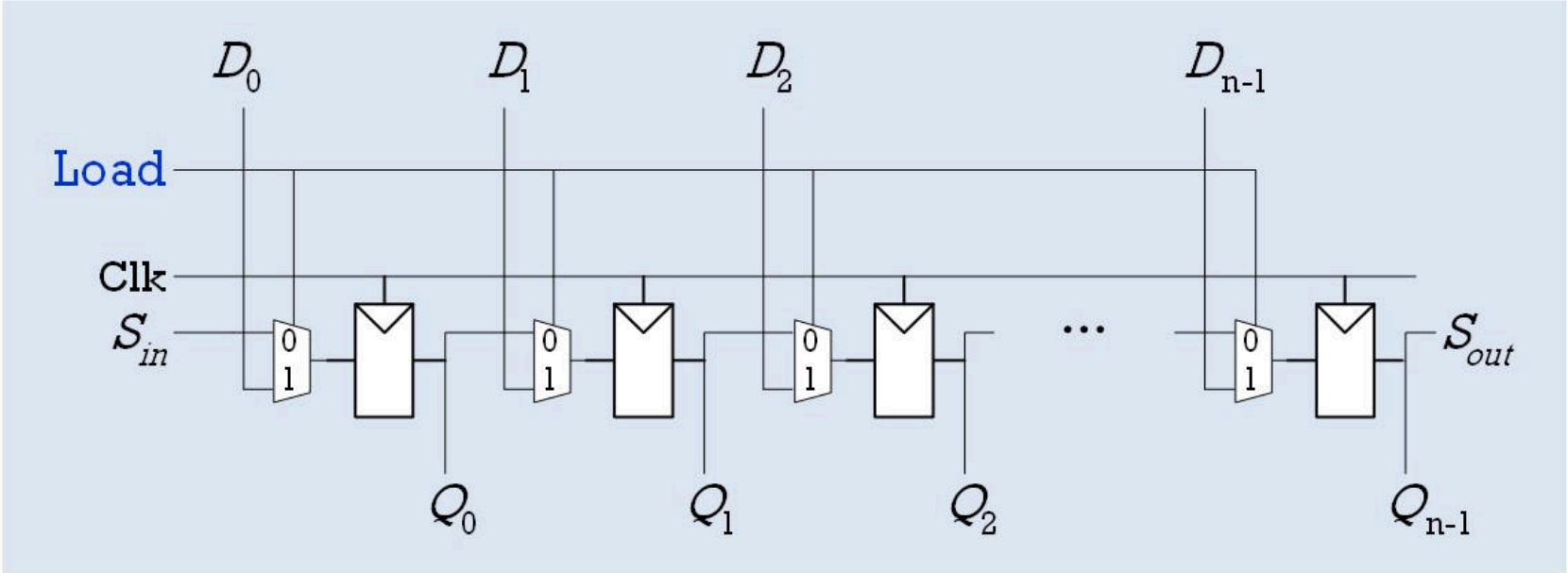
功能符号



实现逻辑

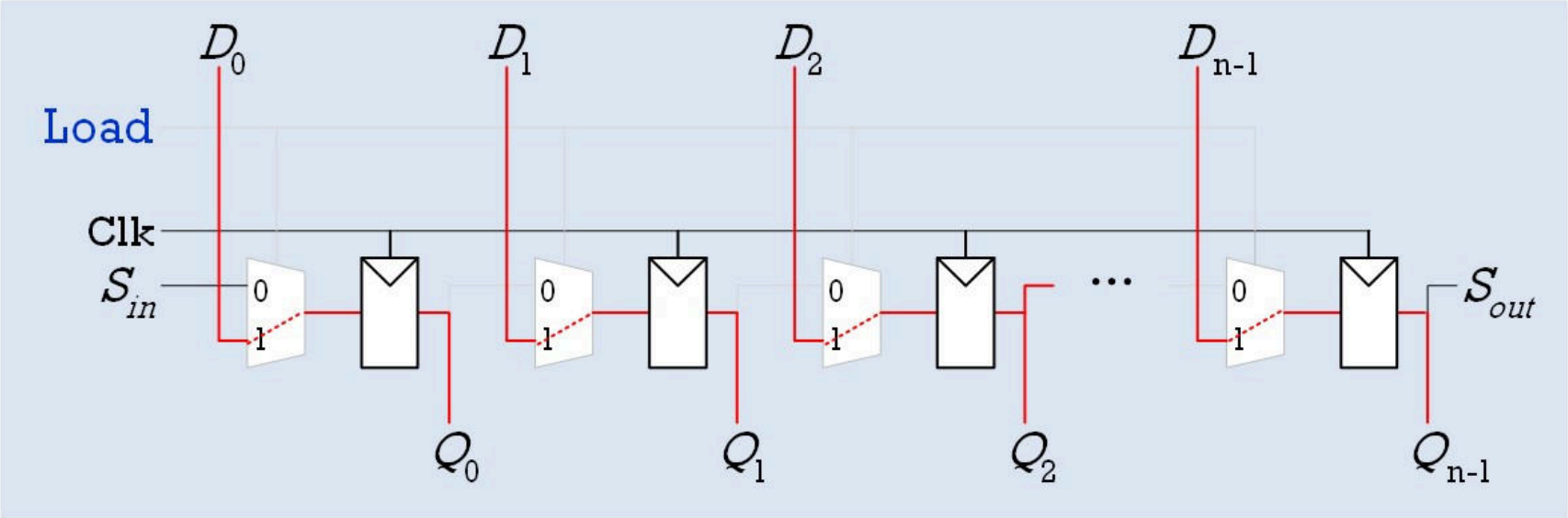
移位寄存器 Shift Registers

- 带并行载入功能的移位寄存器



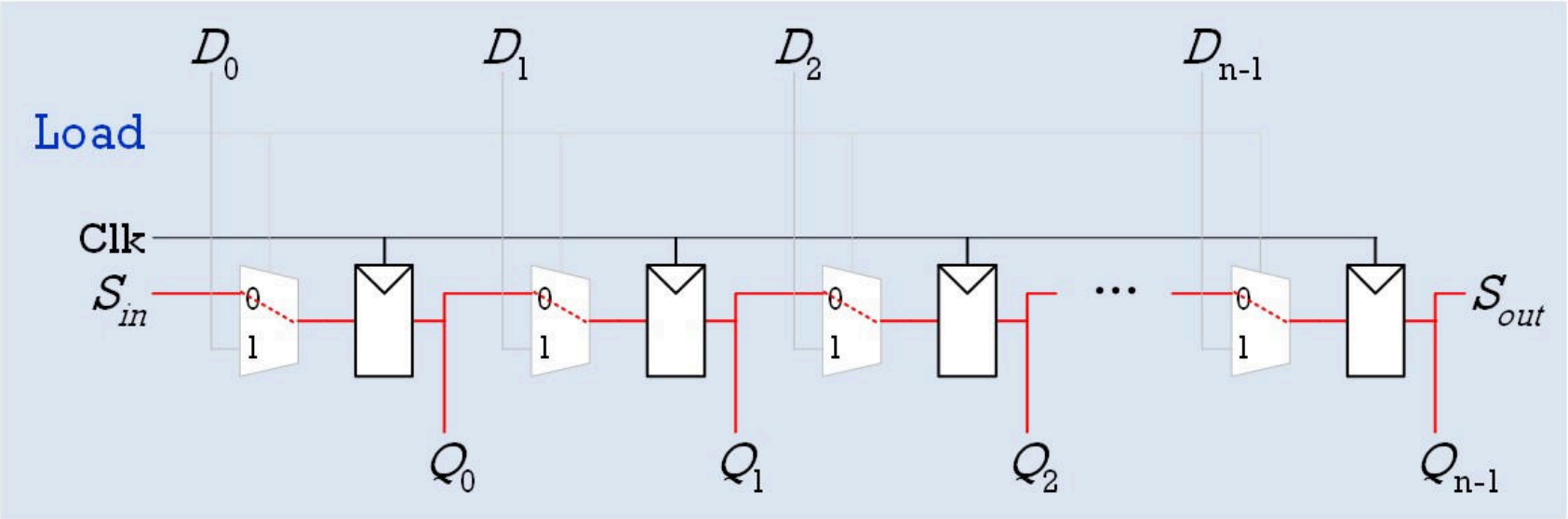
移位寄存器 Shift Registers

- 带并行载入功能的移位寄存器
  - 当 Load = 1时，这个电路就是一个正常的 N位寄存器。



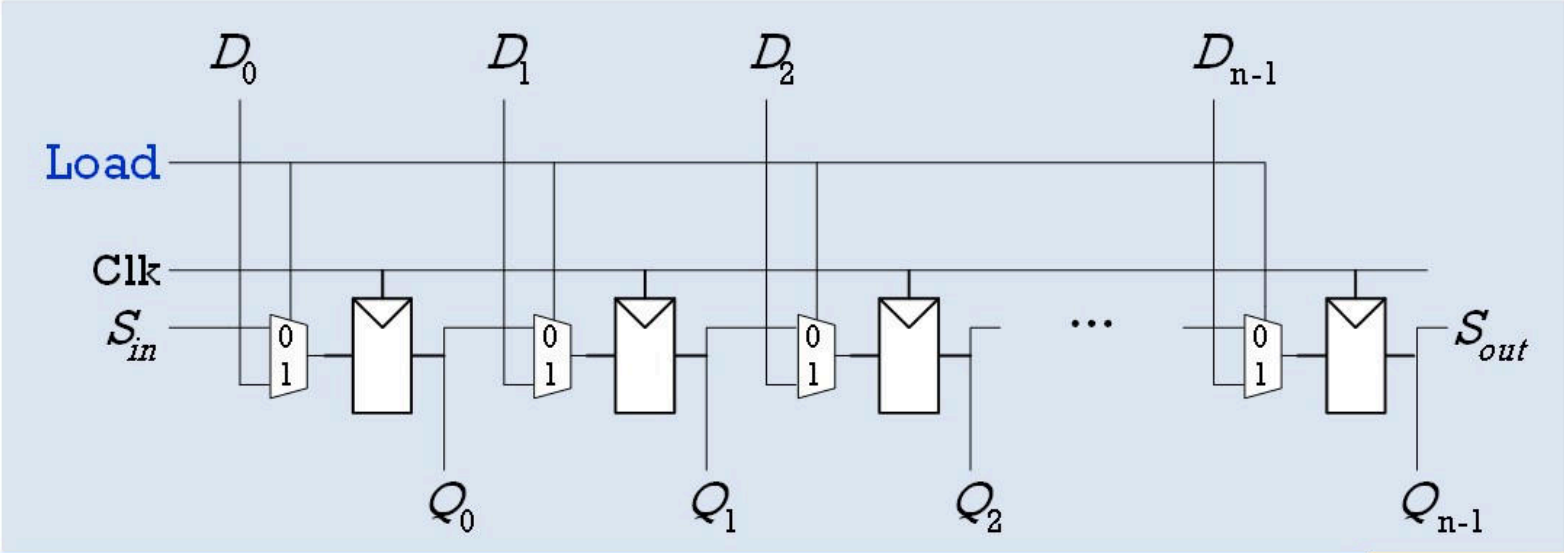
移位寄存器 Shift Registers

- 带并行载入功能的移位寄存器
  - 当 **Load = 0** 时，这个电路是一个移位寄存器。



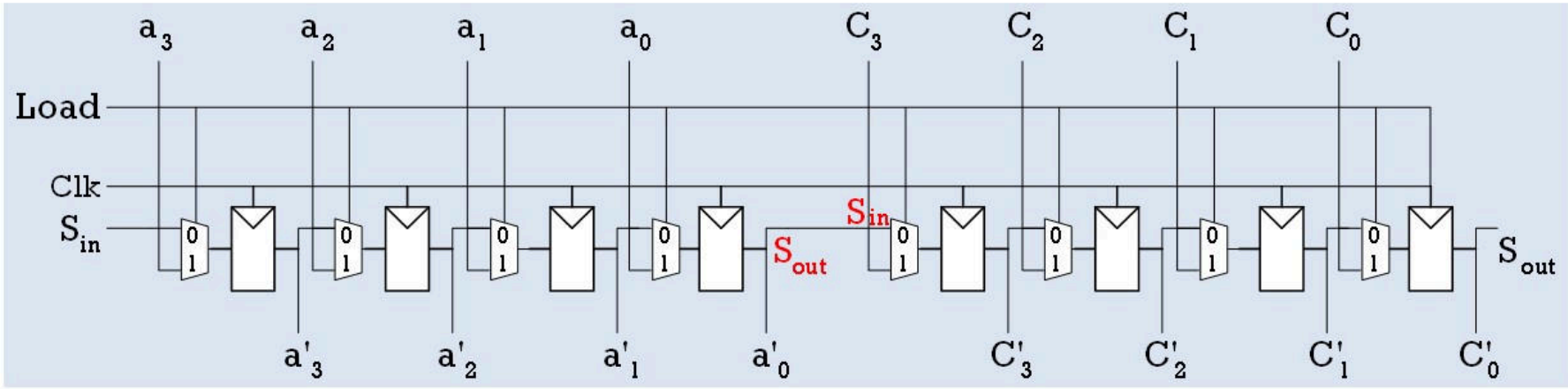
移位寄存器 Shift Registers

- 所以，这个电路
  - 既可以是**串转并**的移位寄存器 (S_{in} to $Q_{0:n-1}$)
  - 也可以是**并转串**的移位寄存器 ($D_{0:n-1}$ to S_{out})



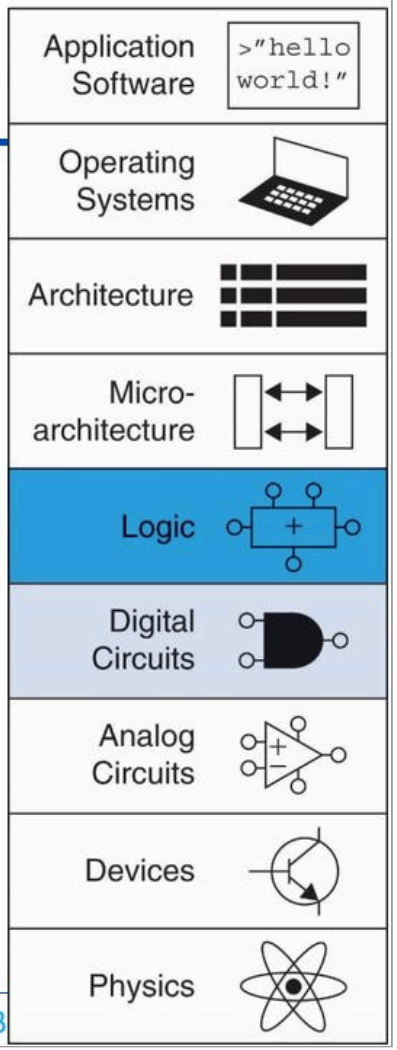
移位寄存器 Shift Registers

- 在乘法器中，实现寄存器 A 和寄存器 C 的联合右移并不难：$\overrightarrow{A,C}$



Chapter 5 :: Topics

- 引言
- 算术运算电路
- 数制系统（重点介绍浮点数）
- 时序电路模块（计数器、移位寄存器）
- **存储器阵列**



存储器类型

课后自学

- 随机访问存储器 (Random access memory, RAM): 易失性volatile
- 只读存储器 (Read only memory, ROM): 非易失性nonvolatile
  - RAM和ROM的命名有历史原因，现在来看有问题
    - 所谓随机访问存储器（RAM）指的是访问任何数据字的延迟都是一样的，相对于顺序访问而言，如磁带。
    - ROM之所以称之为只读存储器，是因为历史上它只能读，不能写。但其实ROM也是随机访问的，而且现在的ROM也可写。
  - **所以现在来区分RAM和ROM最重要的是易失性和非易失性。**
    - 关闭电源的时候能否存储数据

雨课堂 Rain Classroom

存储器的发展

课后自学

- 第一台电子计算机用的是电子管触发器；
- 此后经历过：
  - 汞延迟线
  - 磁带
  - 磁鼓
  - 磁芯（1951年始）
  - 半导体
  - 磁盘—光盘—纳米存储

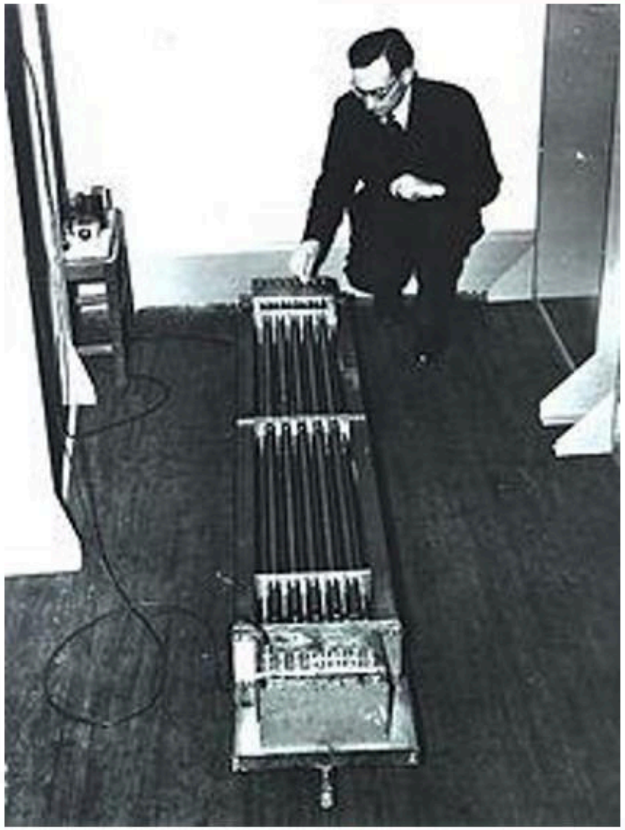
雨课堂 Rain Classroom

水银延迟线

课后自学

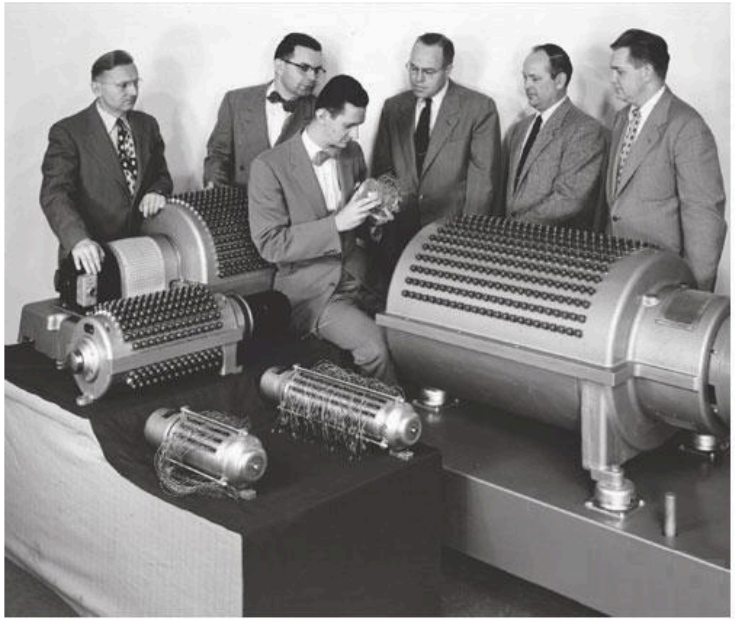
- 英国的早期计算机EDSAC 使用水银延迟线作存储器，分布在32个槽中，每个槽5英尺长，里面包含32个内存位置，共1024个位置。

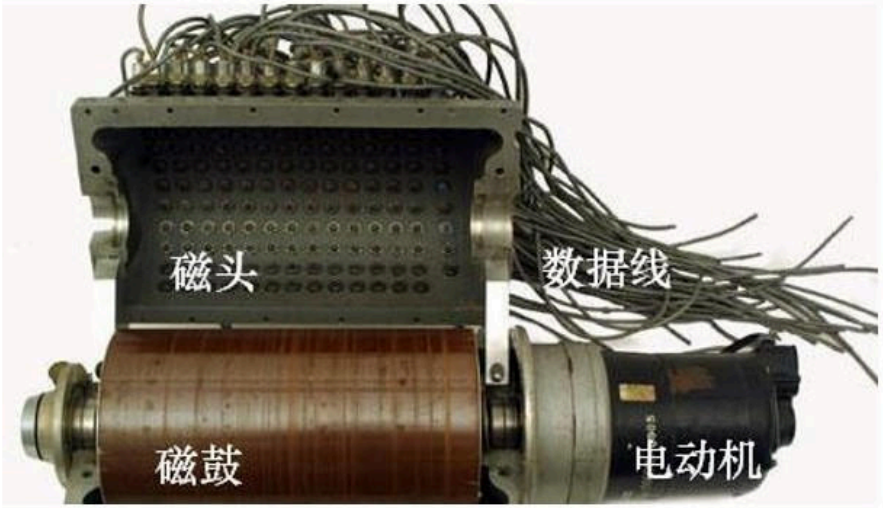
- 建造时只实现了一半，512个字，第二组于1952年添加。1952年增加一个磁带存储，但实际使用中不能良好工作。



磁鼓

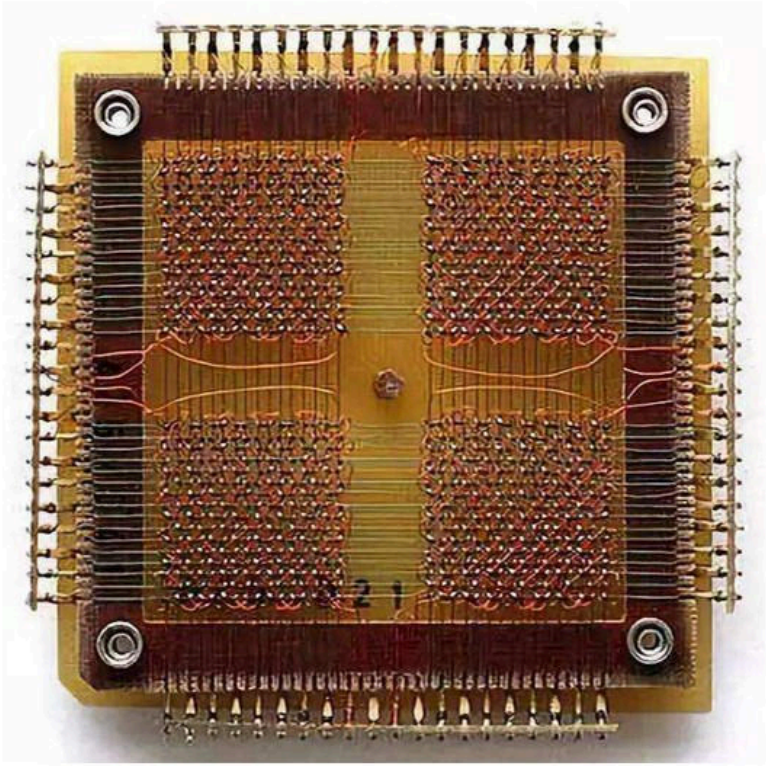
课后自学

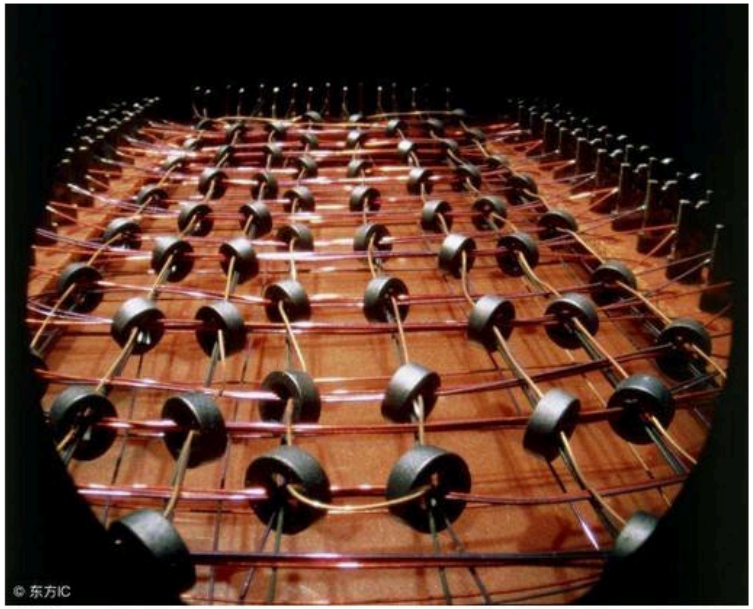




磁芯

课后自学





存储器的发展

课后自学

- 容量 Capacity

Bit(b) Byte(B) KiloByte(KB) MegaByte(MB)

GigaByte(GB) TeraByte (TB)

PetaByte(PB) ExaByte (EB) ZetaByte(ZB)

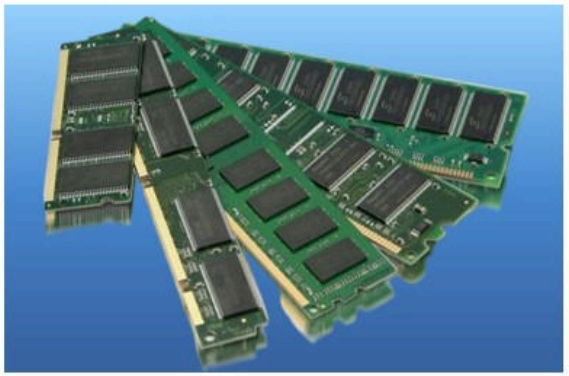
YottaByte(YB) NonaByte(NB) DoggaByte (DB)

单位名称	常规十进制表示	存储器容量表示
K（Kilo）	$1K \approx 10^3 = 1000$	$1K = 2^{10} = 1024$
M（Mega）	$1M \approx 10^6 = 10^3 K$	$1M = 2^{20} = 2^{10}K = 1\,048\,576$
G（Giga）	$1G \approx 10^9 = 10^6 M$	$1G = 2^{30} = 2^{10}M = 1\,073\,741\,824$
T（Tera）	$1T \approx 10^{12} = 10^9 G$	$1T = 2^{40} = 2^{10}G = 1\,099\,511\,627\,776$

随机访问存储器类型

课后自学

- **动态**随机访问存储器DRAM (Dynamic Random Access Memory)
- **静态**随机访问存储器SRAM (Static Random Access Memory)



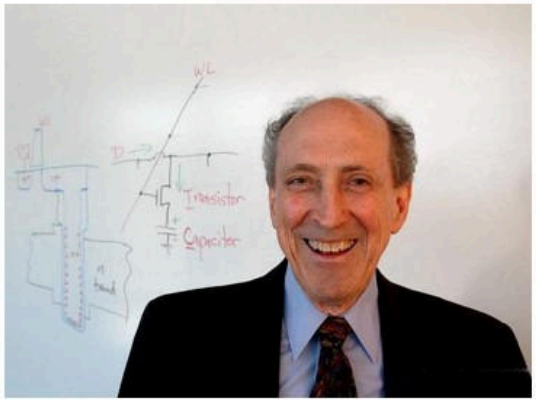
- 二者的数据存储方式不同：
  - DRAM 使用电容capacitor来存储数据
  - SRAM 使用交叉耦合反相器（cross-coupled inverters，类似触发器）

雨课堂 Rain Classroom

DRAM发明者 Robert Dennard

课后自学

- 1966年在IBM发明了DRAM.
- 到1970年代中期，DRAM出现在几乎所有的计算机中.
- DRAM发明者Robert Dennard，被评为2019年SIA最高荣誉Robert N.诺伊斯奖。
- 诺伊斯奖是由SIA（半导体行业协会）颁发的半导体行业最高荣誉奖，为了纪念集成电路发明者，同时也是仙童半导体和英特尔的联合创始人，半导体行业先驱Robert N. Noyce而设立的奖项，以表彰为半导体行业在技术或公共政策方面做出杰出贡献的领导者。



闪存发明者 Fujio Masuoka

课后自学

- 舛冈富士雄，出生于1943年，日本。1971–1994 年任职东芝，负责存储器与高速电路研发。
- 20 世纪 70 年代后期，他利用夜间、周末时间自主开展项目，在未获官方立项的情况下发明了闪存（Flash Memory）。
- 该存储器需要擦除，擦除存储器的过程让他联想到相机闪光灯（flash），闪存由此得名。
- 东芝迟迟未能将该技术商业化；英特尔于 1988 年率先推出商用闪存产品。
- 如今闪存市场规模已发展至每年 250 亿美元。

雨课堂 Rain Classroom

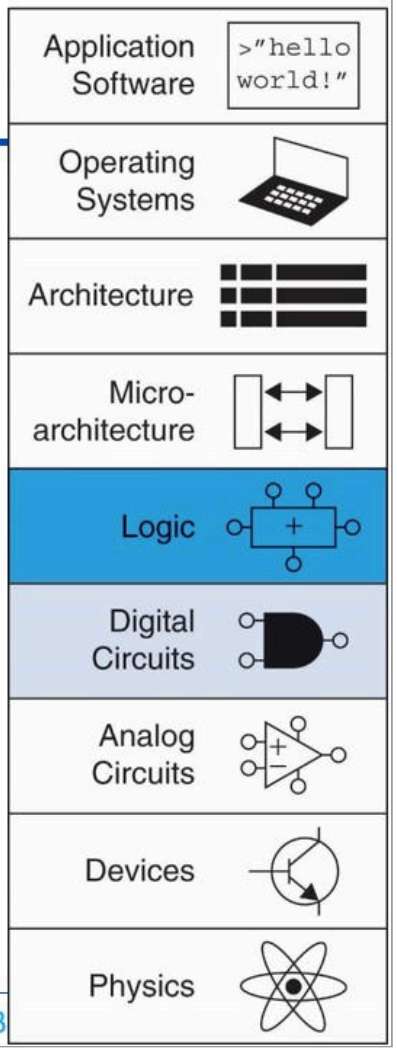
第十五堂课结束了

有问题同学们可以提问讨论

雨课堂 Rain Classroom

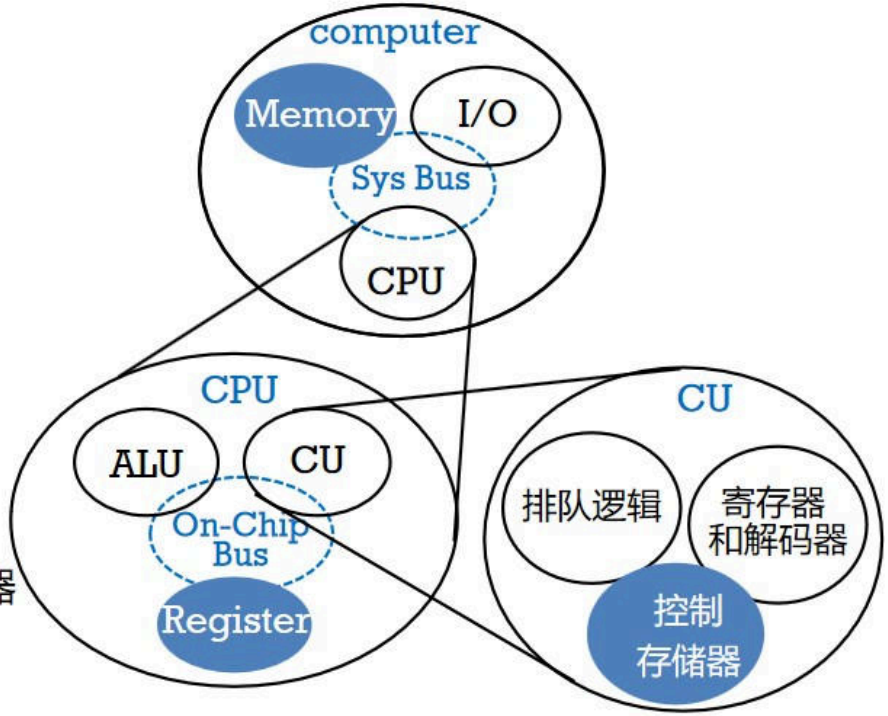
Chapter 5 :: Topics

- 引言
- 算术运算电路
- 数制系统（重点介绍浮点数）
- 时序电路模块（计数器、移位寄存器）
- **存储器阵列**



计算机主机中的存储组件

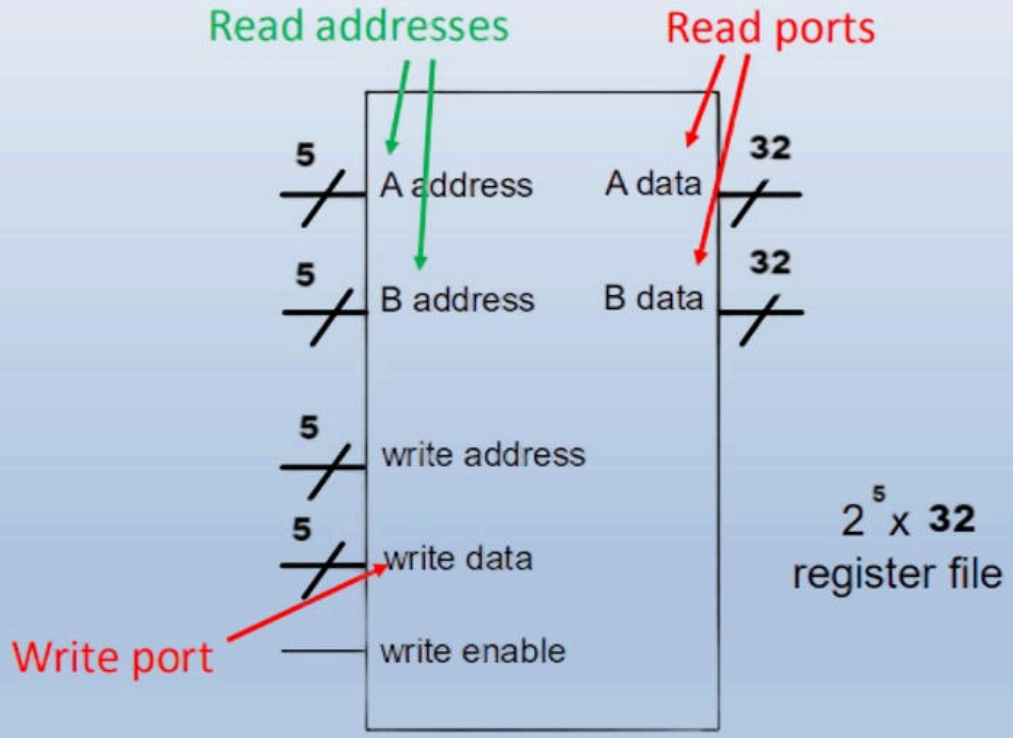
- **寄存器** (Register)
- **内存** (Memory)
  - 随机访问存储器 **RAM**: 易失性
    - 动态随机访问存储器 DRAM
    - 静态随机访问存储器 SRAM
    - 二者的数据存储方式不同:
      - DRAM 使用电容来存储数据
      - SRAM 使用交叉耦合反相器，类似触发器
  - 只读存储器 **ROM**: 非易失性



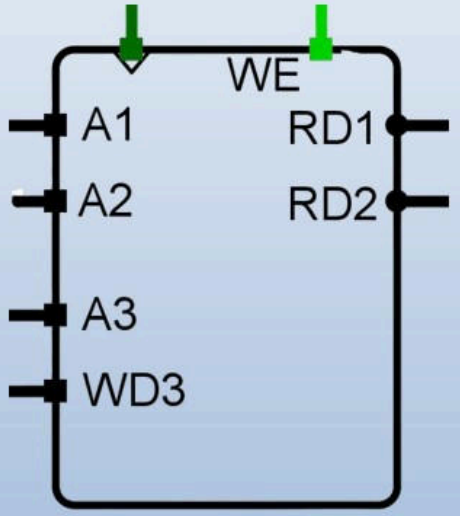
MIPS 的寄存器文件/寄存器堆

Name	Register Number	用途	Usage
$0	0	常量 0	constant value 0
$at	1	汇编器临时变量	assembler temporary
$v0-$v1	2-3	函数返回值	Function return values
$a0-$a3	4-7	函数参数	Function arguments
$t0-$t7	8-15	临时变量	temporaries
$s0-$s7	16-23	保存变量	saved variables
$t8-$t9	24-25	临时变量	more temporaries
$k0-$k1	26-27	操作系统临时变量	OS temporaries
$gp	28	全局指针	global pointer
$sp	29	栈指针	stack pointer
$fp	30	帧指针	frame pointer
$ra	31	函数返回地址	Function return address

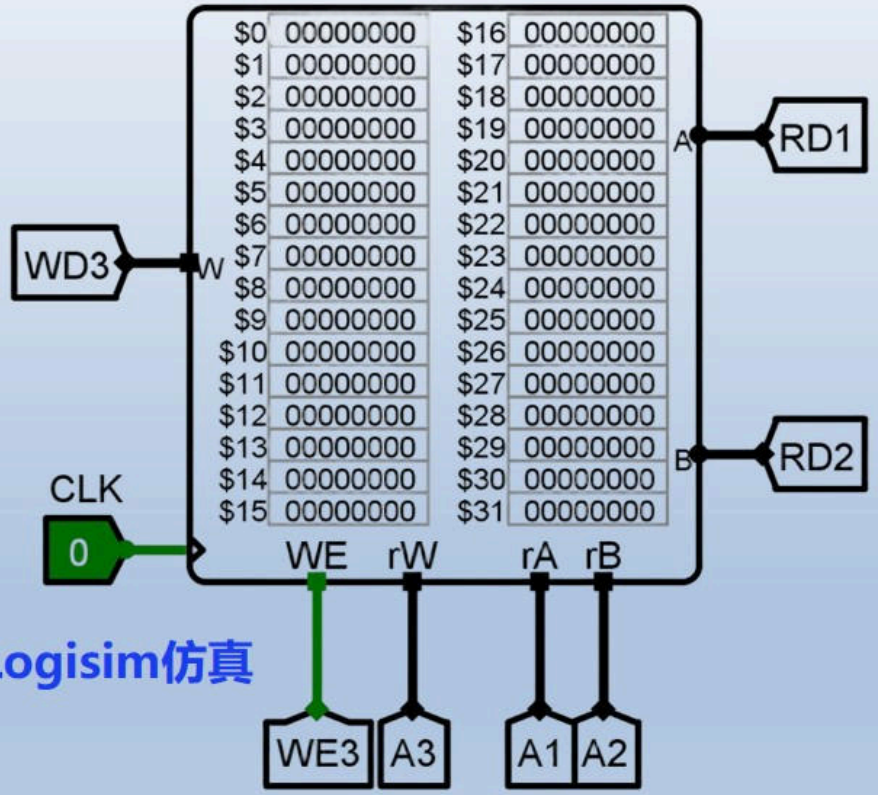
存储器阵列 —— 寄存器文件 Register File



存储器阵列 —— 寄存器文件 Register File

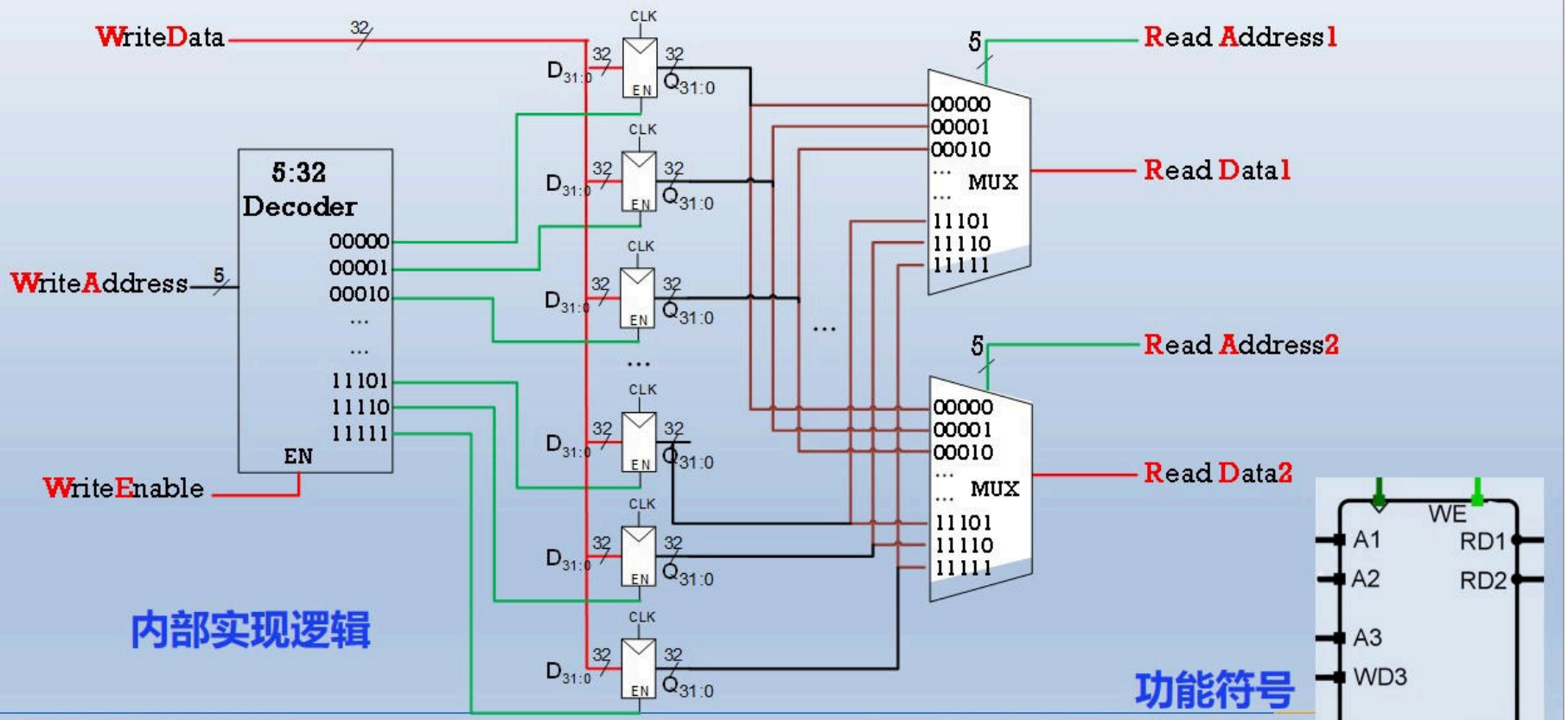


功能符号



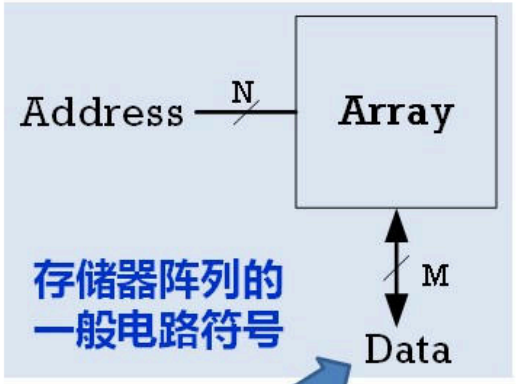
Logisim仿真

存储器阵列 —— 寄存器文件 Register File



存储器阵列 —— 内存 Memory

- 高效存储**大量**数据
  - 对大量数据进行存储的存储器的**基本要求**是：可以**随机**的读或写阵列中的**任一行**(M-bit 数据)。
- 3 种常见类型(每种存储器以不同的方式存储数据):
  - DRAM (Dynamic random access memory)
  - SRAM (Static random access memory)
  - ROM (Read only memory)



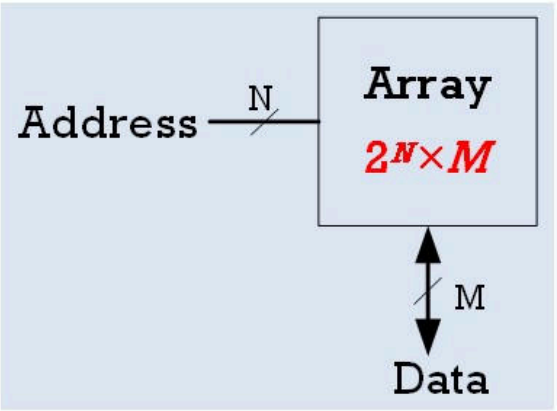
在每个唯一的 N-bit 地址读取/写入的 M-bit 数据值

存储器阵列 —— 内存 Memory

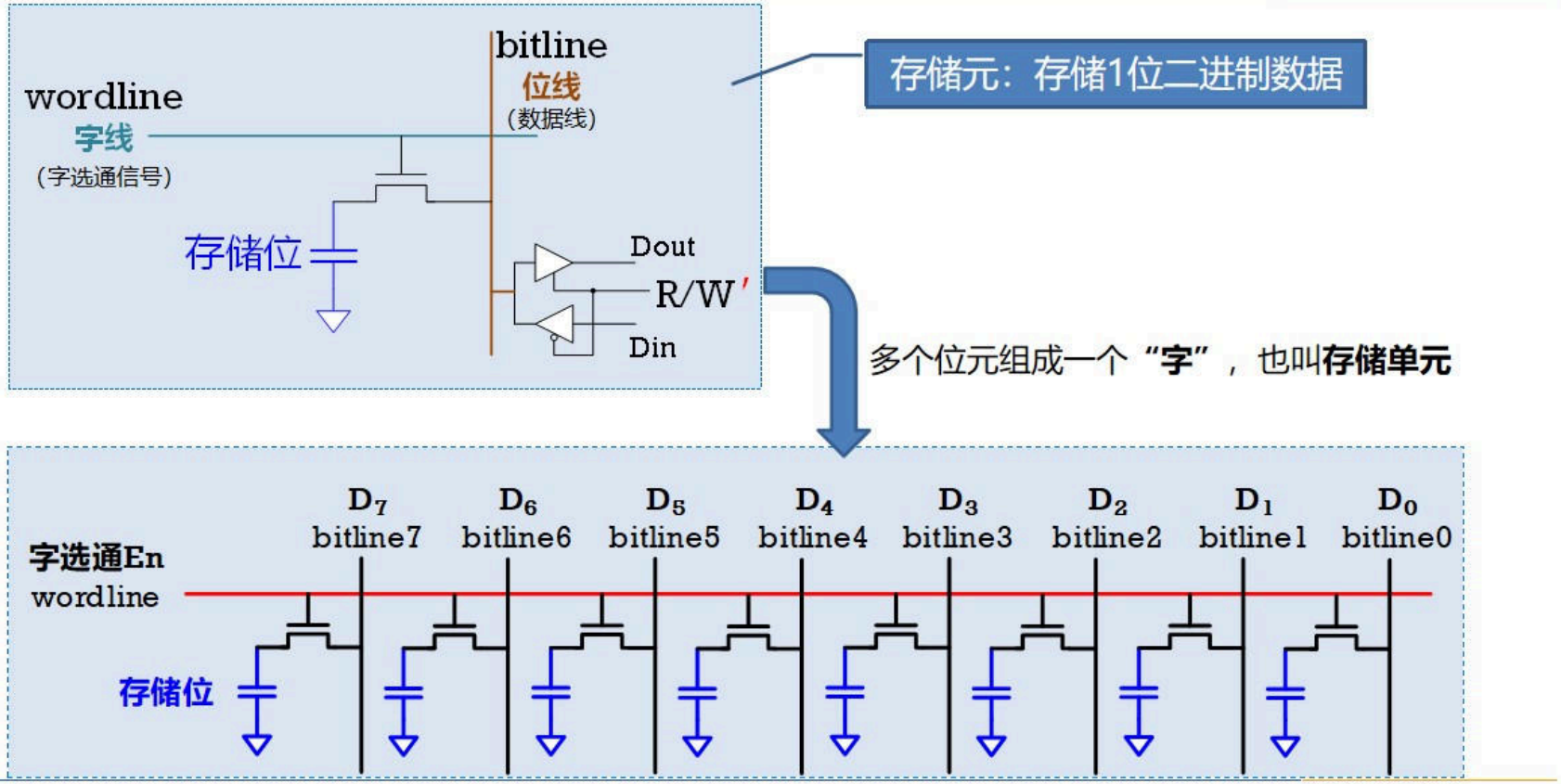
存储器阵列 —— 内存 Memory

- **位元**(bit cell)构成的二维阵列，每个位元存储 1 位二进制数据。

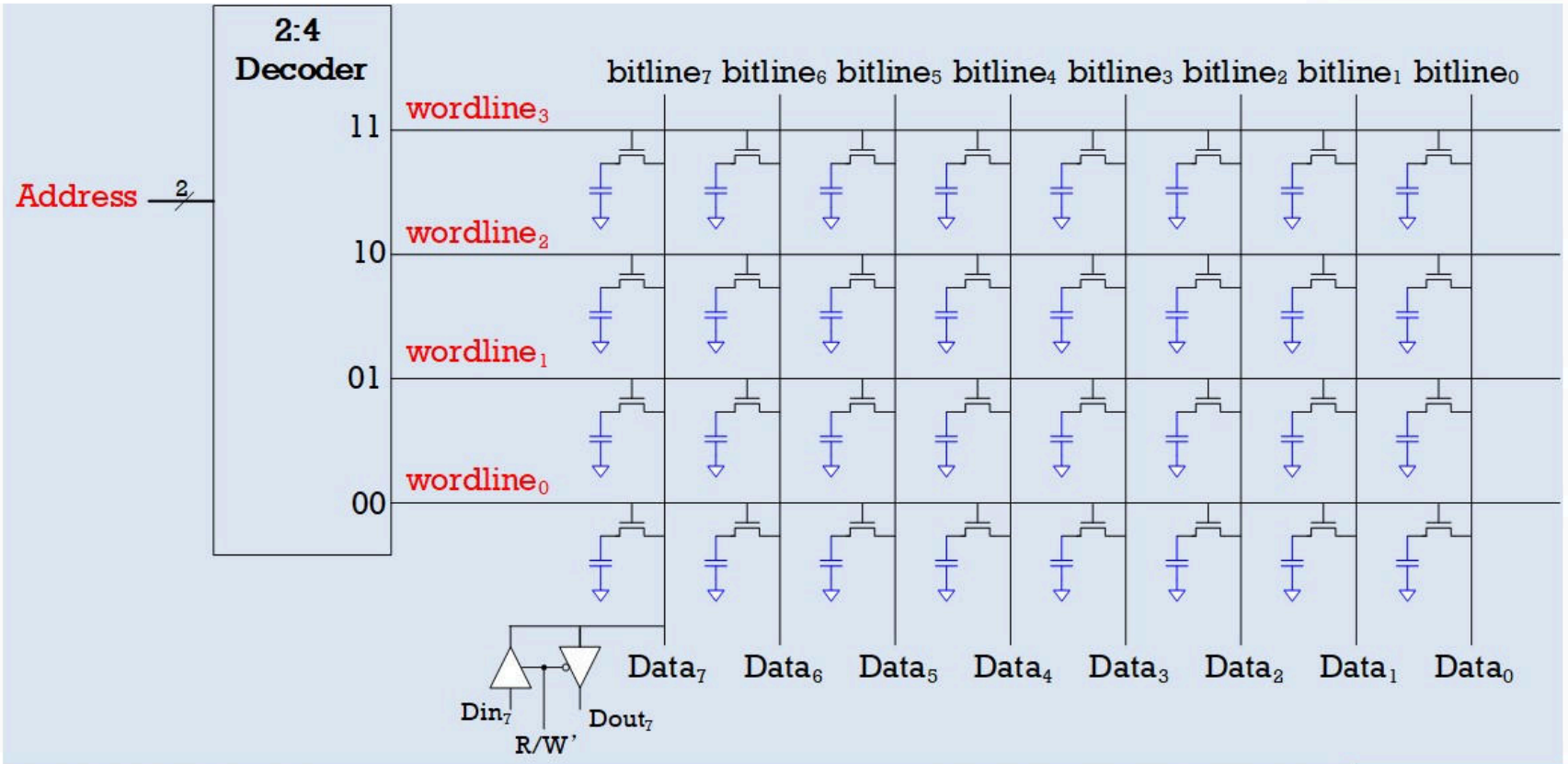
- N 位地址、M 位数据:
  - 具有 2^N 行、M 列
  - 深度(Depth)：行数（存储字的数量）
  - 位宽(Width)：列数（单个字的bit长度）
  - 阵列容量: 深度 × 位宽 = 2^N 字 × M 位



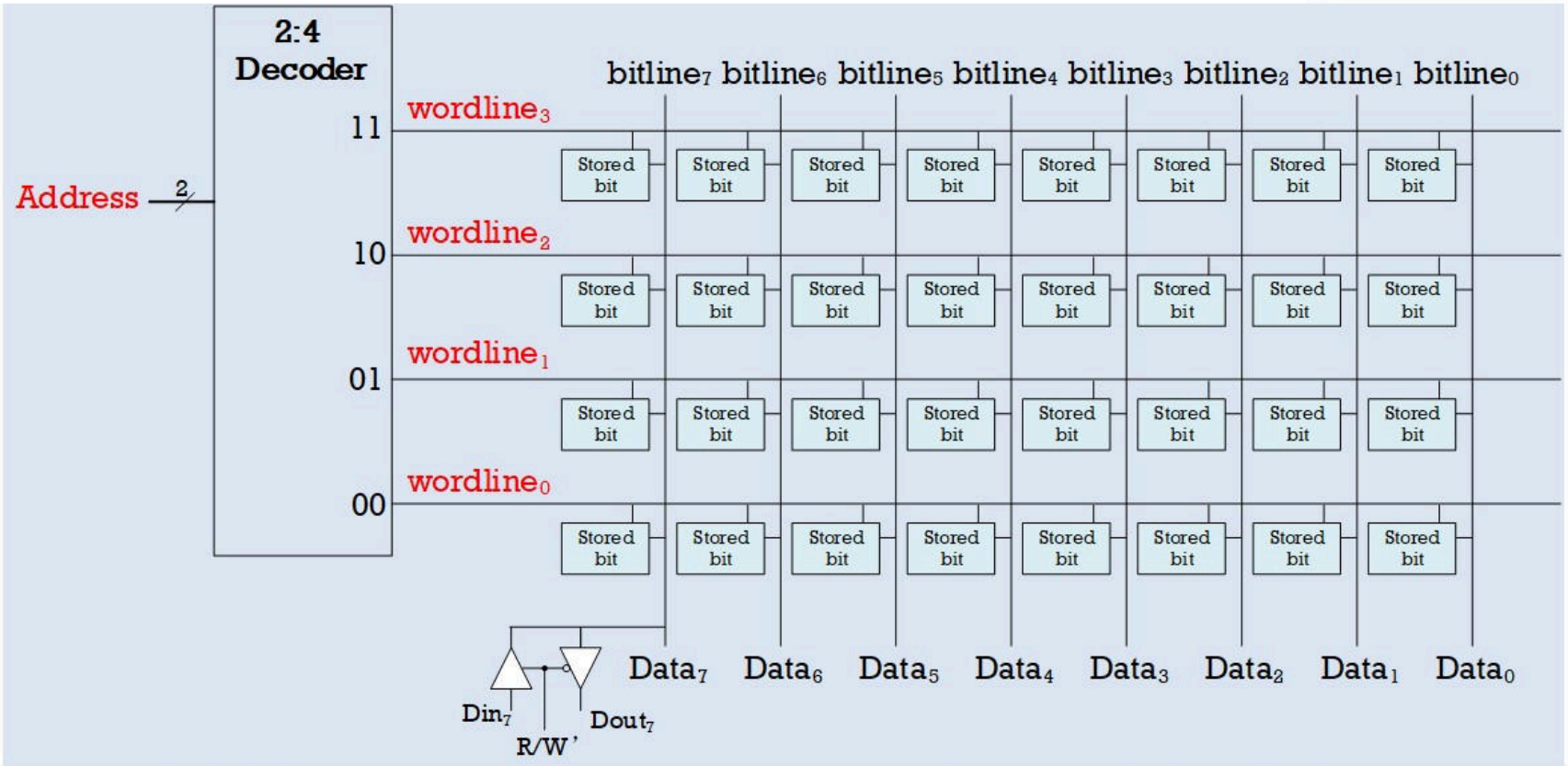
存储器阵列 —— 位元结构（以DRAM位元结构为例）



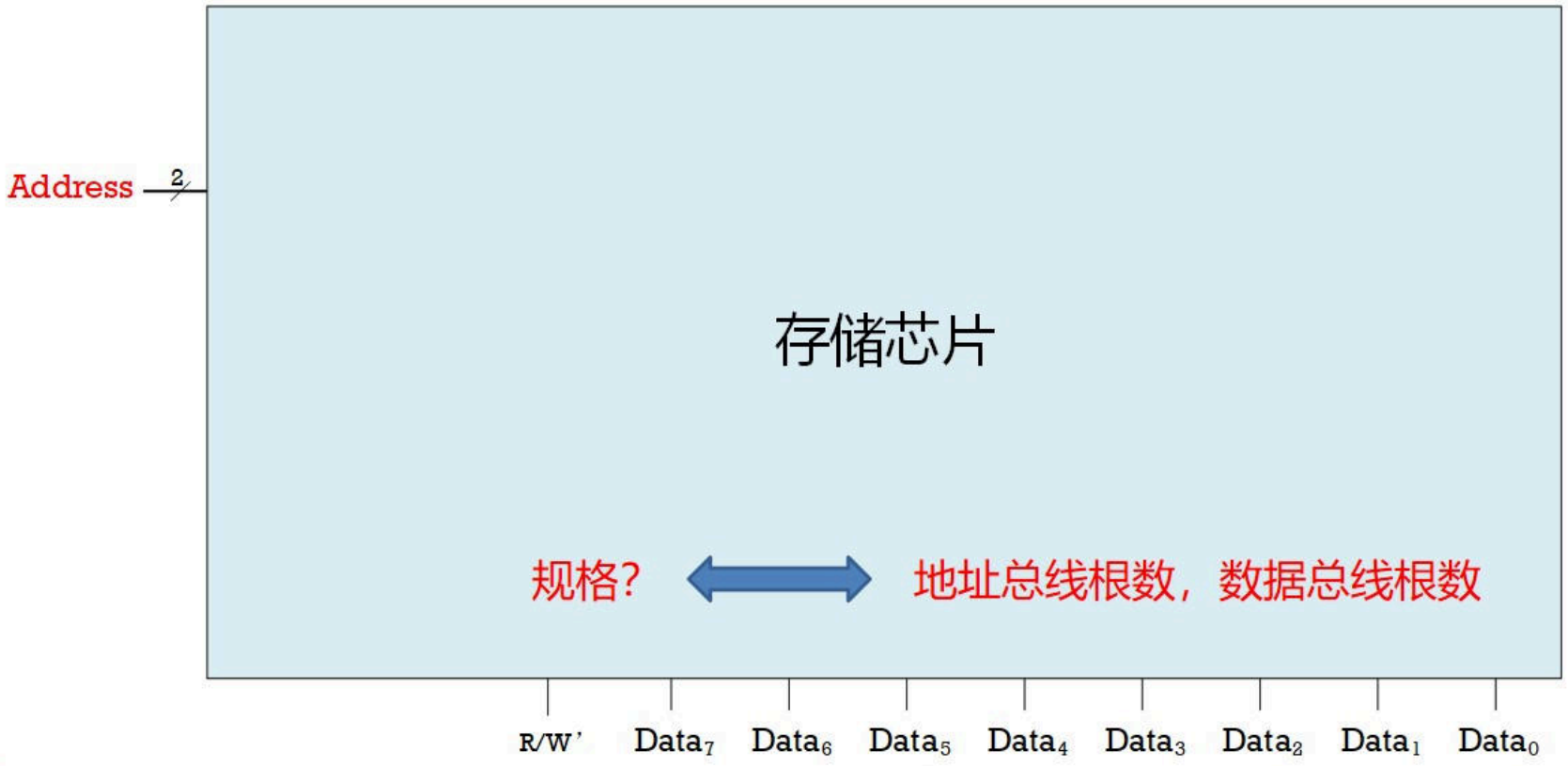
存储器阵列


2:4
Decoder
Address
2
11
10
01
00
wordline3
wordline2
wordline1
wordline0
bitline7 bitline6 bitline5 bitline4 bitline3 bitline2 bitline1 bitline0
Data7 Data6 Data5 Data4 Data3 Data2 Data1 Data0
Din7
R/W'
Dout7

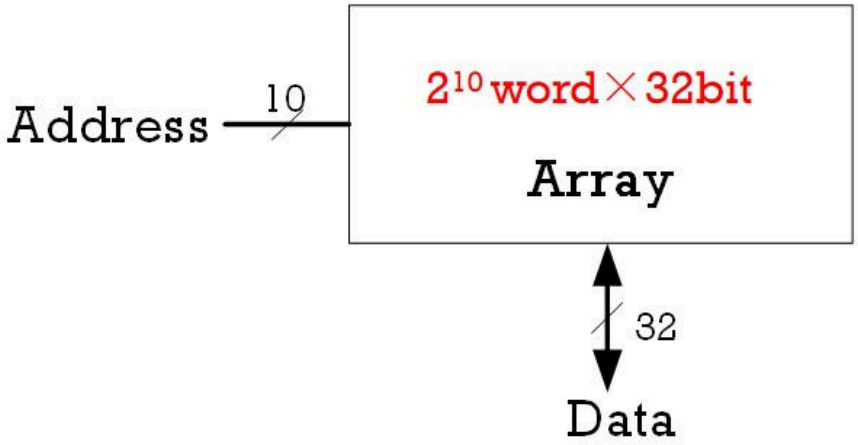
存储器阵列


2:4
Decoder
Address
2
11
10
01
00
wordline3
wordline2
wordline1
wordline0
bitline7 bitline6 bitline5 bitline4 bitline3 bitline2 bitline1 bitline0
Stored bit
Data7 Data6 Data5 Data4 Data3 Data2 Data1 Data0
Din7
Dout7
R/W'

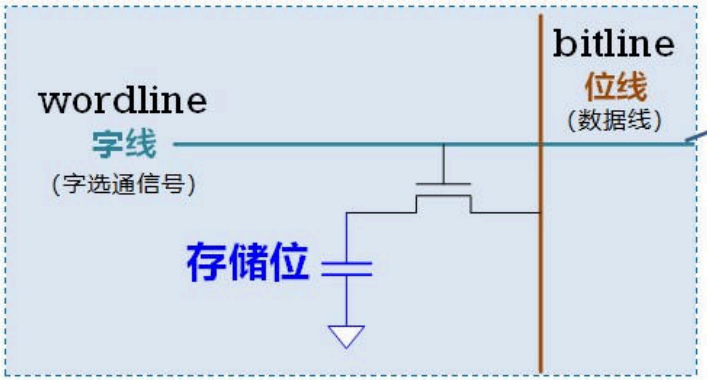
存储器阵列


Address
2
存储芯片
规格?
地址总线根数，数据总线根数
R/W'
Data7
Data6
Data5
Data4
Data3
Data2
Data1
Data0

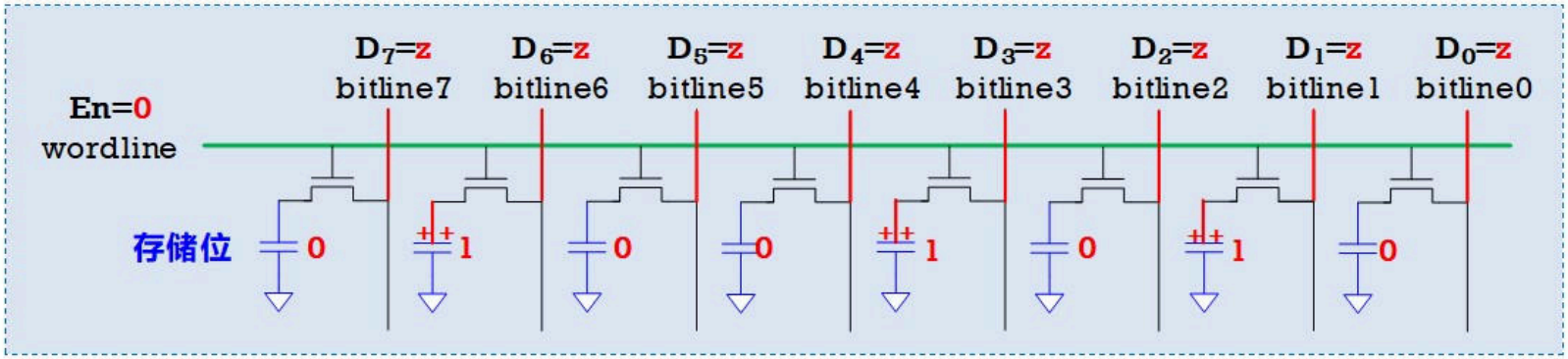
存储器阵列



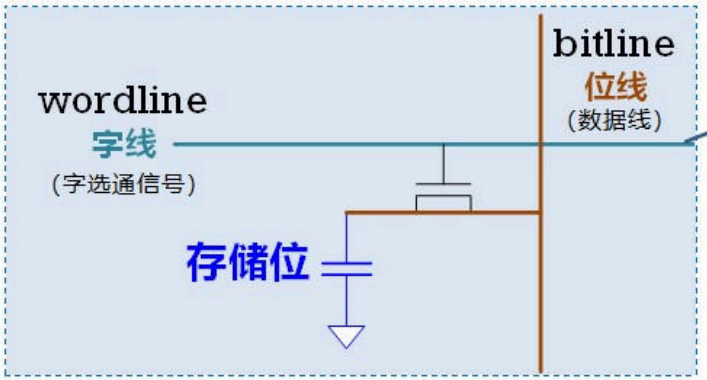
动态随机访问存储器DRAM(Dynamic Random Access Memory)



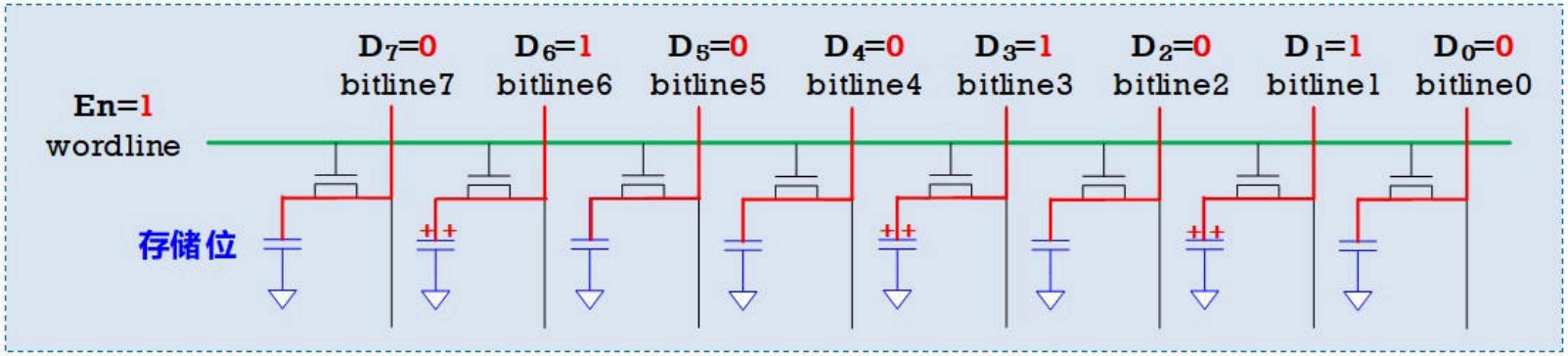
当字线为低电平时，处于保持存储状态



动态随机访问存储器DRAM(Dynamic Random Access Memory)

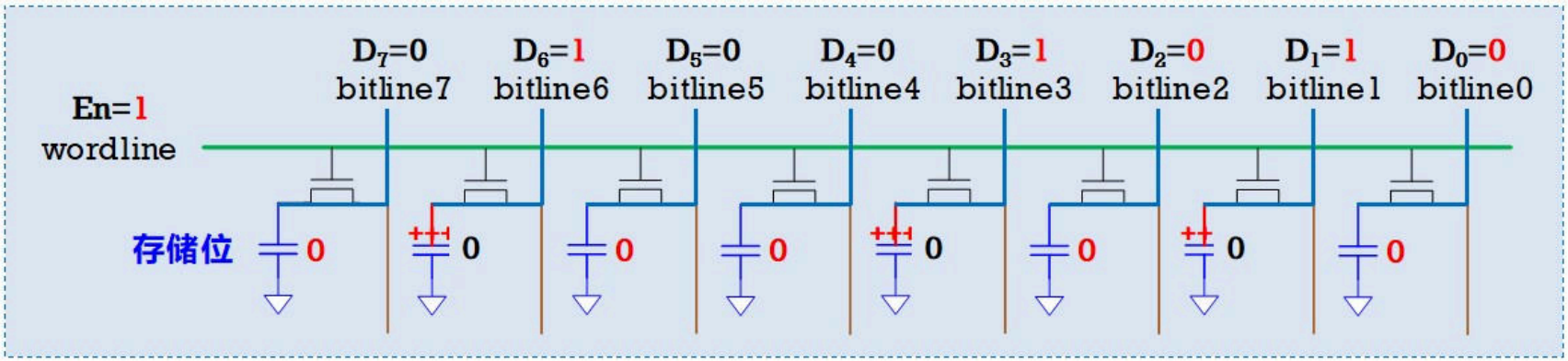


当字线为高电平时，处于读写状态；



动态随机访问存储器DRAM(Dynamic Random Access Memory)

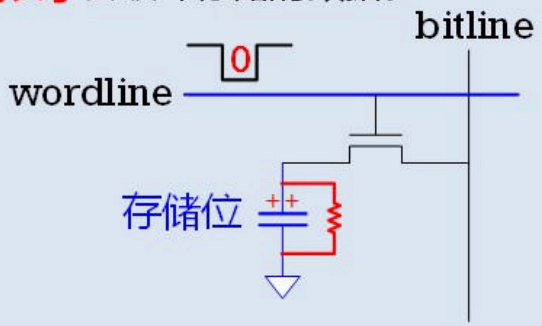
- 读取位单元时，位线初始化为浮空Z
- 随后，字线转换为高电平，允许已存储的值驱动位线为0或者1.
- 读操作会破坏存储的数据



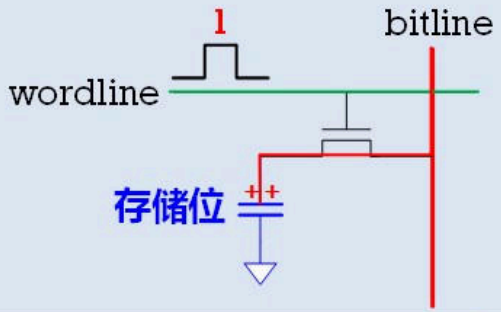
动态随机访问存储器 —— 刷新 Refresh

- DRAM 的数据存储在电容中
- 之所以称之为“**动态(Dynamic)**”或者其具有动态特性，是因为存储信息需要**周期性刷新**，**读取后**也需**重新写入**。

在存储**保持状态**，由于电容**有漏电阻**的存在，**时间长了**会破坏存储的数据。



读操作会带走电容上的电荷，**破坏**存储的**数据**



动态随机访问存储器 —— 刷新 Refresh

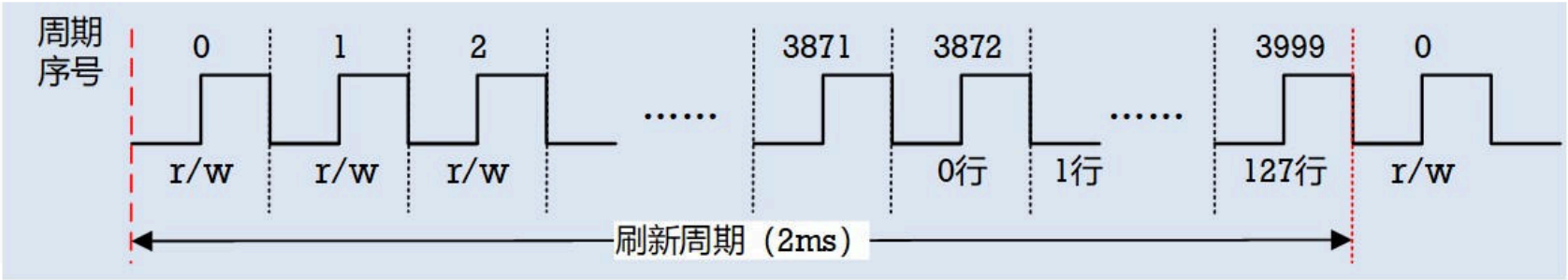
➢ 几个术语

1. **刷新：** 在动态半导体存储器中，由于**漏电**使电容上的**电荷衰减**，需要**定期地重新进行存储**，这个过程称为刷新。
   - ■ 读取操作会破坏数据，需要重新写回，相当于刷新
   - ■ 所以**刷新**就是一次伪**读操作，**只不过**刷新是按行操作**
2. **刷新周期**：对整个 DRAM 必须在一定的时间间隔内完成一次全部单元内容的刷新，否则会出现信息错误。从整个 DRAM 上一次刷新结束到下一次刷新完为止的时间间隔叫刷新周期（刷新周期是不得不重新写入数据的**最长间隔时间**）。
3. **刷新方式**：**集中式**Centralized, **分散式**distributed, **异步式**asynchronous

雨课堂 Rain Classroom

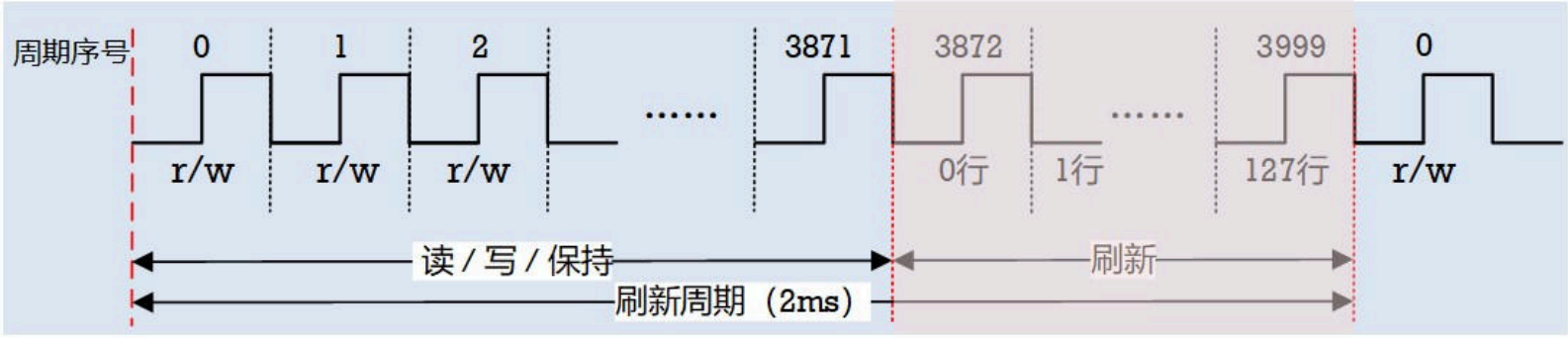
动态随机访问存储器 —— 刷新 Refresh

- 刷新问题举例
  - 假设某 DRAM 存储器阵列结构为**128行**，每个**读写周期为0.5us**，**刷新周期为2ms**。每行需要一个读写周期进行刷新。
  - 则，在一个刷新周期(2ms)内，可以有4000个读写周期(0.5us)
  - 但为了保持数据的正确性，必须在2ms时间内，全部刷新完128行的存储空间。



动态随机访问存储器 —— 刷新 Refresh

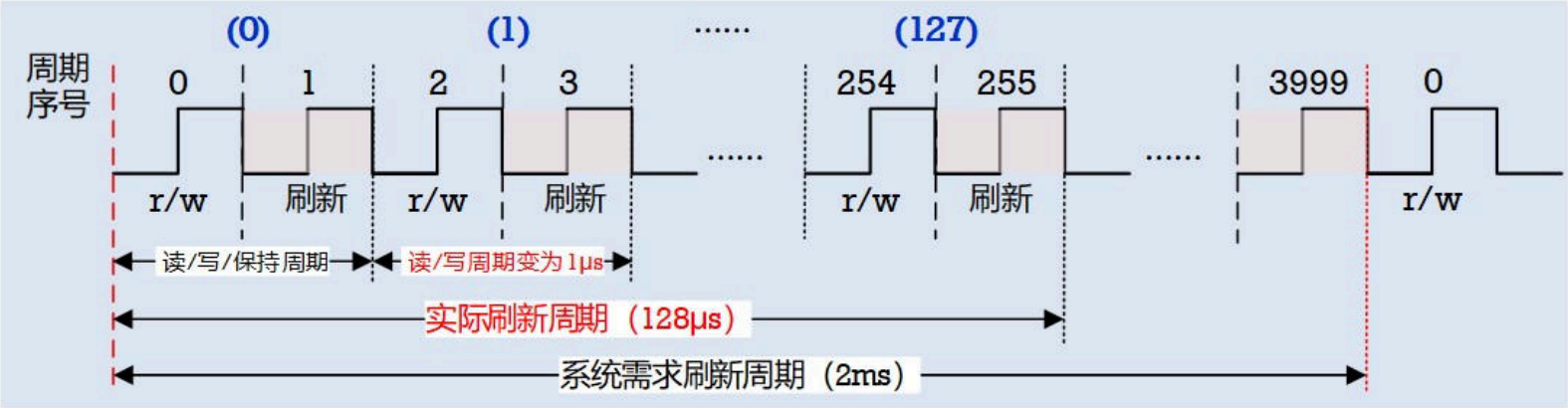
- **集中式刷新：** 在整个刷新周期内，前一段时间正常进行读/写周期或维持周期，等到需要进行刷新操作时，便暂停读/写或维持周期，而逐行刷新整个存储器。适用于高速存储器。



- **优点**：读写周期长度不受影响。
- **缺点**：有大块时间的读写盲区。

动态随机访问存储器 —— 刷新 Refresh

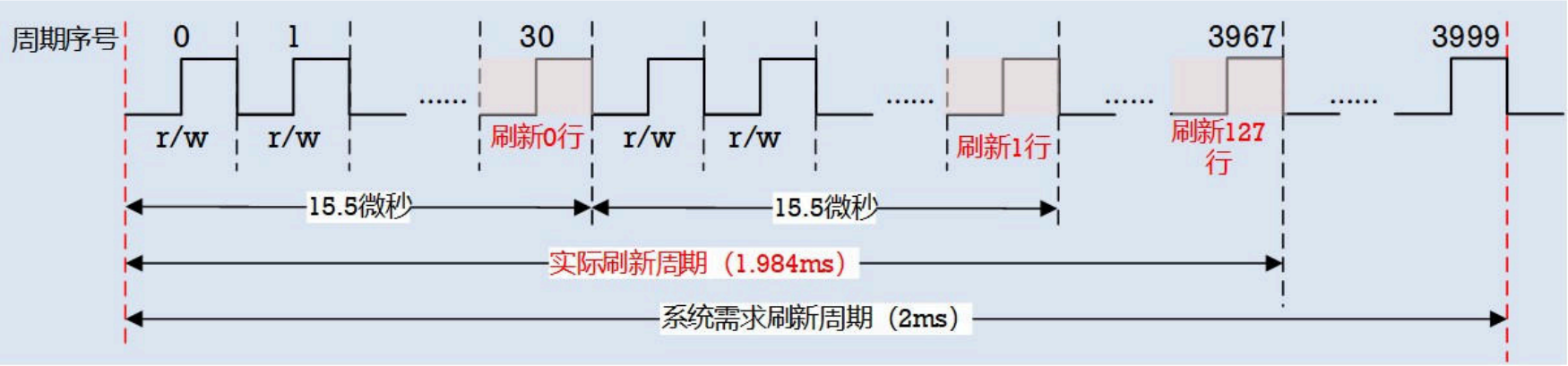
- **分散式刷新：** 把2个系统周期作为一个存储周期Tc，周期前半段（一个系统周期）用来读/写操作或保持信息，周期后半段（一个系统周期）作为刷新操作时间。



- **优点：** 没有大块时间的读写盲区，刷新密集(128μs全部刷新一遍)，数据稳定。
- **缺点：** 读写周期时间变长(慢了一倍，变1μs)，存储速度变慢。

动态随机访问存储器 —— 刷新 Refresh

- **异步刷新方式**: 结合了以上两种优势。4000/128 = 31.25



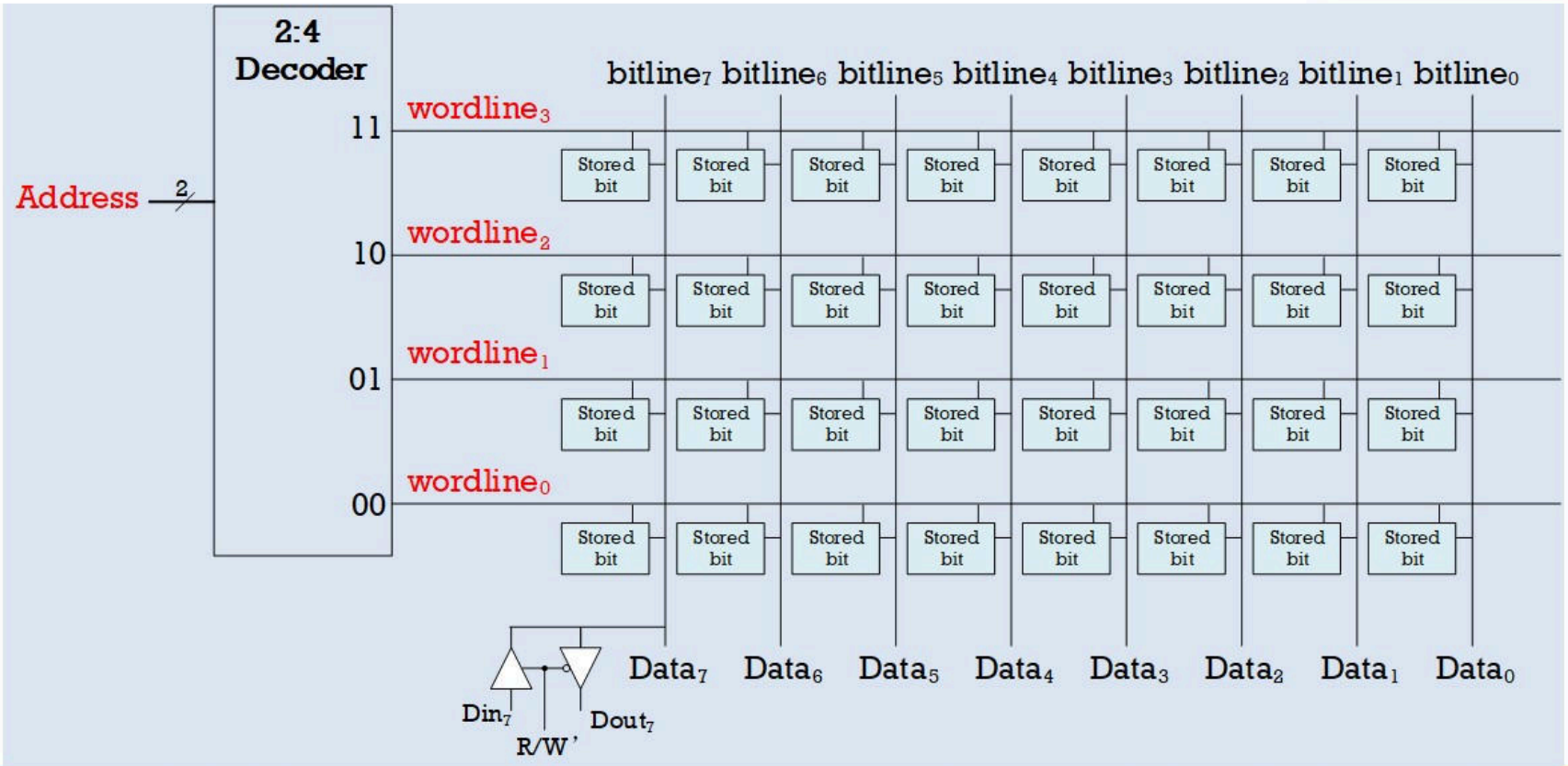
- 将128个刷新使用的读写周期较均匀的分散在2ms的时间内，即每隔31个读写周期拿出一个周期进行刷新
- **优点**：避免集中刷新的读写盲区，也避免了频繁刷新带来的存储速度变慢

第十六堂课结束了

有问题同学们可以提问讨论

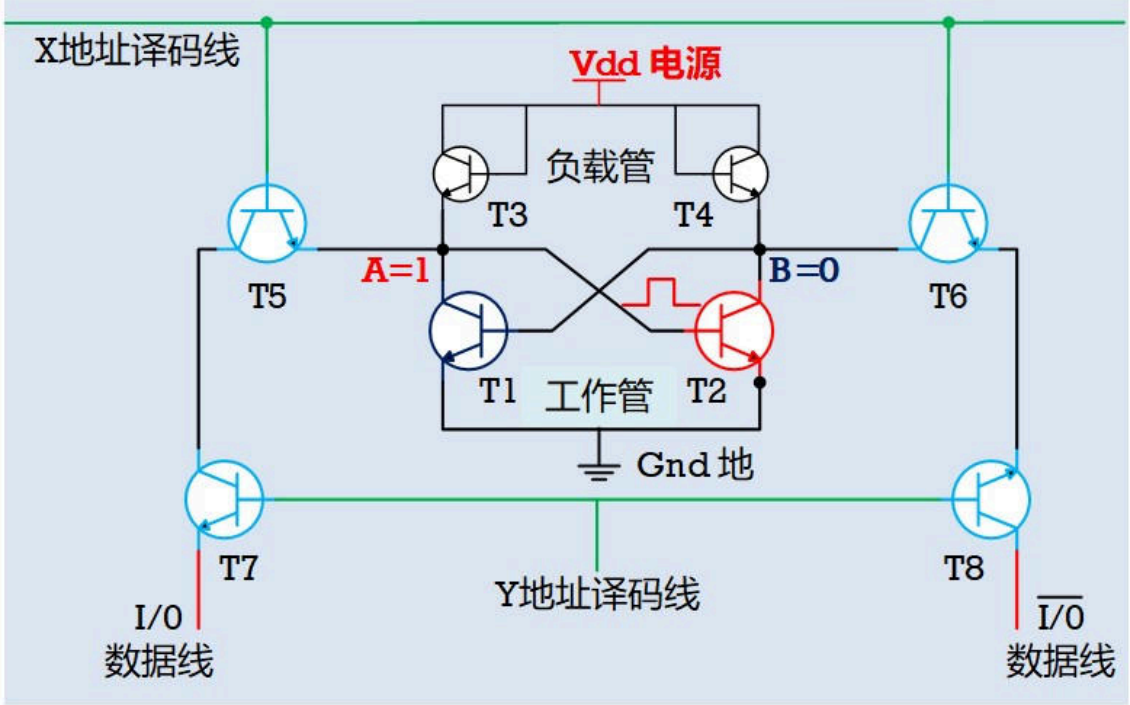
雨课堂 Rain Classroom

存储器阵列


2:4
Decoder
Address
2
11
10
01
00
wordline3
wordline2
wordline1
wordline0
bitline7 bitline6 bitline5 bitline4 bitline3 bitline2 bitline1 bitline0
Stored bit
Data7 Data6 Data5 Data4 Data3 Data2 Data1 Data0
Din7
Dout7
R/W'

静态随机访问存储器SRAM (Static random access memory)

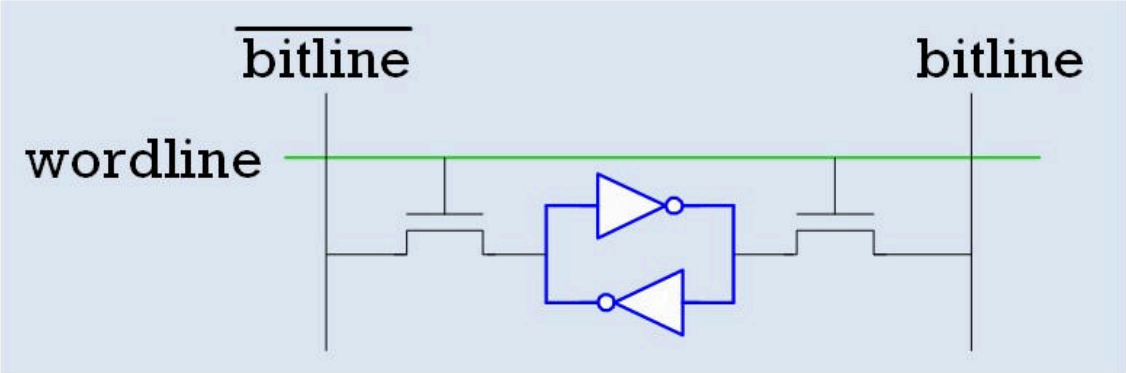
- 6 管 SRAM 存储单元



- **T7, T8**：列译码通路开关，整列共用，属于阵列外围读写电路，不属于单个存储元。

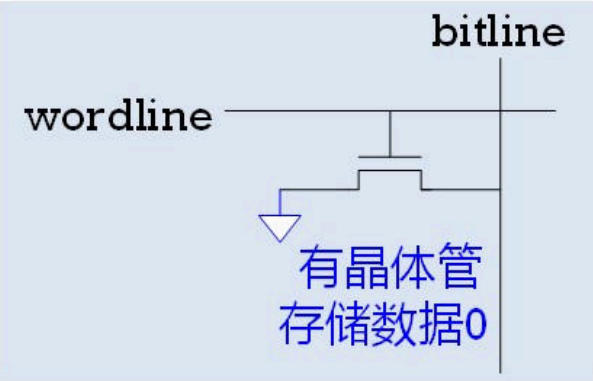
静态随机访问存储器SRAM (Static random access memory)

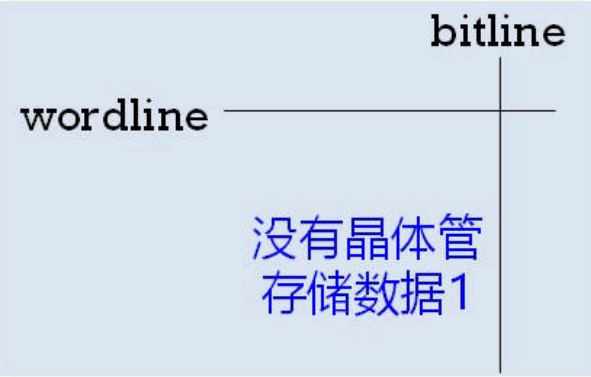
- 教材中的图（如下图），是标准现代 CMOS 六管静态存储元的简化画法。一个反相器相当于 2 个晶体管(PMOS 负载 + NMOS 驱动)。
- 数据存储在交叉耦合反相器（双稳态电路）中，如果噪声减弱了存储位的值，交叉耦合反相器会恢复存储值。所以**不需要刷新存储位**。



只读存储器ROM(Read only memory)

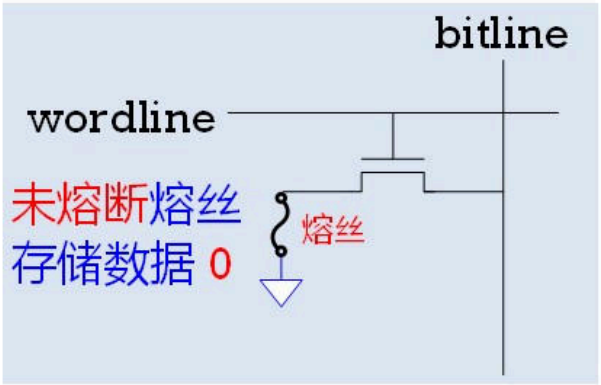
- 只读存储器以晶体管的存在与否来储存一位信息。
- 读这个单元的时候，位线被缓慢的推到高电平，然后字线设置为有效。
  - 如果**晶体管存在**，它会使**位线为低电平，存储数据0。**
  - 如果**晶体管不存在**，**位线**会保持**高电平，存储数据1。**

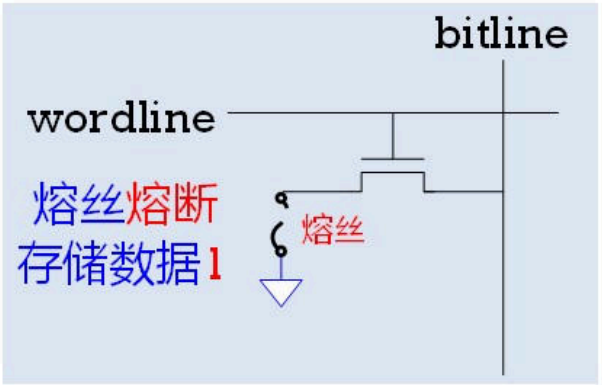




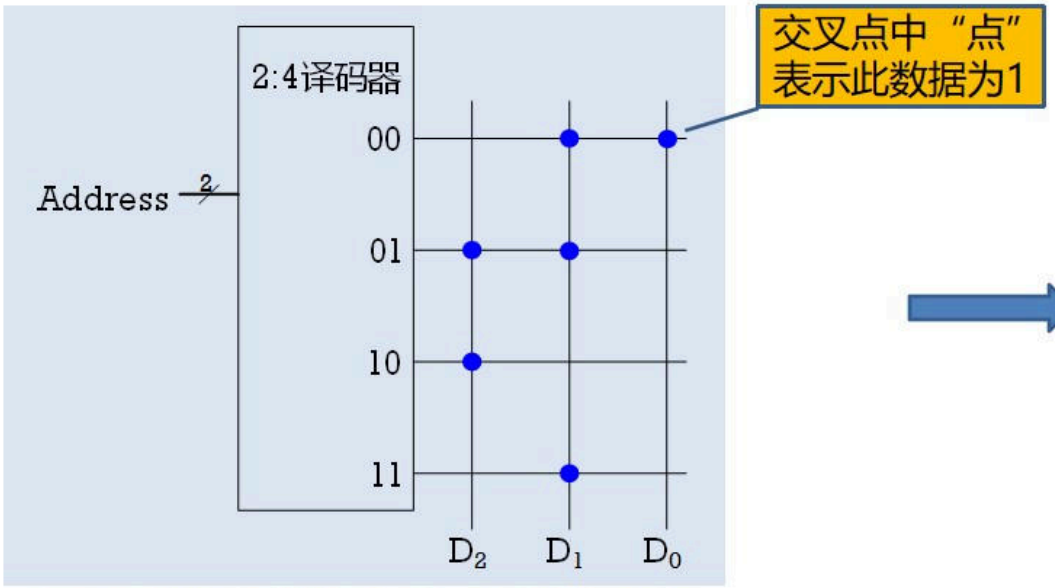
可编程的只读存储器PROM(Programmable Read only memory)

- 应用高电压有选择地熔断熔丝来对ROM编程。
  - 如果熔丝存在，则**晶体管存在**，它会使**位线为低电平，存储数据0**。
  - 如果熔丝熔断，相当于**晶体管不存在(不接地)**，**位线**会保持**高电平，存储数据1**。





只读存储器ROM(Read only memory)



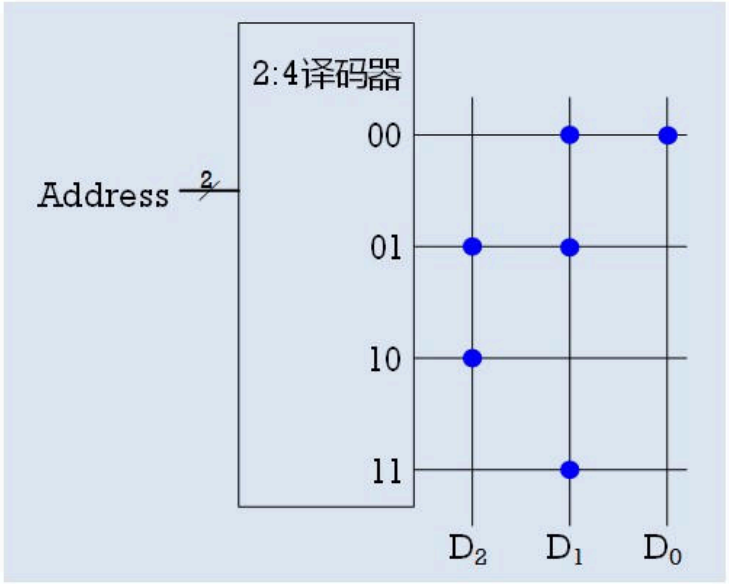
Address	Data		
00	0	1	1
01	1	1	0
10	1	0	0
11	0	1	1

width

存储阵列的简化画法（点表示法）

使用存储器阵列的逻辑

- 尽管存储器阵列最初用于存储数据，但是也可以用于实现组合逻辑功能。
- 如果把地址当成输入，读出的数据当成输出，则此阵列表达的逻辑功能如表：



input: A_1	A_0	output: D_2	D_1	D_0
0	0	0	1	1
0	1	1	1	0
1	0	1	0	0
1	1	0	1	0

$$D_2 = A_1 \oplus A_0$$
$$D_1 = A_1' + A_0$$
$$D_0 = A_1' A_0'$$

使用 ROM 构建逻辑

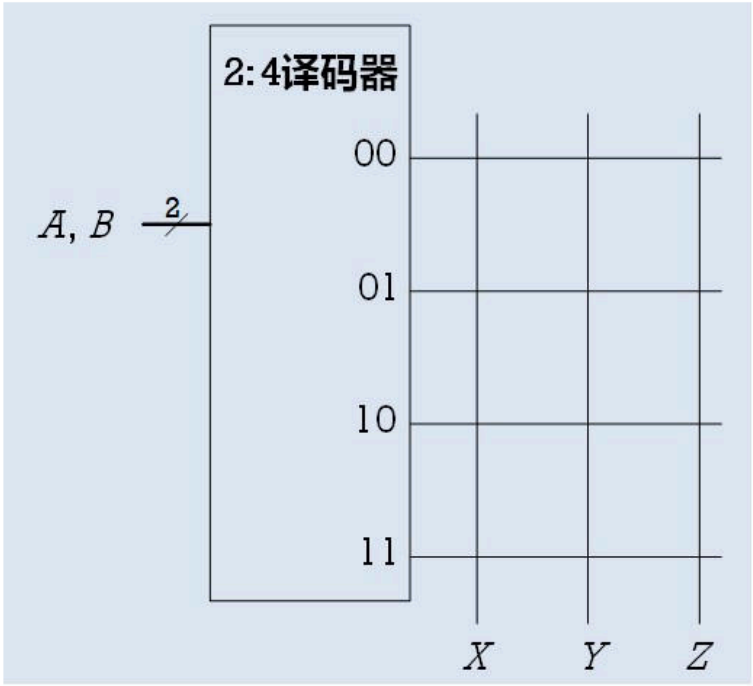
慕课堂提问

- 使用$2^2 \times 3$-bit 的ROM实现以下逻辑功能:

$$X = AB$$

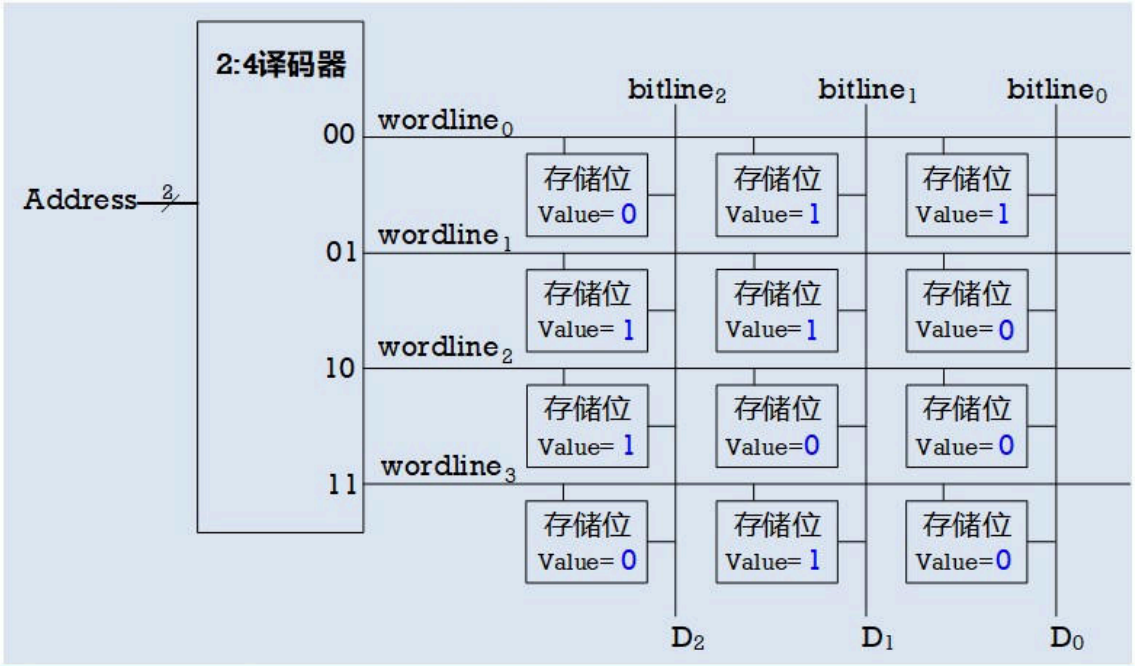
$$Y = A + B$$

$$Z = \overline{AB}$$



使用存储阵列的逻辑

- 原则上说，一个 2^N字×M位 存储器可以实现任何 N输入 和 M输出 的组合逻辑功能。用于执行逻辑的存储阵列称为**查找表** lookup tables (LUTs)



$$D_2 = A_1 \oplus A_0$$
$$D_1 = A_1' + A_0$$
$$D_0 = A_1' A_0'$$

使用存储阵列的逻辑

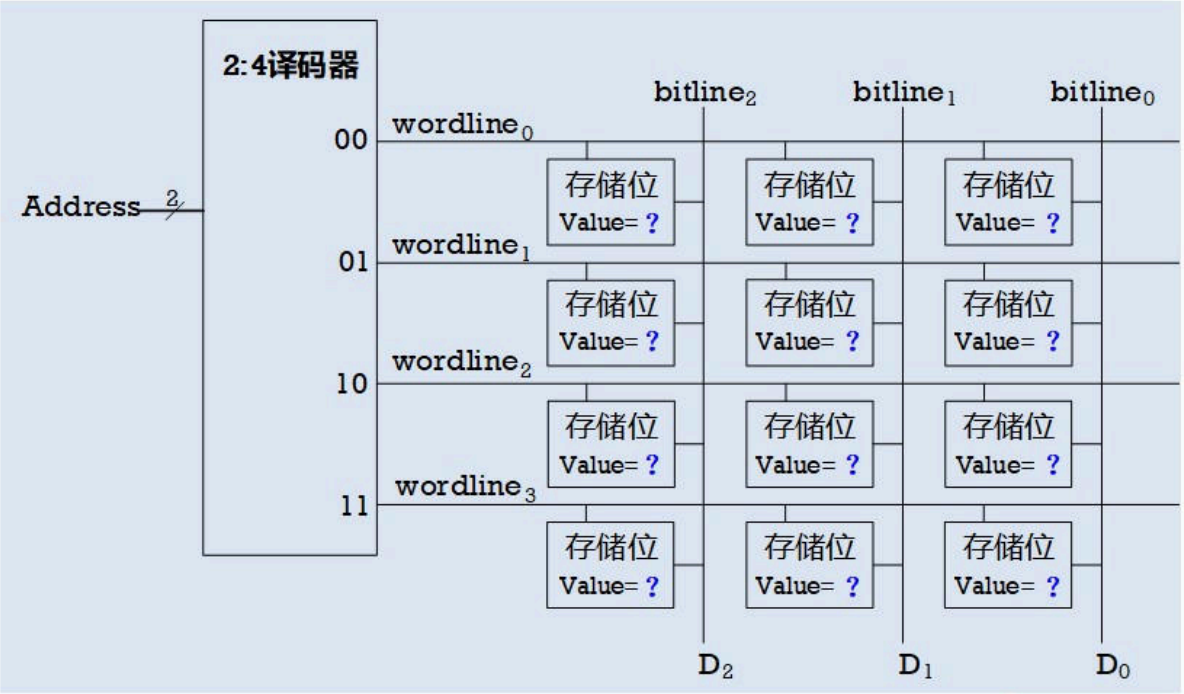
慕课堂提问

- 使用 $2^2 \times 3$-bit 的**存储阵列**实现以下逻辑功能:

$$X = AB$$

$$Y = A + B$$

$$Z = A\,B'$$

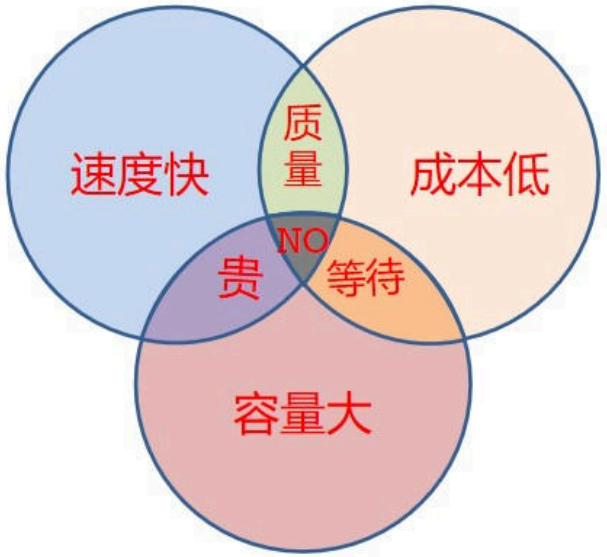


雨课堂 Rain Classroom

存储系统的挑战

- 让存储系统获得接近处理器的访问速度
- 采用存储器**层次结构**
- 理想存储器具备三大特性:
  - 速度快
  - 成本低廉
  - 容量大

但最多只能同时具备两个优势!



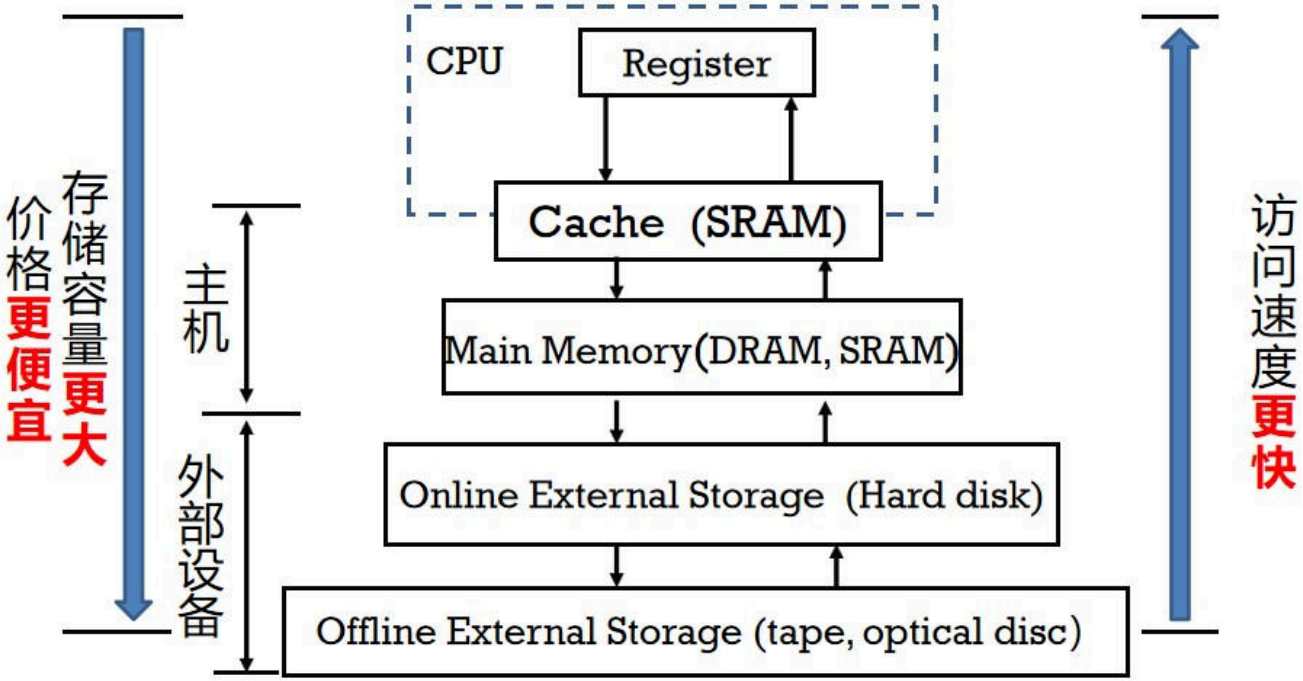
存储系统的挑战

- 存储层次由**主存 - 辅存两级架构**演变为多级存储体系。
- 主存由**单体**发展为**多体并行**结构。
- 采取**虚拟存储**技术（第 8 章介绍）。

雨课堂 Rain Classroom

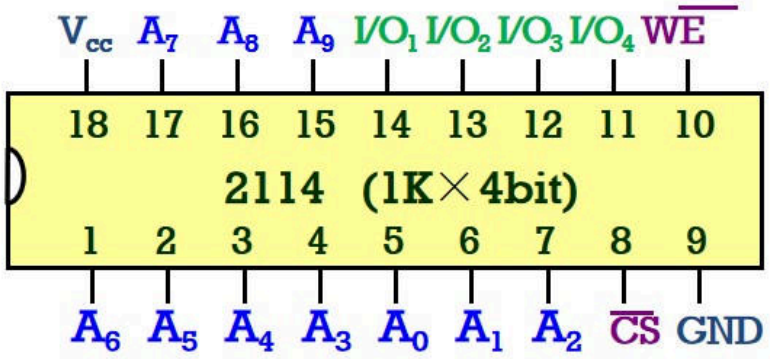
多层次存储体系结构

- 主存的速度总落后于CPU的需要，主存的容量总落后于软件的需要。



随机读写存储器RAM

- 存储器——芯片实例 Intel 2114
  - 外部引脚功能：采用18脚封装，如下图所示：



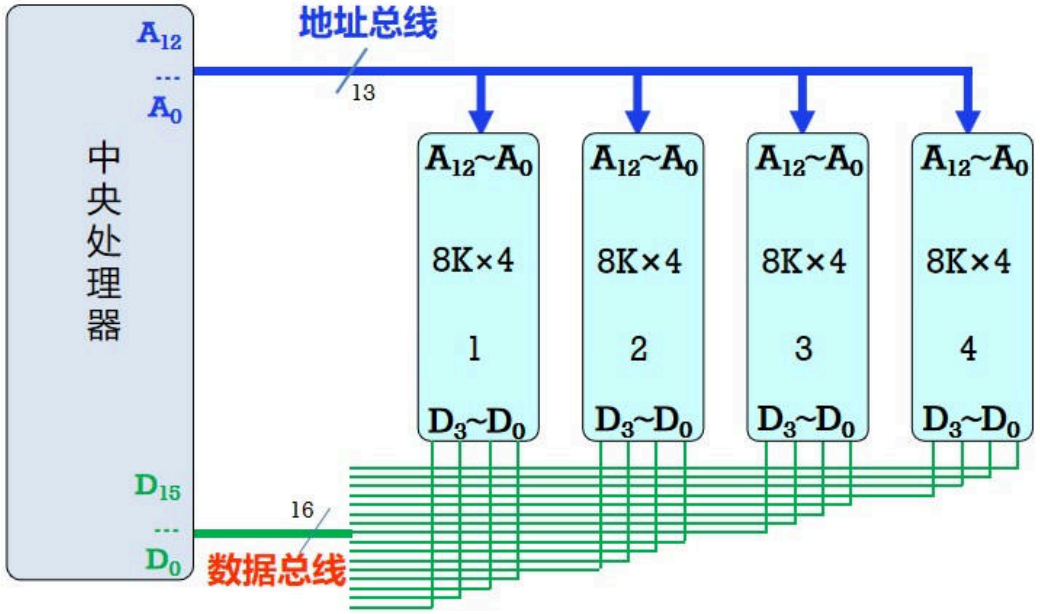
  - 注意：RAM芯片中引脚的安排
  - 引脚功能：电源、数据总线、地址总线、控制总线

随机读写存储器RAM

- RAM存储器——存储器与CPU的连接
  - 存储器芯片的容量是有限的，为了满足实际存储器的容量要求，需要对存储器进行扩展。主要方法有：
    - **位扩展法**：只加大字长，而存储器的字数与存储器芯片字数一致，对片子没有选片要求。
    - **字扩展法**：仅在字向扩充，而位数不变。需由片选信号来区分各片地址。
    - **位字扩展法**：位扩展法和字扩展法的综合

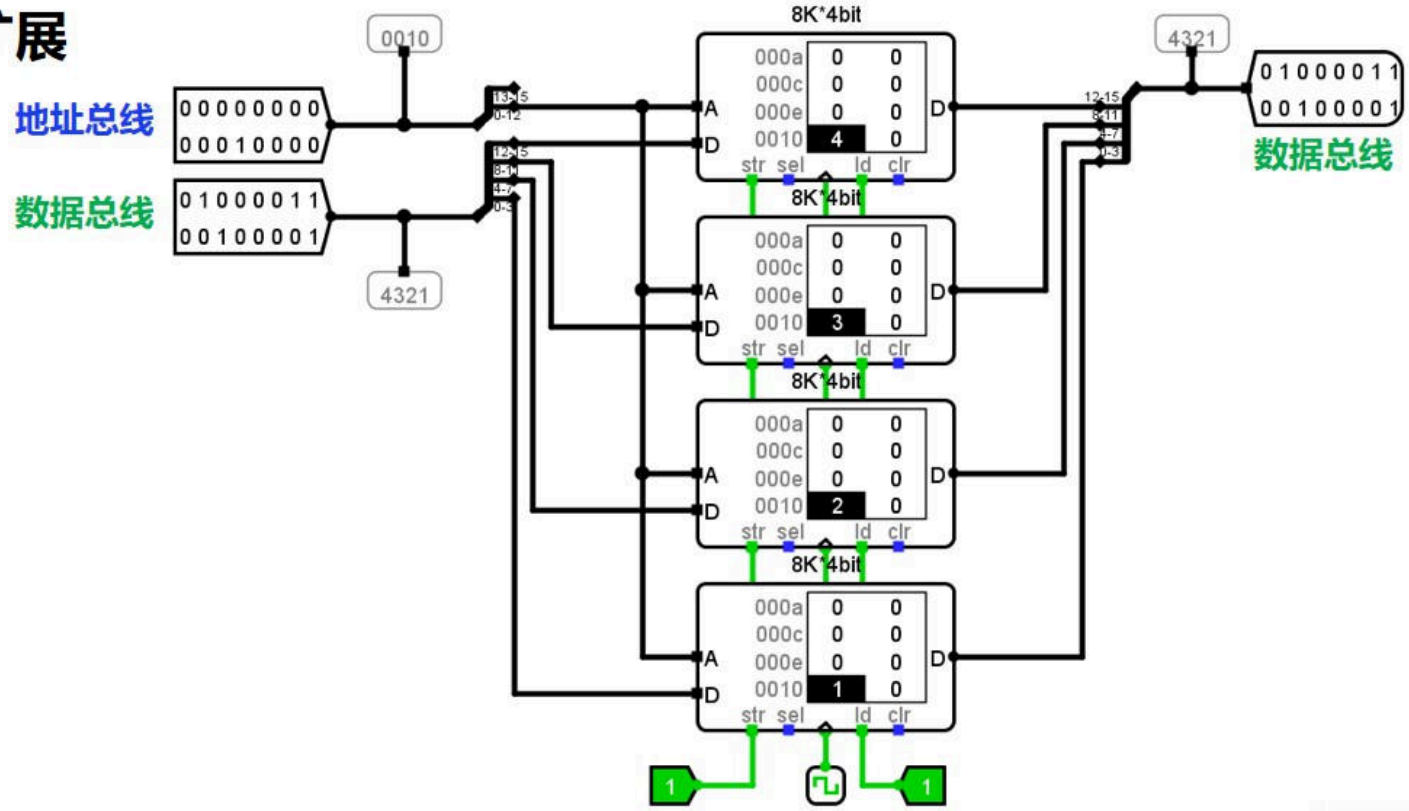
RAM存储器——存储器与CPU的连接

➢ **位扩展**（举例：使用 **8K×4bit** 的RAM存储芯片组成 **8K×16bit** 的存储器）



RAM存储器——存储器与CPU的连接

➢ 位扩展



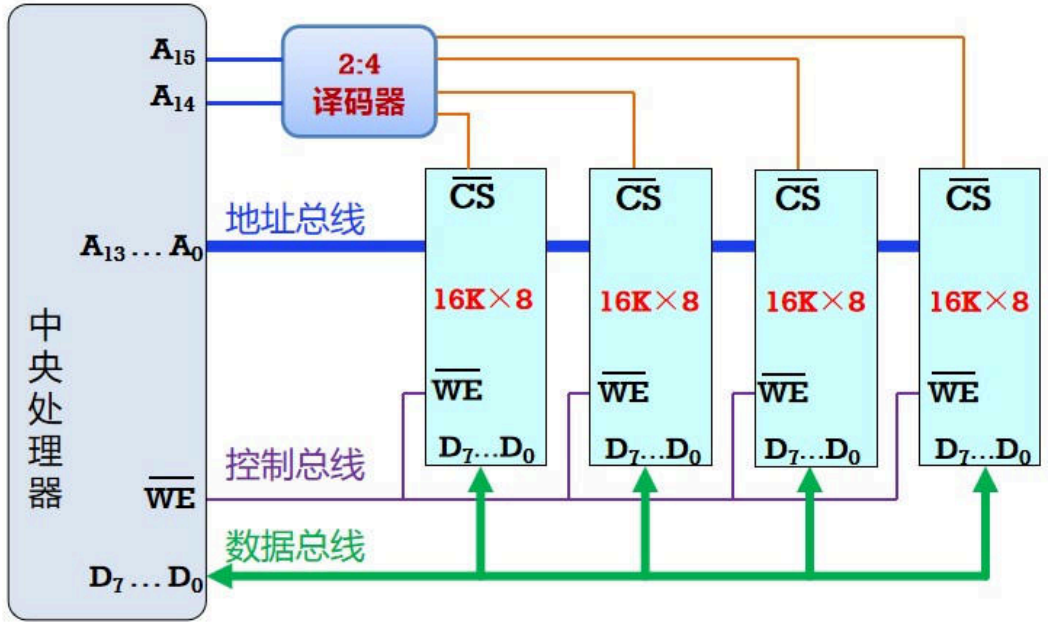
RAM存储器——存储器与CPU的连接

- **字扩展**（举例：用 16K×8bit 的芯片组成 64K×8bit 的存储器）
  - 解: 地址分配与片选逻辑

地址块	片外(片选)	片内			
	$A_{15}A_{14}$	$A_{13}A_{12}$	$A_{11}A_{10}A_9A_8$	$A_7A_6A_5A_4$	$A_3A_2A_1A_0$
1	0 0 0 0	0 0 1 1	0 0 0 0 1 1 1 1	0 0 0 0 1 1 1 1	0 0 0 0 1 1 1 1
2	0 1 0 1	0 0 1 1	0 0 0 0 1 1 1 1	0 0 0 0 1 1 1 1	0 0 0 0 1 1 1 1
3	1 0 1 0	0 0 1 1	0 0 0 0 1 1 1 1	0 0 0 0 1 1 1 1	0 0 0 0 1 1 1 1
4	1 1 1 1	0 0 1 1	0 0 0 0 1 1 1 1	0 0 0 0 1 1 1 1	0 0 0 0 1 1 1 1

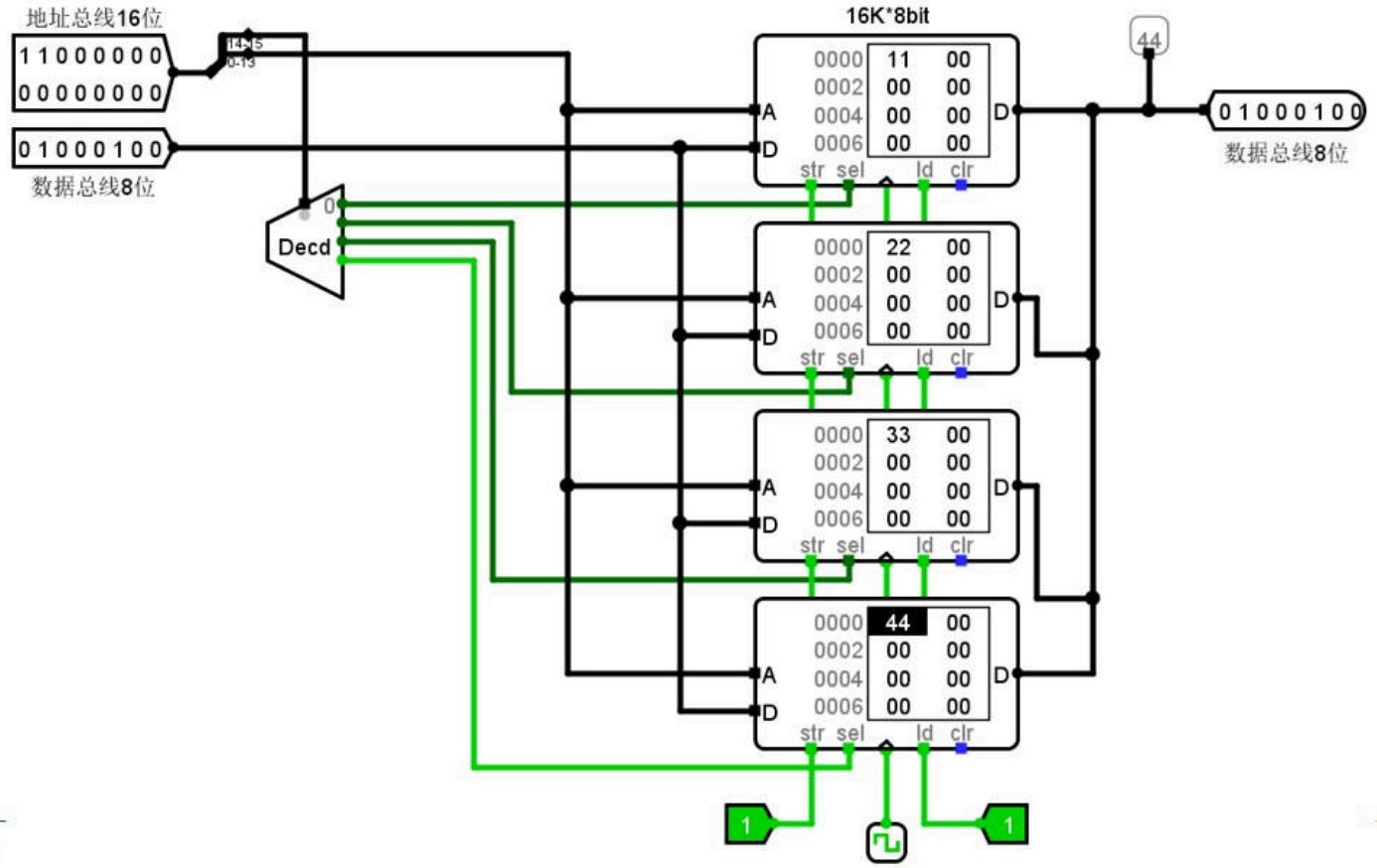
RAM存储器——存储器与CPU的连接

➢ **字扩展**（举例：用 16K×8bit 的芯片组成 64K×8bit 的存储器）



RAM存储器——存储器与CPU的连接

➢ 字扩展



NEU-SWC-A0801040040

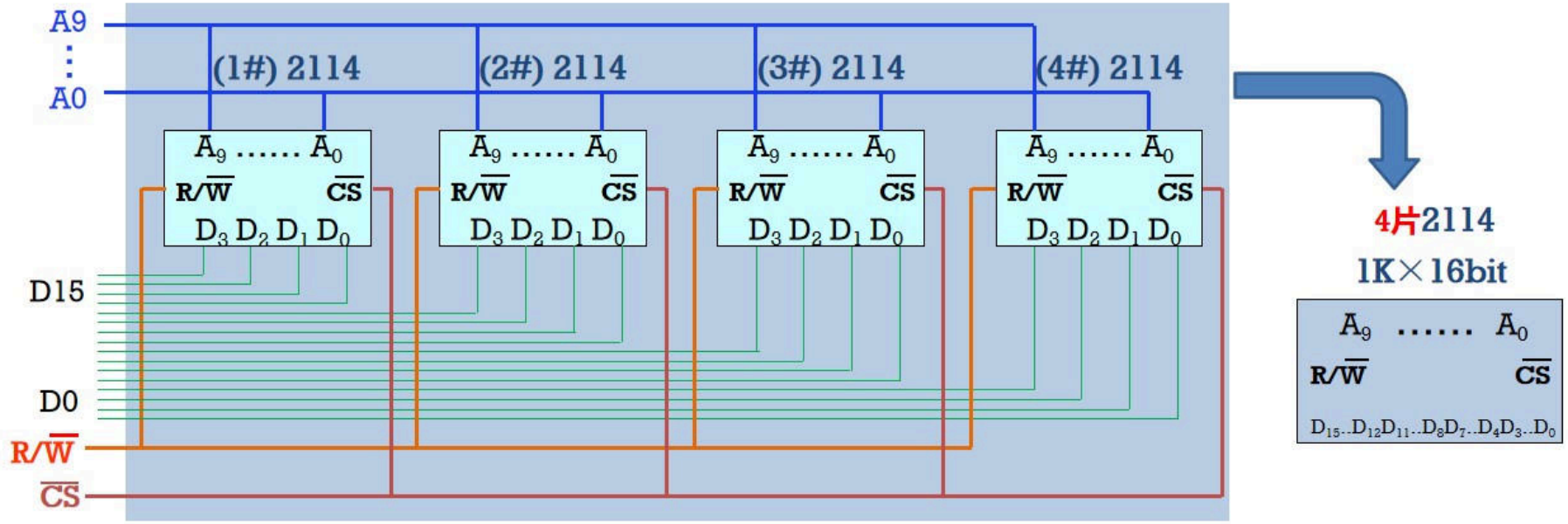
RAM存储器——存储器与CPU的连接

- **位-字 扩展法**（举例：现有2114即1K×4bit RAM芯片，要构成 8K×16bit 主存）
  - 应该用多少片2114？画出扩展、连接图。
    - 首先计算用多少片2114：(8K×16)/(1K×4)=32片；
    - 然后进行位扩展：把1K×4bit 扩成1K×16bit，用16/4=4片；
    - 最后进行字扩展：1K字—8K字，用上面位扩展得到的 1K×16bit 单元共 8K/1K=8个，即总共用2114位 8×4=32 片

雨课堂 Rain Classroom

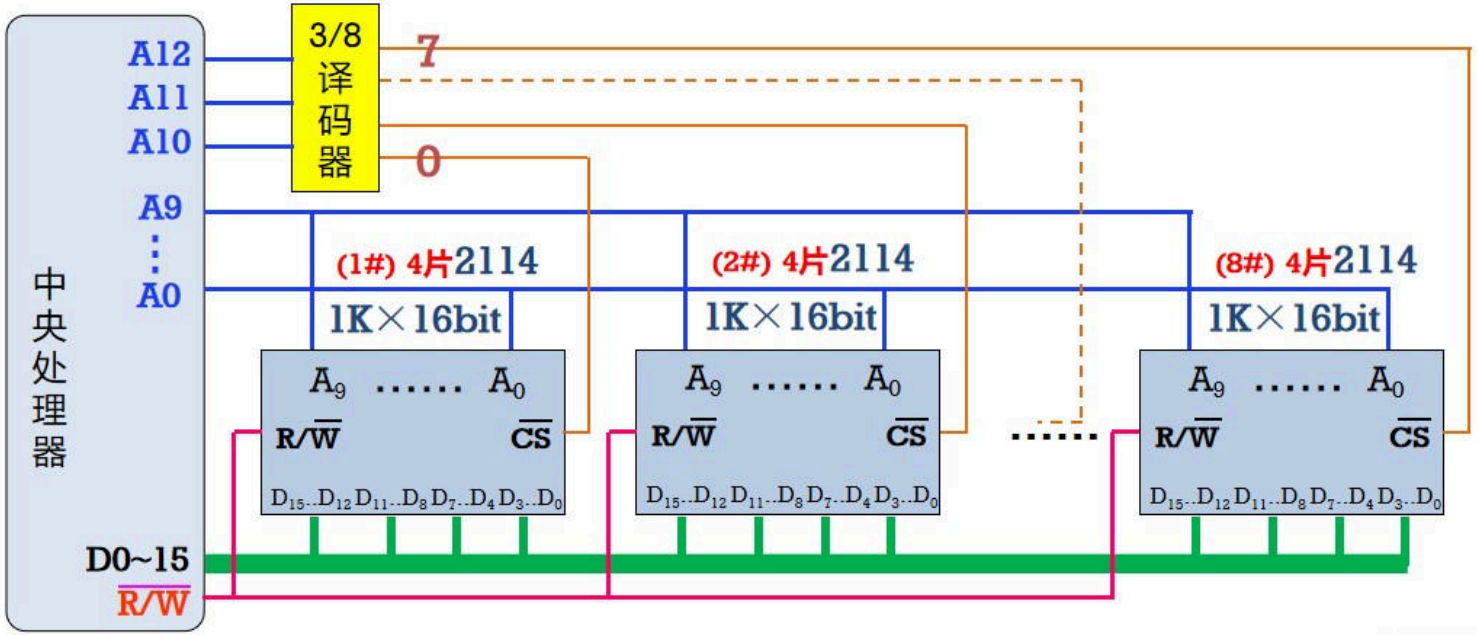
RAM存储器——存储器与CPU的连接

- 位-字 扩展法（举例：用2114即1K×4 RAM芯片，构成8K×16位主存）
  - 进行位扩展：把1K×4扩成1K×16，用16/4=4片；



RAM存储器——存储器与CPU的连接

- 位-字 扩展法（举例：用2114即1K×4 RAM芯片，构成8K×16位主存）
  - 最后进行字扩展：1K字—8K字



RAM存储器——存储器与CPU的连接

- 芯片级存储器逻辑图应表示出:
  - 所用存储芯片
  - 各芯片的地址线、数据线
  - 片选逻辑（CS'）
  - 读/写控制R/W和存储器访问线MREQ

- 注意:
  - 地址线、数据线的数量和方向
  - 选片地址通过译码后产生存储器的片选信号。当各芯片容量相同时，地址范围规整，可选用现成译码器；否则，可选用适当门电路。

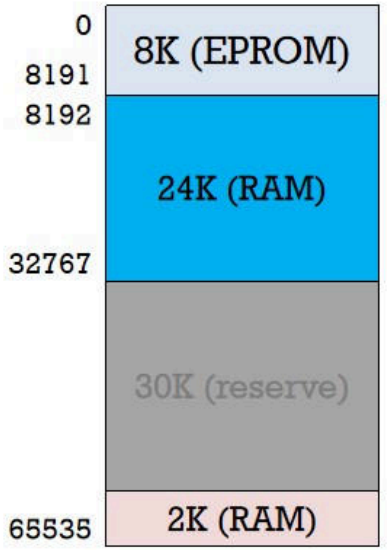
ROM、RAM与CPU的连接

- 按照指定的地址空间分配，正确选择所给各种存储器芯片及其它片子、门电路等，将对应的地址线、数据线、控制线连接起来，构成较完整的处理器与存储器的相连系统。

雨课堂 Rain Classroom

ROM、RAM与CPU的连接

- 举例：CPU的地址总线16根(A15—A0，A0为低位)，双向数据总线8根(D7—D0)，控制总线中与主存有关的信号有MREQ'(允许访存，低电平有效)，R/W'(高电平为读命令，低电平为写命令)。
  - 主存地址空间分配如下：
    - ① 0—8191为系统程序区，由只读存储芯片组成；
    - ② 8192—32767为用户程序区；
    - ③ 最后(最大地址)2K地址空间为系统程序工作区。
    - ■ 上述地址为十进制，按字节编址。

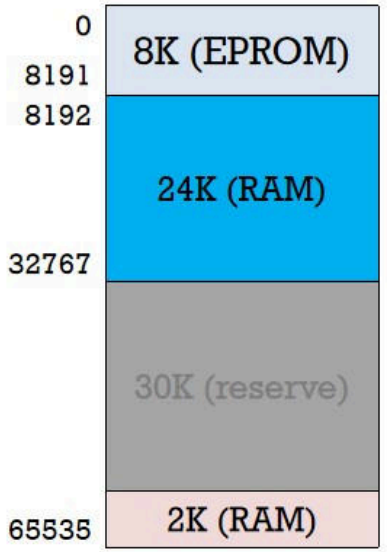


ROM、RAM与CPU的连接

- 现有如下存储器芯片：
  - EPROM：8K×8位(控制端仅有CS)；
  - RAM：16K×1位，2K×8位，4K×8位，8K×8位.

(1) 请从上述芯片中选择适当芯片设计该计算机的存储系统，说明选哪些存储器芯片，选多少片；

(2) 画出存储系统逻辑框图，注意画出片选逻辑（可选用门电路及3:8译码器74LS138）与CPU 的连接。

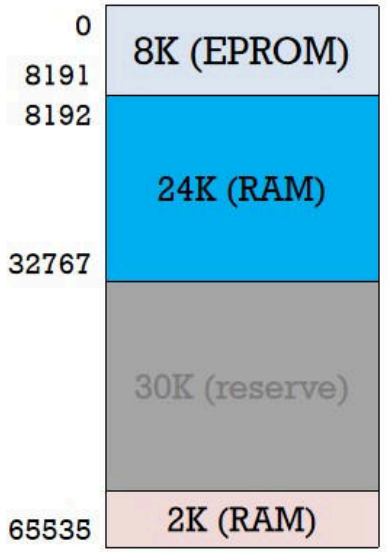


ROM、RAM与CPU的连接

- 现有如下存储器芯片：
  - EPROM：8K×8位(控制端仅有CS)；
  - RAM：16K×1位，2K×8位，4K×8位，8K×8位.

(1) 请从上述芯片中选择适当芯片设计该计算机的存储系统，说明选哪些存储器芯片，选多少片；

(2) 画出存储系统逻辑框图，注意画出片选逻辑（可选用门电路及3:8译码器74LS138）与CPU 的连接。



解： 根据给定条件，选用

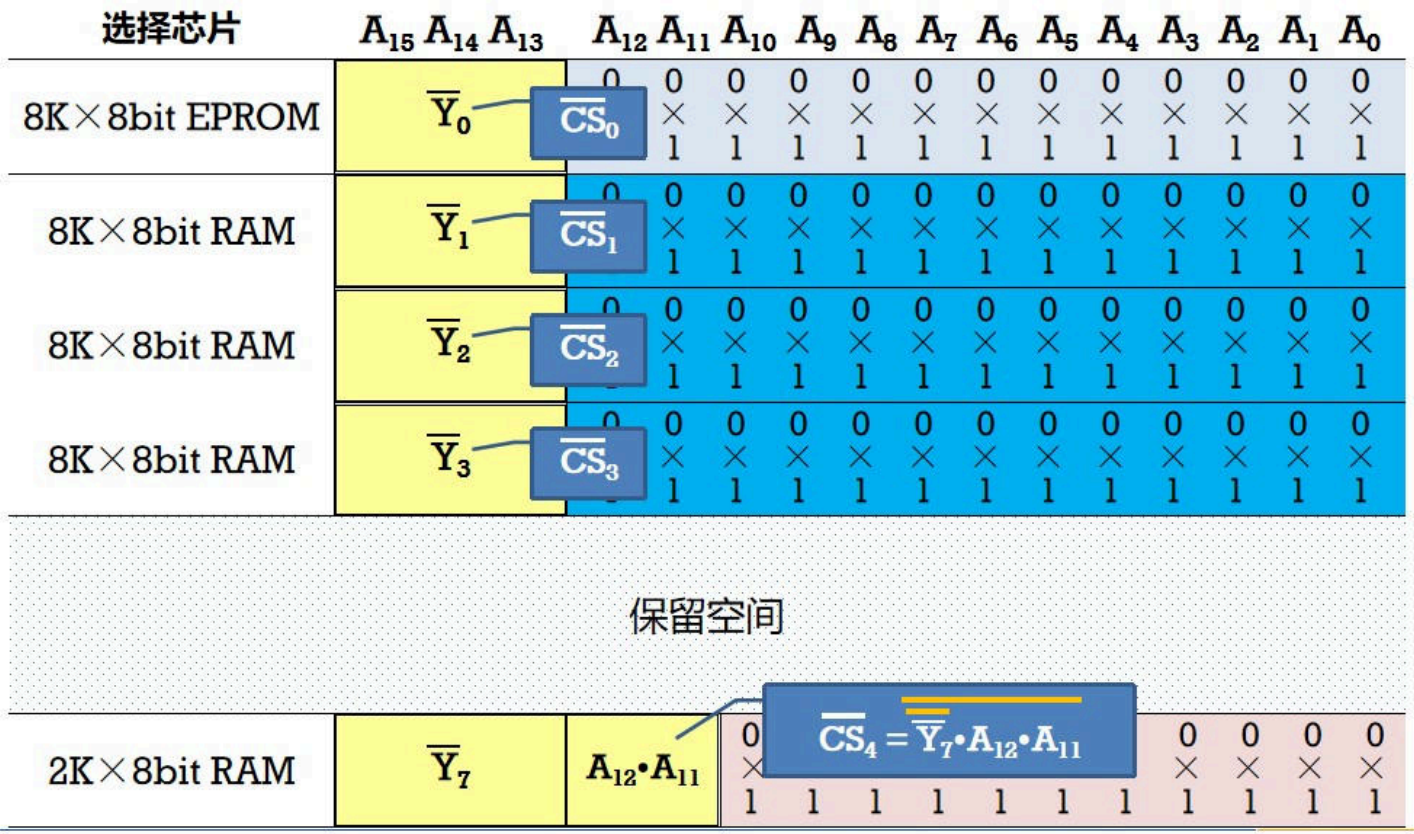
- EPROM：8K×8位芯片1片
- RAM：8K×8位芯片3片，2K×8位芯片1片

完整列出二进制表示的地址空间分配情况

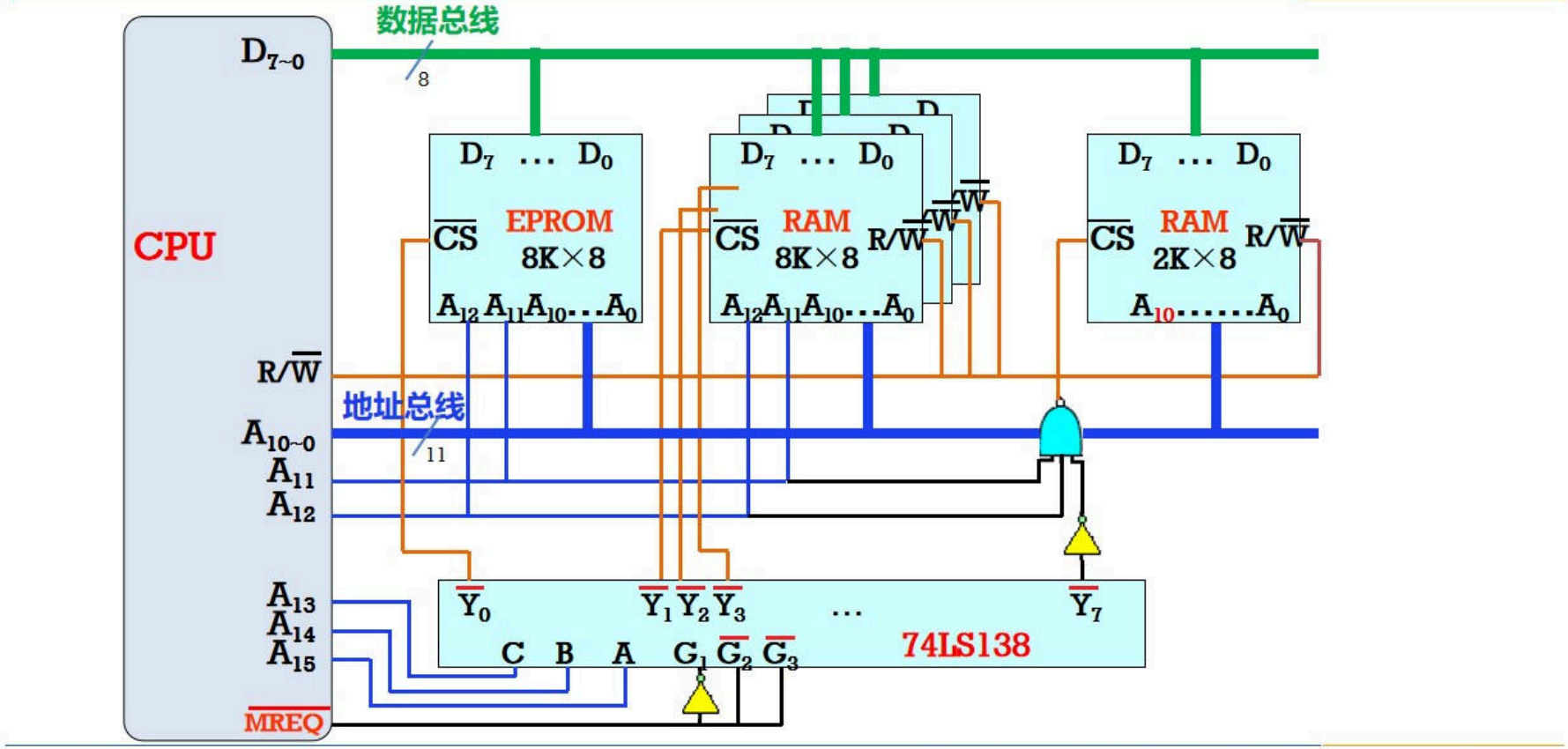
选择芯片	A_{15} A_{14} A_{13}	A_{12} A_{11} A_{10} A_9 A_8 A_7 A_6 A_5 A_4 A_3 A_2 A_1 A_0
8K×8bit EPROM	0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0 0 0 × × × × × × × × × × × × × 1 1 1 1 1 1 1 1 1 1 1 1 1
8K×8bit RAM	0 0 1 0 0 1 0 0 1	0 0 0 0 0 0 0 0 0 0 0 0 0 × × × × × × × × × × × × × 1 1 1 1 1 1 1 1 1 1 1 1 1
8K×8bit RAM	0 1 0 0 1 0 0 1 0	0 0 0 0 0 0 0 0 0 0 0 0 0 × × × × × × × × × × × × × 1 1 1 1 1 1 1 1 1 1 1 1 1
8K×8bit RAM	0 1 1 0 1 1 0 1 1	0 0 0 0 0 0 0 0 0 0 0 0 0 × × × × × × × × × × × × × 1 1 1 1 1 1 1 1 1 1 1 1 1
保留空间		
2K×8bit RAM	1 1 1 1 1 1 1 1 1	1 1 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 × × × × × × × × × × × × × 1 1 1 1 1 1 1 1 1 1 1 1 1

雨课堂 Rain Classroom

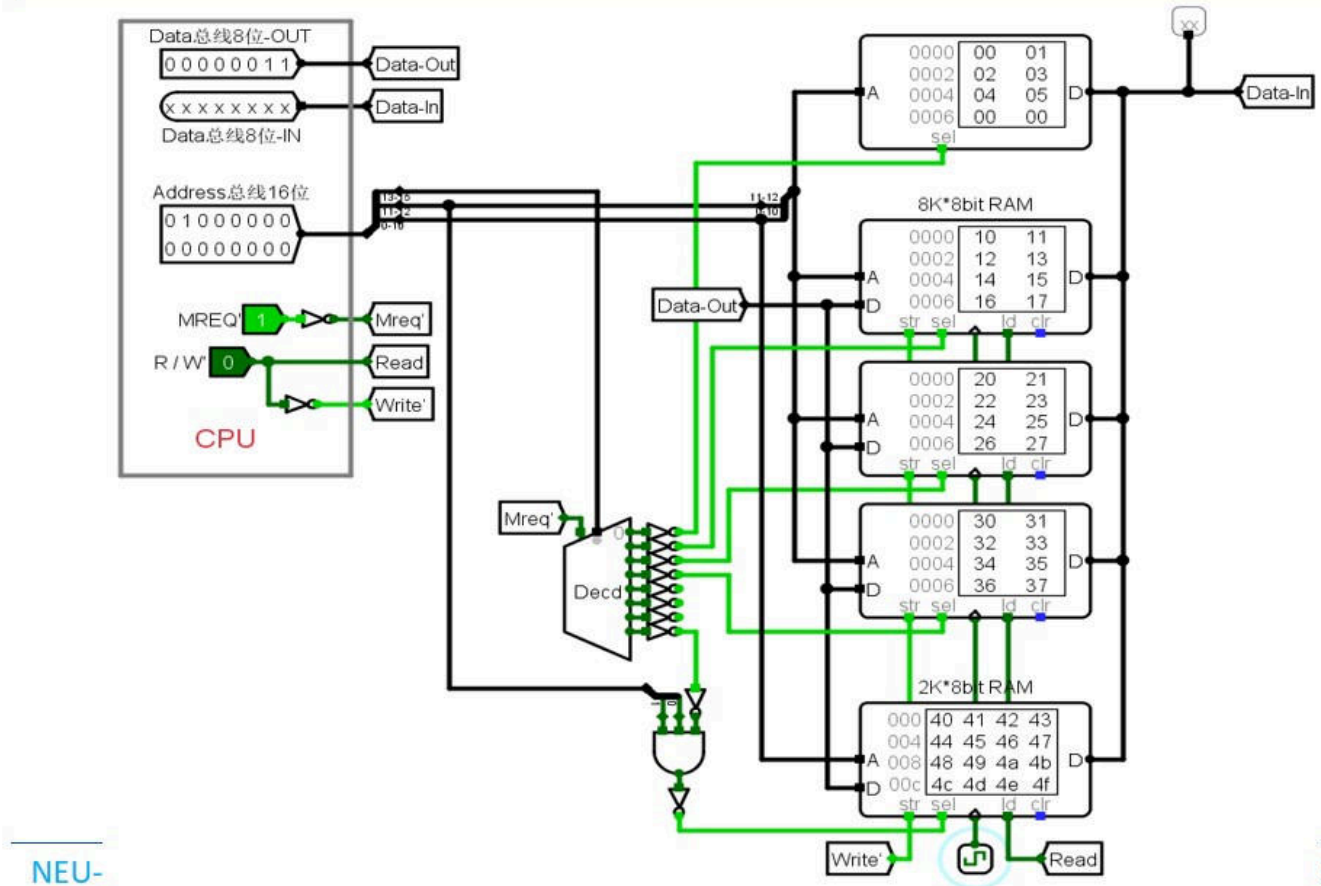
完整列出二进制表示的地址空间分配情况



ROM、RAM与CPU的连接



ROM、RAM与CPU的连接



第十七堂课结束了

有问题同学们可以提问讨论

雨课堂 Rain Classroom

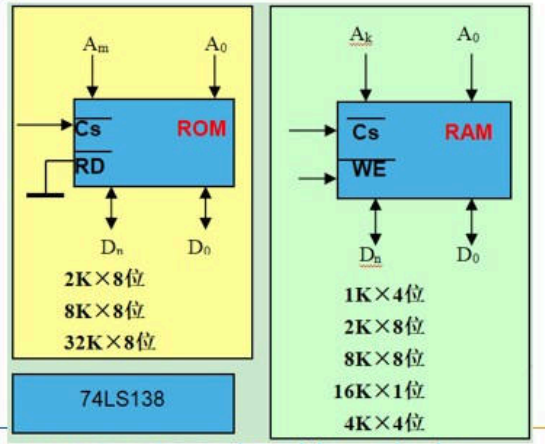
讨论题1：ROM、RAM与CPU的连接

- 设CPU共有16根地址线，8根数据线，并用MREQ'作访存控制信号（低电平有效），用R/W' 作读写控制信号（高电平为读，低电平为写），现有下列芯片及各种门电路（自定），如图。画出CPU与存储器的连接图。要求：

（1）存储芯片地址空间分配为：最高4K地址空间作为系统程序区，相邻的4K空间为系统程序工作区，最低16K空间为用户程序区；

（2）指出选用的存储芯片类型及数量；

（3）详细画出片选逻辑。

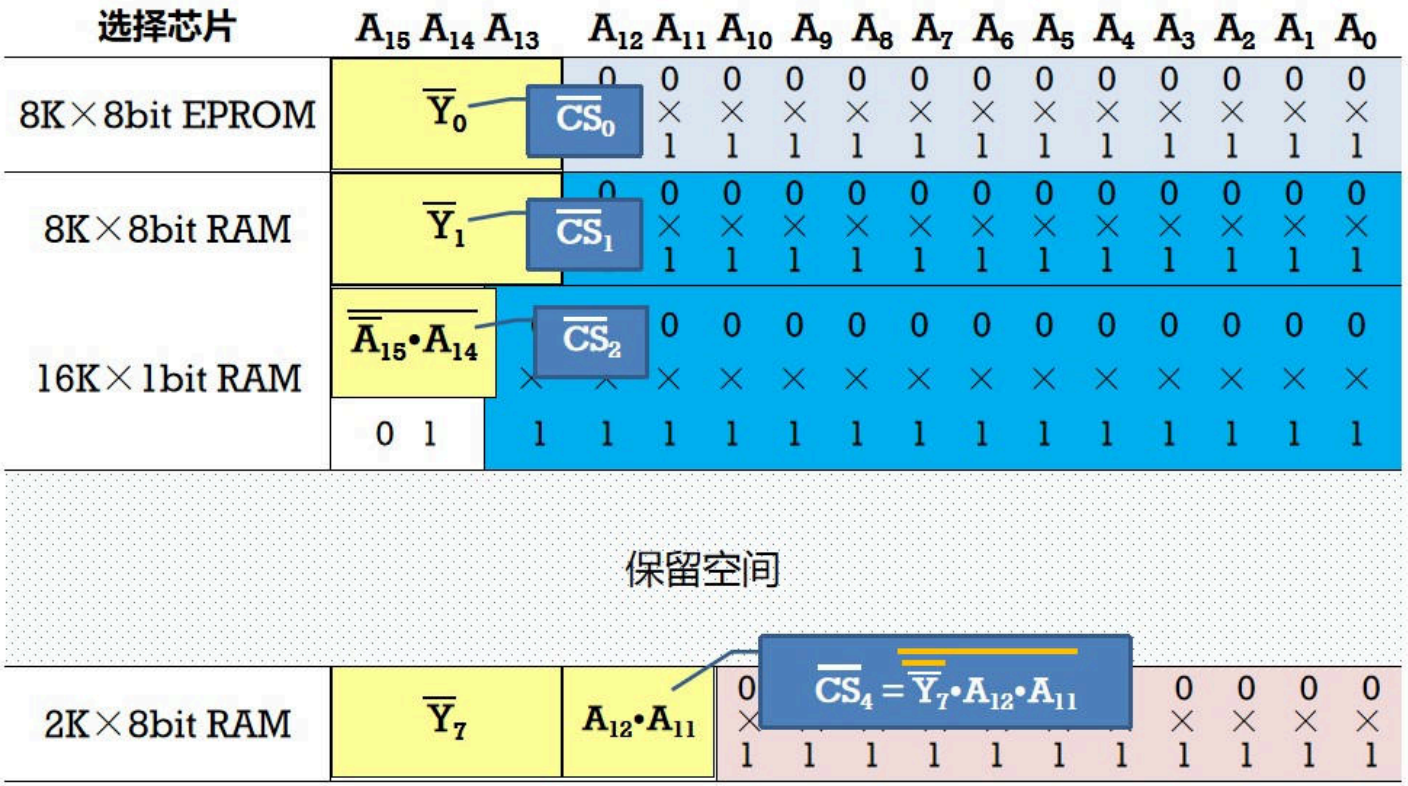


完整列出二进制表示的地址空间分配情况

选择芯片	A_{15}	A_{14}	A_{13}	A_{12}	A_{11}	A_{10}	A_9	A_8	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0
8K×8bit EPROM	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	×	×	×	×	×	×	×	×	×	×	×	×	×
	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1
8K×8bit RAM	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	1	×	×	×	×	×	×	×	×	×	×	×	×	×
	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1
16K×1bit RAM	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	1	×	×	×	×	×	×	×	×	×	×	×	×	×	×
	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
保留空间																
2K×8bit RAM	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0
	1	1	1	1	1	×	×	×	×	×	×	×	×	×	×	×
	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

雨课堂 Rain Classroom

完整列出二进制表示的地址空间分配情况



完整列出二进制表示的地址空间分配情况

选择芯片	A_{15} A_{14} A_{13}	A_{12}	A_{11}	A_{10}	A_9	A_8	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0
8K×8bit EPROM	$\overline{Y_0}$ ($\overline{CS_0}$)	0	0	0	0	0	0	0	0	0	0	0	0	0
		×	×	×	×	×	×	×	×	×	×	×	×	×
		1	1	1	1	1	1	1	1	1	1	1	1	1
4K×8bit RAM	$\overline{Y_1}$	$\overline{A_{12}}$ ($\overline{CS_1} = \overline{Y_1} \cdot \overline{A_{12}}$)	0	0	0	0	0	0	0	0	0	0	0	0
			×	×	×	×	×	×	×	×	×	×	×	×
			1	1	1	1	1	1	1	1	1	1	1	1
4K×8bit RAM	$\overline{Y_1}$	A_{12} ($\overline{CS_2} = \overline{Y_1} \cdot A_{12}$)	×	×	×	×	×	×	×	×	×	×	×	×
4K×8bit RAM	0 1 0	0	×	×	×	×	×	×	×	×	×	×	×	×
4K×8bit RAM	0 1 0	1	×	×	×	×	×	×	×	×	×	×	×	×
4K×8bit RAM	0 1 1	0	×	×	×	×	×	×	×	×	×	×	×	×
4K×8bit RAM	0 1 1	1	×	×	×	×	×	×	×	×	×	×	×	×
保留空间														
2K×8bit RAM	1 1 1 1 1 1 1 1 1	1 1 1 1 1 1	0	0	0	0	0	0	0	0	0	0	0	0
			×	×	×	×	×	×	×	×	×	×	×	×
			1	1	1	1	1	1	1	1	1	1	1	1

雨课堂 Rain Classroom

存储体系结构的发展

① 由主-辅二级结构发展到多层次存储体系结构。

② 主存由单体发展到多体交叉（并行）。

③ 采用了虚拟存储技术。（第八章介绍）

雨课堂 Rain Classroom

多模块交叉存储器——线性地址的编址方式

- 线性编址的含义：即连续编址。
- 线性地址在多模块中有两种安排方式：

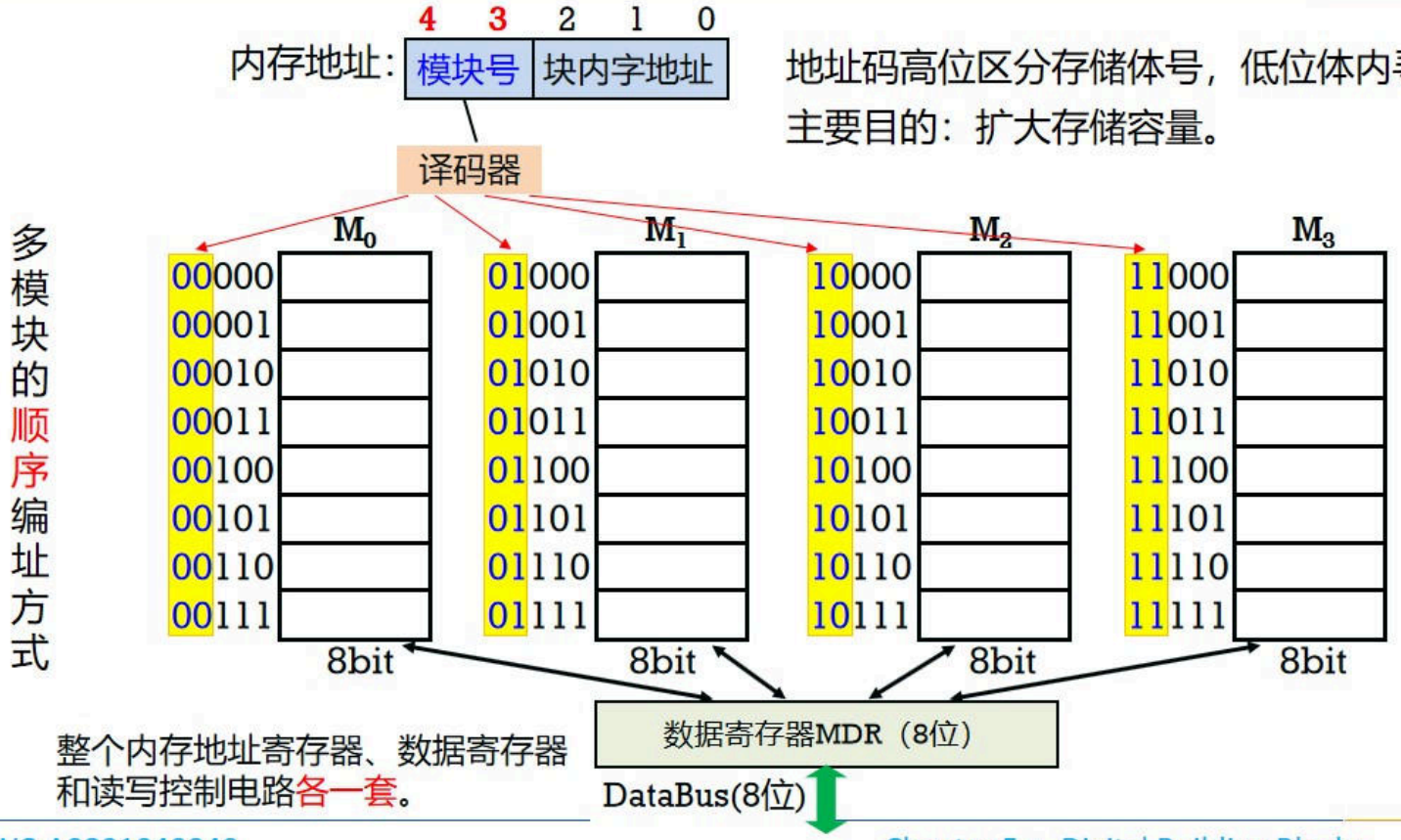
① 顺序方式

- 线性地址逐单元走，依次排完一个模块的所有存储单元，再排下一个模块……；模块地址在高位。

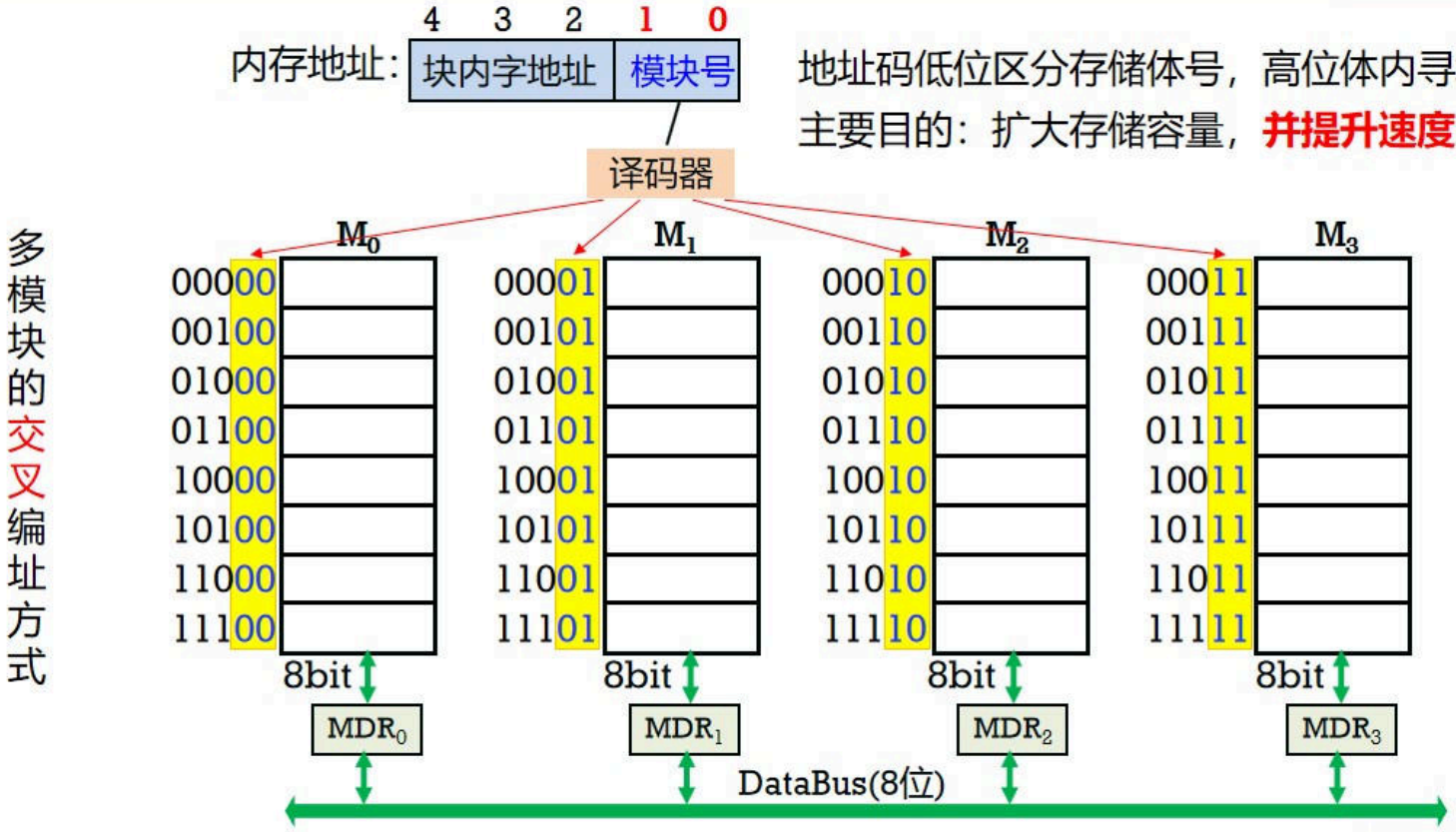
② 交叉方式

- 线性地址逐模块走，所有块依次排完一个存储单元，再回到第一模块……；编址为低位交叉（模块地址在低位）；
- 真正的并行主存系统即所谓多体交叉存储器就是指这种低位交叉方式。

多模块交叉存储器——顺序线性地址的编址方式

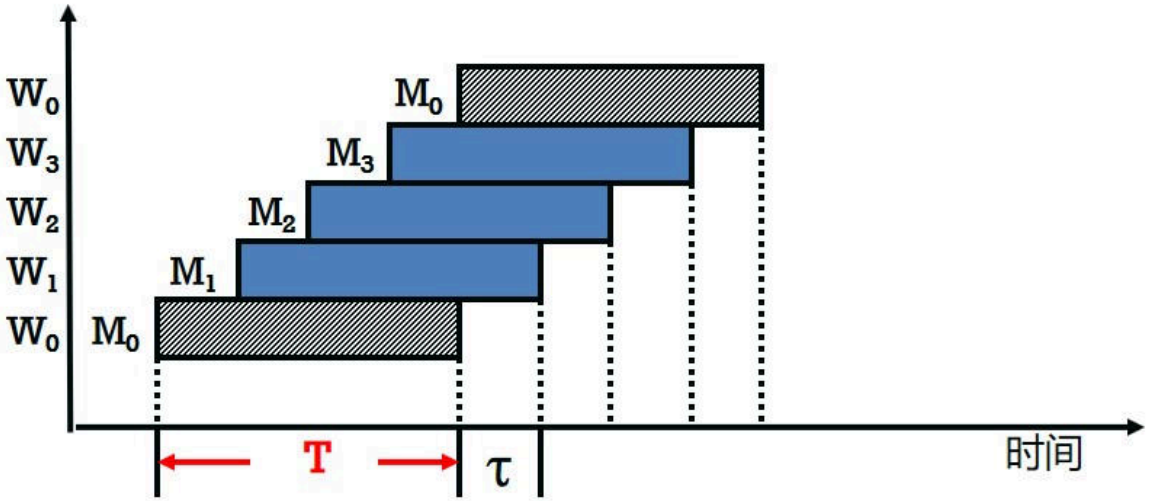


多模块交叉存储器——交叉线性地址的编址方式



多模块交叉存储器

- 例：设有4体交叉的存储器，分时即每隔1/4Tm启动一个分体，见图示。



多体交叉存储器（以4体为例）的分时工作原理示意图

- 可见多体交叉存储器是一种采用流水方式工作的并行存储器系统。

多模块交叉存储器——举例

- 用16M×8位的存储芯片构成一个64M×16位的主存储器。要求既能够扩大存储器的容量，又能够缩短存储器的访问周期（提高访问速度）。

(1)计算需要多少个存储器芯片。

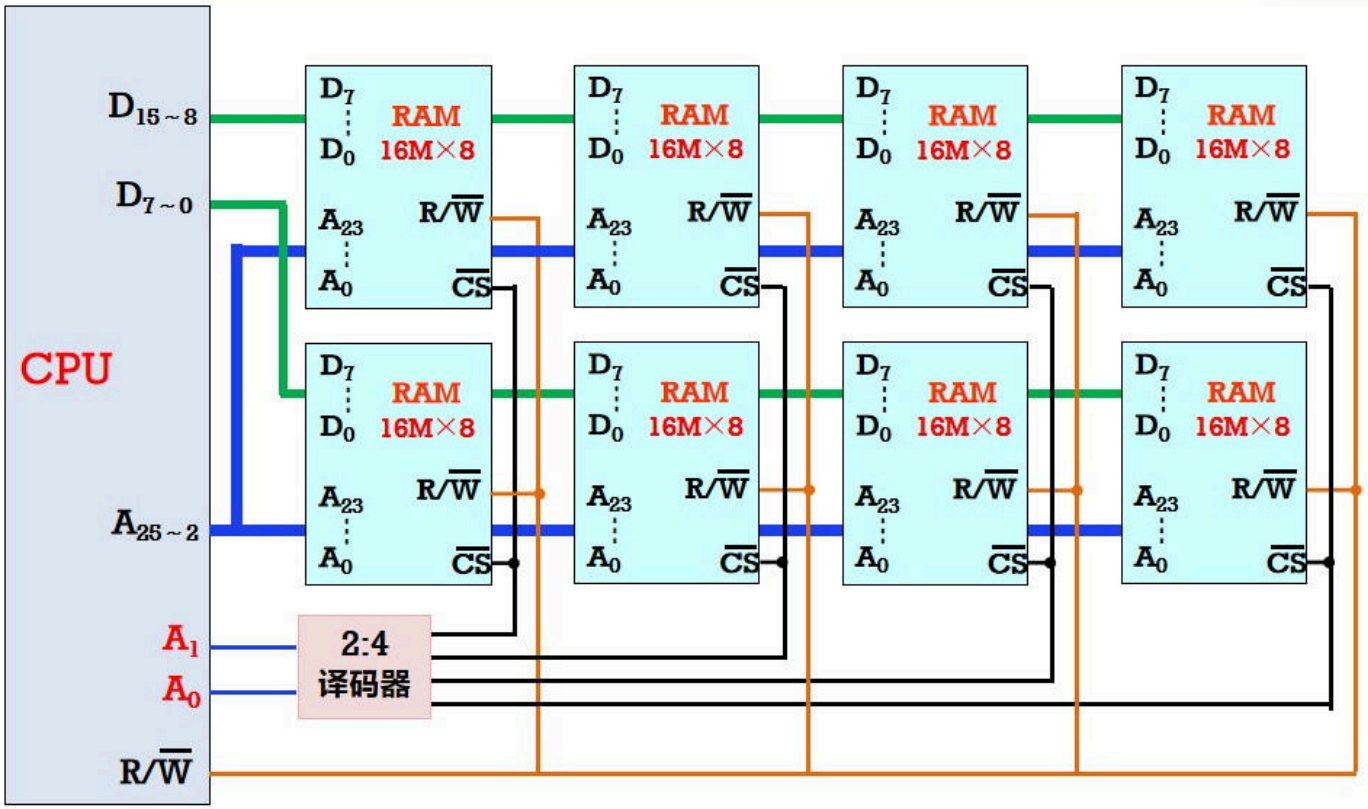
(2)存储器芯片和主存储器的地址长度各需要多少位?

(3)画出用存储器芯片构成主存储器的逻辑示意图。

(4)用16进制表示的地址0x1234567，其模块内地址和模块号是多少?

雨课堂 Rain Classroom

多模块交叉存储器——举例



雨课堂 Rain Classroom

讨论题2：多模块交叉存储器

- 用16M×8位的存储芯片构成一个64M×16位的主存储器。要求既能够扩大存储器的容量，又能够缩短存储器的访问周期（提高访问速度）。

(1)计算需要多少个存储器芯片。

(2)存储器芯片和主存储器的地址长度各需要多少位?

(3)画出用存储器芯片构成主存储器的逻辑示意图。

(4)用16进制表示的地址1234567，其体内地址和体号是多少?

雨课堂 Rain Classroom

第十八堂课结束了

有问题同学们可以提问讨论

雨课堂 Rain Classroom